

**CAN THO UNIVERSITY
COLLEGE OF INFORMATION AND COMMUNICATION TECHNOLOGY**



**GRADUATION THESIS
BACHELOR OF ENGINEERING
IN SOFTWARE ENGINEERING
(HIGH-QUALITY PROGRAM)**

**DEVELOPMENT OF A FASHION
E-COMMERCE SYSTEM
MODULE: MOBILE APPLICATION FEATURING
VIRTUAL FITTING ROOM AND PRODUCT
RECOMMENDATION**

**Student: Vuong Phan Quoc Cuong
Student ID: B2203544
Class: 2022-2026 (K48)**

Can Tho, 8/2026

**CAN THO UNIVERSITY
COLLEGE OF INFORMATION AND COMMUNICATION TECHNOLOGY
DEPARTMENT OF INFORMATION TECHNOLOGY**



**GRADUATION THESIS
BACHELOR OF ENGINEERING IN
SOFTWARE ENGINEERING
(HIGH-QUALITY PROGRAM)**

**DEVELOPMENT OF A FASHION
E-COMMERCE SYSTEM
MODULE: MOBILE APPLICATION FEATURING
VIRTUAL FITTING ROOM AND PRODUCT
RECOMMENDATION**

**Advisor
Bui Vo Quoc Bao, Ph.D.
Truong Minh Thai, Ph.D.**

**Student
Student: Vuong Phan Quoc Cuong
Student ID: B2203544
Class: 2022-2026 (K48)**

Can Tho, 8/2026

ACKNOWLEDGEMENTS

First, I would like to express my sincere gratitude to Can Tho University, especially the College of Information and Communication Technology, for providing a supportive learning environment, facilities, and learning resources throughout my studies and this graduation thesis.

I would like to sincerely thank all lecturers of the College of Information and Communication Technology for their dedicated teaching and valuable guidance. Their unwavering support have provided me with a solid foundation for my academic and professional development.

My deepest gratitude goes to Bui Vo Quoc Bao, Ph.D and Truong Minh Thai, Ph.D for their constructive advice and continuous encouragement throughout my research. Such support has helped me overcome many challenges and successfully complete this thesis.

I would also like to express my sincere gratitude to my thesis partner for the cooperation, support, and shared efforts during this project. Collaborating together has been a valuable experience that contributed significantly to this thesis.

Finally, I would like to express my sincere appreciation to everyone who has supported and encouraged me throughout this journey. I wish you all good health, happiness, and continued success.

Can Tho, /, 20.....
Student
(Signature and full name)

Vuong Phan Quoc Cuong

[illegible]

This image shows a full page of a handwriting practice worksheet. It consists of numerous horizontal rows, each defined by two closely spaced dotted lines. The rows are evenly spaced and extend across the entire width of the page, providing a guide for letter height and placement. There is no text or other markings on the page.

DECLARATION OF RESULTS

I hereby confirm that the thesis titled “Development of a Fashion E-commerce System Module: Mobile Application Featuring Virtual Fitting Room and Product Recommendation” is my own work and has been completed through my own research efforts. The results of this thesis have not been submitted for any other degree at this or any other university.

All sources, references, and figures used in this thesis have been clearly acknowledged and cited. I bear full responsibility for the accuracy, originality, and academic integrity of the presented work.

Can Tho, /, 20.....

Student

(Signature and full name)

Vuong Phan Quoc Cuong

TABLE OF CONTENTS

ACKNOWLEDGEMENTS	i
ADVISER’S COMMENT	ii
REVIEWER’S COMMENT	iii
DECLARATION OF RESULTS	iv
TABLE OF CONTENTS	v
LIST OF TABLES	x
LIST OF FIGURES.....	xii
LIST OF ABBREVIATIONS	xiv
ABSTRACT	xvii
PART 1: INTRODUCTION	1
1. Problem Statement	1
2. Related Work	1
3. Research Objectives	4
3.1. General Objectives	4
3.2. Specific Objectives.....	4
4. Scope And Delimitation.....	4
4.1. Research Subject	4
4.2. Research Scope	5
5. Research Methodology.....	6
5.1. Research Process	6
5.2. Core Technologies Used	6
5.3. Supporting Tools and Software.....	7
6. Thesis Module Assignment.....	7
7. Contribution of the Thesis.....	7
8. Thesis Structure.....	8
PART 2: CONTENT	9
CHAPTER 1. PROBLEM DESCRIPTION.....	9
1.1. Detail Problem Description.....	9
1.1.1. Overall Description	9
1.1.2. Product Functions.....	9
1.1.3. User Classes and Characteristics.....	12

1.2. Functional Requirements	13
1.2.1. Use Case Diagram.....	13
1.2.2. Use Case Specification.....	16
1.3. Non-functional Requirements	49
1.3.1. Performance Requirements	49
1.3.2. Safety Requirements	49
1.3.3. Operational Requirements.....	49
1.3.4. Interface Requirements	50
1.3.5. Security Requirements	50
1.4. Operating Environment.....	50
1.5. Implementation and Design Constraints	51
1.5.1. Implementation Constraints	51
1.5.2. Design Constraints	51
CHAPTER 2. THEORETICAL BACKGROUND.....	52
2.1. Mobile Fashion E-commerce and System Architecture	52
2.2. Product Recommendation Systems.....	52
2.2.1. Recommendation principles.....	52
2.2.2. Signals and evaluation datasets	53
2.2.3. Deployed scoring functions	54
2.2.4. Interpretation and scope	55
2.3. AI-Based Virtual Try-On System	55
2.3.1. Scope and Technical Position of the VTO Module	55
2.3.2. Inputs, Outputs, and Constraints	55
2.3.3. Models and Responsibilities	56
2.4. Software Testing and System Evaluation	57
CHAPTER 3. DESIGN AND IMPLEMENTATION	59
3.1. System Overview	59
3.2. System Architecture	60
3.3. System Decomposition.....	61
3.4. Data Design.....	62
3.5. Functional Design	64
3.5.1. Sign In	64
3.5.2. Sign Up.....	66

3.5.3. View Product List	68
3.5.4. Search Product	70
3.5.5. Filter Products	73
3.5.6. View Product Detail.....	75
3.5.7. Manage Shopping Cart.....	77
3.5.8. Place Order and Make Payment.....	80
3.5.9. View Order History.....	83
3.5.10. Review and Rate Products	85
3.5.11. Contact Support.....	88
3.5.12. Manage Notifications	89
3.5.13. View Recommendations	91
3.5.14. Use Virtual Try-On	93
3.5.15. Manage Personal Information	97
3.5.16. Manage Customer Accounts	100
3.5.17. Manage Roles and Permissions.....	102
3.5.18. Manage Orders	104
3.5.19. Manage Customer Support.....	107
3.5.20. Search & Recommendation Analytics	109
3.5.21. Manage Virtual Try-On	112
3.6. Recommendation System Design	115
3.6.1. Formula selection procedure	115
3.6.2. Online ranking workflow	116
3.7. Virtual Try-On System Design	116
3.7.1. End-to-End Processing Procedure	116
3.7.2. Prompt Construction Method.....	118
3.7.3. Asynchronous Job State and Failure Handling.....	119
CHAPTER 4. TESTING AND EVALUATION	121
4.1. Introduction	121
4.1.1. Testing Objective	121
4.1.2. Testing Scope	121
4.2. Test Plan.....	121
4.2.1. Features To Be Tested.....	121
4.2.2. Features Not To Be Tested.....	122

4.2.3. Testing Approach	122
4.2.4. Pass/Fail Criteria	122
4.2.5. Suspension and Resumption Criteria	122
4.3. Test Management	123
4.3.1. Test Execution Process	123
4.3.2. Test Environment	123
4.3.3. Testing Risks	123
4.4. Test Scenarios	124
4.5. Test Cases	125
4.6. Test Results	125
4.7. Recommendation System Offline Evaluation	125
4.7.1. Evaluation protocol	125
4.7.2. Hyperparameter selection	125
4.7.3. Comparison results	126
4.7.4. Validity limits	127
PART 3: CONCLUSION	128
1. Achieved Results	128
1.1. Theoretical Contributions	128
1.2. Software Implementation Results	129
1.3. Practical Applications	129
2. Limitation	130
3. Future Development Direction	130
REFERENCES	131
APPENDIX A. REPRESENTATIVE TEST CASES	133
APPENDIX B. DETAILED TEST EXECUTION RESULTS	137
APPENDIX C. DATABASE COLLECTION DATA DICTIONARY	138
APPENDIX D. VIRTUAL TRY-ON PROMPT CONFIGURATION	146
D.1 Assembly Order and Placeholders	146
D.2 Exact ComfyUI 2x2 Image Wrapper	146
D.3 Exact Source and Framing Block	147
D.4 Exact Garment Description and Role Blocks	148
D.5 Exact Context Blocks	150
D.6 Exact Garment-Fidelity and Realism Blocks	152

D.7 Exact Negative Prompt.....	152
D.8 Fully Expanded Example: Full-Body Top + Bottom, Casual Context	153
D.9 Exact Video Prompt	154
APPENDIX E. RECOMMENDATION DATA, BASELINES, AND EVALUATION EQUATIONS.....	156
E.1 Datasets and Reproducibility Links.....	156
E.2 Traditional Baseline Formulas.....	156
E.3 Evaluation Equations	157
E.4 Auxiliary Paired Significance Test.....	158
E.5 Recommendation Symbol Glossary	159

LIST OF TABLES

Table 0.1 Thesis Module Responsibilities	7
Table 1.1 Sign In use case specification	16
Table 1.2 Sign Up use case specification.....	18
Table 1.3 View Product List use case specification.....	20
Table 1.4 Search Product use case specification.....	21
Table 1.5 Filter Products use case specification	22
Table 1.6 View Product Detail use case specification	24
Table 1.7 Add Favorite use case specification.....	25
Table 1.8 Manage Shopping Cart use case specification.....	27
Table 1.9 Place Order and Make Payment use case specification	28
Table 1.10 View Order History use case specification.....	30
Table 1.11 Review and Rate Products use case specification	32
Table 1.12 Contact Support use case specification.....	33
Table 1.13 View Recommendations use case specification	35
Table 1.14 Use Virtual Try-On use case specification	36
Table 1.15 Manage Personal Information use case specification	38
Table 1.16 Manage Customer Accounts use case specification	40
Table 1.17 Manage Roles and Permissions use case specification.....	41
Table 1.18 Manage Orders use case specification	42
Table 1.19 Manage Customer Support use case specification.....	44
Table 1.20 Search & Recommendation Analytics use case specification	46
Table 1.21 Manage Virtual Try-On use case specification.....	47
Table 2.1 Base interaction weights for recommendation events	53
Table 2.2 Runtime inputs and outputs of the VTO pipeline	56
Table 2.3 Models and techniques used by the implemented system	56
Table 3.1 Main interface components for sign-in.....	64
Table 3.2 Data used by sign-in.....	65
Table 3.3 Main interface components for sign up	67
Table 3.4 Data used by sign up	67
Table 3.5 Main interface components for viewing product list	69
Table 3.6 Data used by view product list.....	70
Table 3.7 Main interface components for search product.....	71
Table 3.8 Data used by search product	72
Table 3.9 Main interface components for filter products.....	74
Table 3.10 Data used by filter products	74
Table 3.11 Main interface components for viewing product details	76
Table 3.12 Data used by view product detail.....	76
Table 3.13 Main interface components for managing the shopping cart.....	78
Table 3.14 Data used by manage shopping cart.....	79
Table 3.15 Main interface components for place order and make payment.....	82
Table 3.16 Data used by place order and make payment.....	82
Table 3.17 Main interface components for view order history	84
Table 3.18 Data used by view order history	84

Table 3.19 Main interface components for reviewing and rating products	86
Table 3.20 Data used by review and rate products	86
Table 3.21 Main interface components for contact support	88
Table 3.22 Data used by contact support	89
Table 3.23 Main interface components for managing notifications	90
Table 3.24 Data used by manage notifications	91
Table 3.25 Main interface components for view recommendations.....	92
Table 3.26 Data used by view recommendations	92
Table 3.27 Main interface components for use in virtual try-on	95
Table 3.28 Data used by virtual try-on	95
Table 3.29 Main interface components for managing personal information.....	98
Table 3.30 Data used by manage personal information.....	99
Table 3.31 Main interface components for managing customer accounts.....	101
Table 3.32 Data used by manage customer accounts.....	101
Table 3.33 Main interface components for managing roles and permissions.....	103
Table 3.34 Data used by manage roles and permissions.....	103
Table 3.35 Main interface components for manage orders.....	106
Table 3.36 Data used by manage orders	106
Table 3.37 Main interface components for managing customer support.....	108
Table 3.38 Data used by manage customer support.....	108
Table 3.39 Main interface components for Search & Recommendation Analytics	111
Table 3.40 Data used by Search & Recommendation Analytics	111
Table 3.41 Main interface components for managing virtual try-on	113
Table 3.42 Data used by manage virtual try-on	113
Table 3.43 Persisted VTO state model.....	119
Table 4.1 Test Scenario Summary	124
Table 4.2 Offline recommendation evaluation design	125
Table 4.3 Grid-search audit and selected configurations	126
Table 4.4 Locked final recommendation results.....	126
Table A.1 Representative Test Cases.....	133
Table B.1 Automated Test Execution Results	137
Table C.1 Data dictionary for the users collection.....	138
Table C.2 Data dictionary for the categories collection.....	138
Table C.3 Data dictionary for the brands collection	139
Table C.4 Data dictionary for the products collection	139
Table C.5 Data dictionary for the inventories collection	140
Table C.6 Data dictionary for the inventoryreservations collection	140
Table C.7 Data dictionary for the imports collection.....	140
Table C.8 Data dictionary for the inventoryreceipts collection	141
Table C.9 Data dictionary for the carts collection	141
Table C.10 Data dictionary for the orders collection.....	142
Table C.11 Data dictionary for the transactions collection.....	143
Table C.12 Data dictionary for the reviews collection	143
Table C.13 Data dictionary for the virtualtryonjobs collection	144

Table E.1 Recommendation symbols and notation.....	159
--	-----

LIST OF FIGURES

Figure 0.1 ASOS mobile application interface	2
Figure 0.2 DressX mobile application interface	3
Figure 0.3 YODY mobile application interface.....	3
Figure 0.4 CANIFA mobile application interface.	4
Figure 1.1 Customer use case diagram	14
Figure 1.2 Admin and staff use case diagram.....	15
Figure 1.3 Guest use case diagram.....	16
Figure 1.4 Sign-in use case diagram	16
Figure 1.5 Sign-up use case diagram	18
Figure 1.6 Product listing, filtering and searching use case diagram	20
Figure 1.7 Product details and favorites use case diagram	24
Figure 1.8 Shopping cart use case diagram.....	27
Figure 1.9 Checkout and payment use case diagram	28
Figure 1.10 Order history use case diagram	30
Figure 1.11 Product review and rating use case diagram	31
Figure 1.12 Customer support use case diagram	33
Figure 1.13 Product recommendations use case diagram.....	35
Figure 1.14 Virtual try-on use case diagram.....	36
Figure 1.15 Manage personal information use case diagram.....	38
Figure 1.16 Customer account management use case diagram	40
Figure 1.17 Role and permission management use case diagram	41
Figure 1.18 Order management use case diagram	42
Figure 1.19 Customer support management use case diagram.....	44
Figure 1.20 Search and recommendation analytics use case diagram	46
Figure 1.21 Virtual try-on management use case diagram	47
Figure 2.1 Conceptual architecture of the context-aware hybrid ranker.	53
Figure 3.1 Overall system architecture	60
Figure 3.2 Decomposition diagram.....	62
Figure 3.3 MongoDB collection relationship diagram	63
Figure 3.4 Mobile sign-in interface.....	64
Figure 3.5 Mobile sign-in processing flow	66
Figure 3.6 Mobile account-registration interface.....	67
Figure 3.7 Mobile account-registration processing flow	68
Figure 3.8 Mobile product-discovery and product-list interface	69
Figure 3.9 Mobile view product list flow	70
Figure 3.10 Mobile product-search suggestions and results interface	71
Figure 3.11 Mobile product-search and result-navigation flow.....	73
Figure 3.12 Mobile product-filter interface	74
Figure 3.13 Mobile Product Filtering and Retrieval Flow.....	75
Figure 3.14 Mobile product-detail interface	76

Figure 3.15 Mobile product-detail retrieval flow.....	77
Figure 3.16 Mobile add-to-favorites flow.....	77
Figure 3.17 Mobile shopping-cart management interface	78
Figure 3.18 Mobile shopping-cart update flow.....	80
Figure 3.19 Mobile checkout and payment interface.....	81
Figure 3.20 Mobile order-success and order-detail interface	81
Figure 3.21 Mobile checkout and payment processing flow	83
Figure 3.22 Mobile order-history interface	84
Figure 3.23 Mobile product-review and rating interface	85
Figure 3.24 Mobile product-review submission flow	87
Figure 3.25 Mobile review-list and review-submission flow	87
Figure 3.26 Mobile customer-support interface.....	88
Figure 3.27 Mobile customer-support FAQ and ticket flow.....	89
Figure 3.28 Mobile notification-management interface	90
Figure 3.29 Mobile notification viewing and settings flow	91
Figure 3.30 Mobile recommendation interfaces on home, product detail, and cart ..	92
Figure 3.31 Mobile recommendation display flow by context	93
Figure 3.32 Mobile virtual try-on source-selection and history interface	94
Figure 3.33 Mobile virtual try-on outfit-configuration interface	94
Figure 3.34 Mobile virtual try-on processing and result interface.....	95
Figure 3.35 Mobile virtual try-on orchestration flow	97
Figure 3.36 Mobile personal-information profile, address, and security interface...	98
Figure 3.37 Mobile profile-update validation flow.....	100
Figure 3.38 Admin customer-account management interface	101
Figure 3.39 Admin customer-account management flow	102
Figure 3.40 Admin role and permission management interface	103
Figure 3.41 Admin staff-account and permission management flow	104
Figure 3.42 Admin order lookup and invoice interface	105
Figure 3.43 Admin order operations interface	105
Figure 3.44 Admin order-management flow.....	107
Figure 3.45 Admin customer-support management interface.....	108
Figure 3.46 Admin customer-support management flow	109
Figure 3.47 Admin search analytics interface.....	110
Figure 3.48 Admin recommendation analytics interface	110
Figure 3.49 Admin search and recommendation analytics flow.....	112
Figure 3.50 Admin virtual try-on management interface	113
Figure 3.51 Admin virtual try-on operations and settings flow	114
Figure 3.52 Data flow from event tracking to ranked recommendations.	116
Figure 3.53 AI virtual try-on processing and job-state flow.....	118
Figure 4.1 Comparison of recommendation formulas for the Home context	127
Figure 4.2 Comparison of recommendation formulas for Product Detail context ..	127
Figure 4.3 Comparison of recommendation formulas for the Cart context.....	127

LIST OF ABBREVIATIONS

No	Term/Abbreviation	Definition/Explanation
1	ADM	Administration; identifier prefix for the administration module
2	AI	Artificial Intelligence
3	AL	Alternate Flow; identifier prefix in use-case specifications
4	API	Application Programming Interface
5	Bi-LSTM	Bidirectional Long Short-Term Memory; neural sequence model referenced for fashion compatibility
6	COD	Cash on Delivery
7	CSS3	Cascading Style Sheets Level 3
8	CSV	Comma-Separated Values
9	CTR	Click-Through Rate
10	DINO	Grounding DINO; open-set object-detection model used for garment localization
11	EX	Exception Flow; identifier prefix in use-case specifications
12	FAQ	Frequently Asked Questions
13	FCM	Firebase Cloud Messaging
14	GHN	Giao Hàng Nhanh; shipping and delivery service provider
15	GPU	Graphics Processing Unit
16	HTML5	HyperText Markup Language 5
17	HTTP	Hypertext Transfer Protocol
18	ID	Identifier
19	IDE	Integrated Development Environment
20	iOS	Apple's mobile operating system
21	JPEG/JPG	Joint Photographic Experts Group image format
22	JSON	JavaScript Object Notation

No	Term/Abbreviation	Definition/Explanation
23	KPI	Key Performance Indicator
24	MB	Megabyte
25	MOB	Mobile; identifier prefix for the mobile application module
26	OpenCV	Open Source Computer Vision Library
27	ORM	Object-Relational Mapping
28	OTP	One-Time Password
29	P95	95th percentile; a performance-latency reporting measure
30	PNG	Portable Network Graphics
31	RBAC	Role-Based Access Control
32	REST	Representational State Transfer
33	SAM	Segment Anything Model
34	SDK	Software Development Kit
35	SKU	Stock Keeping Unit
36	SMS	Short Message Service
37	SRS	Software Requirements Specification
38	TC	Test Case; identifier prefix in the test-results appendix
39	TS	Test Scenario; identifier prefix in the test-results appendix
40	UC	Use Case; identifier prefix in functional specifications
41	UI/UX	User Interface / User Experience
42	URL	Uniform Resource Locator
43	VITON	Virtual Try-On Network; image-based virtual try-on model
44	VITON-HD	High-resolution VITON model for virtual try-on
45	VNPay	Vietnamese online payment gateway integrated by the system

No	Term/Abbreviation	Definition/Explanation
46	VTO	Virtual Try-On; digital visualization of garments on a person image
47	YOLO/YOLO11	You Only Look Once; YOLO11 pose-estimation model used by the systems

ABSTRACT

This thesis presents the development of a modern Android-oriented fashion e-commerce system using React Native, Expo, TypeScript, React, Node.js, Express, MongoDB, and supporting AI microservices. Beyond the core shopping workflow, the system integrates a multi-context product recommendation engine and an interactive Virtual Fitting Room. Redis is used as a shared operational data store for short-lived cache entries and atomic rate-limit counters. In contrast, BullMQ uses a dedicated Redis database to coordinate asynchronous image and video generation jobs. Socket.IO remains a separate real-time delivery channel, with REST polling used as a fallback. The application services are packaged with Docker Compose to provide a repeatable local deployment environment for the Web Administration Portal, backend API, image-validation service, and garment-processing service.

The completed system supports the customer shopping journey, contextual product recommendations, virtual try-on processing, operational administration, resilient caching, and background job execution. The recommendation algorithm was evaluated offline on controlled Amazon Fashion and Polyvore datasets using ranking, coverage, diversity, and paired significance measures. Automated verification covered backend, mobile, Web Administration, end-to-end, image-validation, and garment-processing behavior. The results demonstrate the feasibility of combining mobile commerce, explainable recommendation logic, computer vision, generative AI, distributed caching, and Redis-backed job processing within one modular fashion retail platform. The reported evaluation remains an engineering and offline assessment rather than evidence of commercial uplift under production traffic.

PART 1: INTRODUCTION

1. Problem Statement

In recent years, e-commerce has grown rapidly and plays an important role in the fashion industry, allowing consumers to search, compare, and buy products without location or time limitations. Mobile applications offer clear advantages by using smartphone features such as cameras, push notifications, and user interaction data to provide a more convenient, smooth shopping experience. As mobile devices become the main channel for online shopping, delivering an intuitive, visual, and personalized user experience has become essential for fashion retail.

Despite the popularity of mobile shopping, current fashion e-commerce applications still have several key limitations for customers. First, users often struggle to find items that match their personal style in large product catalogs when search keywords are not specific. Second, standard popular product lists often do not match personal tastes or suggest suitable items to complete an outfit. Third, users find it difficult to see how a garment or outfit combination would look on their own body before buying, making it harder to decide.

From a technical perspective, solving these problems requires building an efficient client-server system that connects the mobile application with backend APIs, database management, cloud storage, real-time messaging, and specialized AI services. The mobile application must handle complex integration requirements, including system stability, fast response times, asynchronous processing for heavy image generation tasks, and reliable fallback options when network connections are interrupted.

Recent developments in recommendation systems, computer vision, and generative image models provide effective ways to solve these challenges. User interaction data captured on mobile devices can be analyzed to rank items and recommend personalized or complementary fashion products. At the same time, machine learning models can process user photos and garment images to create realistic try-on images, allowing customers to preview outfits interactively on their smartphones.[1], [2]

Therefore, this thesis focuses on developing a mobile fashion e-commerce application integrated with a personalized product recommendation system and a virtual try-on room. The proposed system provides a complete mobile shopping workflow while solving two main problems in fashion e-commerce: personalized product selection and outfit visualization before purchase.

2. Related Work

Currently, the project "Development of a Fashion E-commerce System Module: Mobile Application Featuring Virtual Fitting Room and Product Recommendation" has few direct previous studies that combine both features into one mobile app. Therefore, looking at real-world mobile fashion shopping apps is

necessary to build and complete this project. These references provide a good foundation to help guide and develop the system effectively.

International Scope:

ASOS Mobile App: is a popular international mobile fashion shopping app based in the United Kingdom. With a wide range of products, the app is known for its easy-to-use mobile interface and smart product recommendation system based on user browsing history and shopping habits. [3]

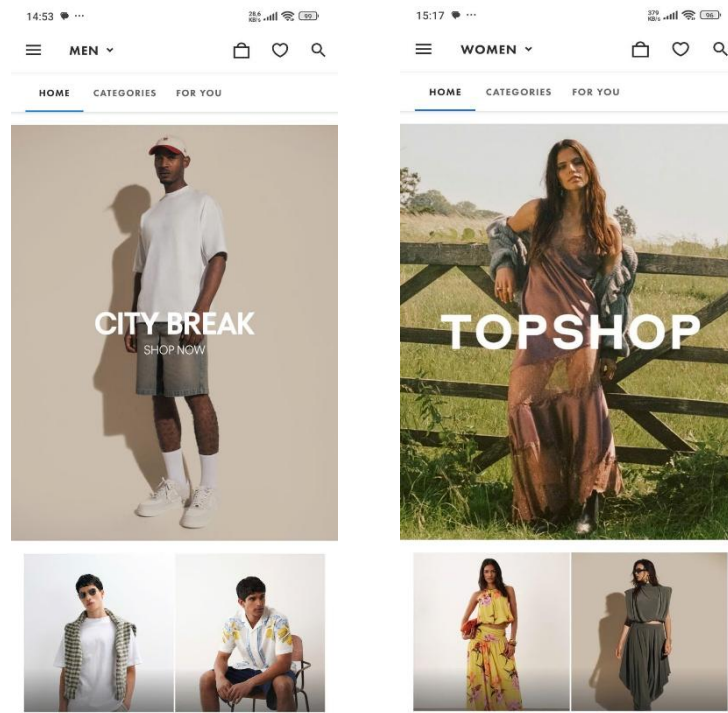


Figure 0.1 ASOS mobile application interface

DressX Mobile App: is a leading international mobile application specializing in digital fashion and virtual try-on technology. The app allows users to upload personal photos to virtually try on digital clothing, helping customers visualize outfits and styles directly on their smartphones. [4]

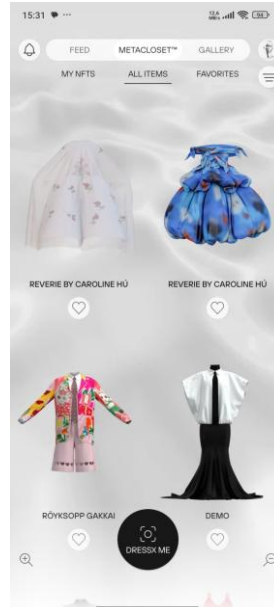


Figure 0.2 DressX mobile application interface

Domestic Scope:

YODY Mobile App is the official mobile shopping app of the YODY fashion brand in Vietnam. This mobile app supports clothes search, filters by size and color, and related item recommendations for users. Notably, YODY's product catalog is used as a sample dataset for this thesis. [5]

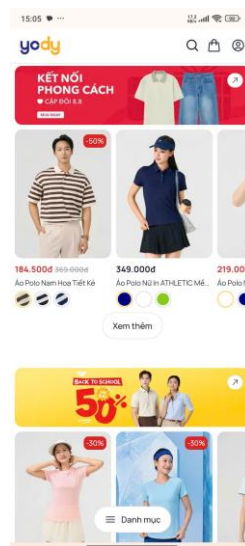


Figure 0.3 YODY mobile application interface.

CANIFA Mobile App is the official mobile app of the CANIFA fashion brand in Vietnam. This app offers a wide variety of useful functions for online shopping, including smart product filtering and suggestions for related items. [6]



Figure 0.4 CANIFA mobile application interface.

3. Research Objectives

3.1. General Objectives

To develop a complete fashion e-commerce system featuring an Android mobile application for customers and a Web Administration system for administrators and staff. In addition to essential e-commerce functions, the system integrates two core intelligent features: personalized product recommendations and an interactive virtual try-on room to enhance user experience and support purchasing decisions.

3.2. Specific Objectives

- Develop an Android-focused mobile fashion e-commerce application supporting account management, product discovery, and online shopping.
- Implement the core purchasing workflow, including cart management, payment, delivery, and order tracking.
- Develop a product recommendation feature based on user interactions, product information, and outfit-matching rules.
- Develop a virtual try-on feature that lets users upload images, select garments, generate try-on results, and save or purchase selected outfits.
- Integrate the mobile application with backend services, databases, third-party services, and AI-based image-processing workflows.

4. Scope And Delimitation

4.1. Research Subject

From a business perspective, the study focuses on developing and operating a mobile fashion e-commerce application. The application allows customers to search

for products, place orders, track order status, and use the virtual try-on feature. Administrator and staffs can use the web administration system to manage products, orders, and related information.

In terms of architecture and technology, the mobile application is built with React Native, Expo, and TypeScript, primarily for Android devices. RESTful APIs exchange data between the mobile application, backend server, and AI services. Socket.IO supports real-time updates for long-running processes.

The system integrates Cloudinary for image storage and external services for payment processing and shipping fee calculation. Redis supports caching, rate limiting, and BullMQ-based virtual try-on queues, while MongoDB remains the primary database. Socket.IO provides real-time job updates, with REST polling as a fallback.

For product recommendations, the system uses product attributes, user interactions, popular products on the market, and basic outfit-matching rules. Then, the system suggests suitable products on the home page, product detail page, and shopping cart.

For virtual try-on, the system uses OpenCV, YOLO11 Pose, Grounding DINO, and SAM 2.1 to analyze and process user and garment images. The processed images are then used in ComfyUI workflows to generate virtual try-on images and optional videos.

4.2. Research Scope

The theoretical scope of this study covers the foundational concepts required to develop a mobile fashion e-commerce system. It examines mobile shopping workflows, user interaction behaviors, and mobile UI/UX design principles. The study also explores client-server architecture, RESTful APIs, and Socket.IO for real-time event handling to enable communication between the mobile application and backend services. In addition, it investigates recommendation methods based on implicit user feedback, preference profiling, and complementary outfit rules. It also examines computer vision and generative AI techniques, particularly garment localization, body keypoint estimation, promptable image segmentation, and image-based virtual try-on synthesis. Through these disciplines, the study aims to improve practical skills in React Native development, asynchronous job processing, and AI microservice integration.

The technical scope includes a customer-facing mobile application, a web-based administration portal, backend services, a database, and AI microservices. The mobile application is built with React Native, Expo, and TypeScript, with primary testing on Android devices. Administrators and staff use the existing web portal to manage catalog data and orders and to monitor system logs. Server-side services are implemented with Node.js and Express.js using a RESTful API architecture. MongoDB stores user profiles, products, variants, shopping carts, orders, interaction events, recommendation data, and virtual try-on job records.

For AI-powered functionality, the system integrates Python microservices using OpenCV, YOLO11 Pose, Grounding DINO, SAM 2.1, and ComfyUI. These technologies support image validation, garment processing, body and clothing analysis, and virtual try-on generation. The system also integrates several third-party services, including Cloudinary for image storage, VNPay for online payments, GHN for shipping, Socket.IO for real-time progress updates, and Firebase for mobile push notifications.

The technical scope also includes Redis-based caching, rate limiting, BullMQ job queues, service health monitoring, graceful shutdown, and Docker Compose deployment. The current containers cover the web portal, backend API, and AI services, while MongoDB Atlas, Redis Cloud, Cloudinary, VNPay, GHN, Firebase, and ComfyUI remain externally managed services.

The project does not include training a new deep learning model from scratch or deploying the system at a large commercial scale. Instead, it focuses on integrating and applying existing technologies within a small-scale, Android-focused mobile fashion retail system.

5. Research Methodology

5.1. Research Process

The research process is conducted systematically, beginning with an investigation of the operational processes and workflow requirements of mobile fashion e-commerce platforms. Based on this foundation, the essential features of the customer-facing mobile application and the web-based administration system are identified. The study then explores personalized product recommendation mechanisms, outfit-matching logic, computer vision techniques, and image-generation workflows to support the virtual fitting room functionality. These findings form the basis for software requirements analysis and the development of the Software Requirements Specification (SRS). Subsequently, the system architecture, database schema, API structure, and user interfaces are designed, and the program modules are implemented incrementally according to the defined functional requirements. Finally, unit, integration, and mobile device testing are conducted to evaluate the system's functionality and reliability, followed by documenting system specifications and completing the final thesis report.

5.2. Core Technologies Used

- Mobile Frontend: React Native, Expo SDK 54, TypeScript, React Navigation, React Native Reanimated.
- Web Admin: React.js, JavaScript, HTML5, CSS3.
- Backend: Node.js, Express.js (RESTful APIs), Redis, BullMQ.
- Database: MongoDB (Mongoose ORM), Redis Cloud (caching, rate limiting, and BullMQ queues).

- AI & Image Processing Microservices: Python, FastAPI, OpenCV, Ultralytics YOLO11 pose, Grounding DINO Tiny, SAM 2.1, ComfyUI workflows.
- Infrastructure and Deployment: Docker, Docker Compose.
- Third-Party Services: Cloudinary (cloud image storage), Firebase (Firebase Cloud Messaging - FCM for push notifications), Socket.IO (real-time messaging).

5.3. Supporting Tools and Software

- System Design Tools: Figma, Draw.io.
- Development & IDE Tools: Visual Studio Code, PyCharm.
- Container & Environment Tools: Docker Desktop.
- Database & Cloud Management Tools: MongoDB Compass, Cloudinary Dashboard, Firebase Console.
- Testing & Execution Tools: Postman, Jest, jest-expo, pytest, Android Studio (Emulator), Physical Android devices.
- Version Control Tools: Git, GitHub.
- Documentation Tools: Microsoft Word 2016, Google Docs.

6. Thesis Module Assignment

Table 0.1 Thesis Module Responsibilities

Team Member	Main Platform	Web Admin Responsibilities	Advanced Feature
Vuong Phan Quoc Cuong	Mobile	Manage Customer Accounts Manage Roles and Permissions Manage Orders Manage Customer Support Search & Recommendation Analytics Manage Virtual Try-On	Product Recommendation System Virtual Dressing Room
Nguyen Phat Dat	Web	Manage Vouchers Manage Membership Manage Brands Manage Categories Manage Products Manage Product Inventory Manage Reviews and Ratings	Image-Based Product Search

7. Contribution of the Thesis

- Social and User Aspect: Developing a mobile fashion e-commerce application with personalized recommendations and a virtual fitting room delivers significant value to consumers and the community. The application offers a

more convenient, user-friendly mobile platform that simplifies product discovery and lets users preview clothing choices directly on their own photos. This interactive experience reduces uncertainty when purchasing fashion items online, allowing users to select clothing fitting their body shape and personal style, which enhances overall shopping confidence. Furthermore, the project serves as a practical academic reference for applying mobile technologies and generative artificial intelligence to solve real-world retail challenges.

- **Economic and Commercial Aspect:** The system offers fashion businesses an innovative mobile sales channel that drives digital transformation. Personalized recommendations boost engagement, increase average order values, and improve sales. Customers can visualize outfits before checkout, reducing returns and operational costs. The system captures user data and try-on analytics, helping managers make data-driven decisions on inventory, marketing, and product development.

8. Thesis Structure

This document provides detailed information about the developed system. The document consists of 03 parts: Introduction, Content, and Conclusion.

- Part 1: Introduction:

Presents the background leading to the thesis, clearly states the thesis objectives, and identifies the main problems to be solved.

- Part 2: Content:

- **Chapter 1:** Describes the overview of features and system requirements, including functional and non-functional requirements of the system.
- **Chapter 2:** Presents the theoretical background of React Native, Expo, product recommendation algorithms, and computer vision models for virtual try-on.
- **Chapter 3:** Design and solution implementation. Presents project components including data schemas, interface mockups, and thesis results: completed system interfaces.
- **Chapter 4:** Presents testing results and result evaluation.

- Part 3: Conclusion:

Presents the results achieved after completing the thesis. States the accomplishments, limitations, and future development directions for the system.

References: Notes all documents referenced during the thesis, including websites, lectures, books, and scientific papers.

Appendix: Presents representative test cases and detailed test execution results, the database collection data dictionary, the exact Virtual Try-On prompt configuration, and recommendation datasets, baselines, and evaluation equations.

PART 2: CONTENT

CHAPTER 1. PROBLEM DESCRIPTION

1.1. Detail Problem Description

1.1.1. Overall Description

The thesis project, titled "Mobile Application Featuring Virtual Fitting Room and Product Recommendation," aims to build an online fashion shopping platform that effectively connects fashion businesses with retail customers. The system consists of two major subsystems: a mobile application for end users and a web administration panel for store administrators and staff.

For fashion businesses, the system provides digital tools to display, manage, and sell products online, reducing dependence on physical stores and expanding market reach. For customers, the mobile application offers an intuitive and interactive shopping experience directly on their smartphones.

A key innovation of the proposed system is the integration of a Virtual Fitting Room and an intelligent Product Recommendation System. The Virtual Fitting Room enables customers to select, mix, and match fashion items such as tops, bottoms, shoes, and accessories on an interactive digital canvas to build and preview complete outfits before deciding to purchase. Meanwhile, the Product Recommendation System analyzes user preferences, browsing history, and item compatibility to suggest personalized products and complementary styling options, improving product discovery and enhancing customer satisfaction.

The platform provides various functions for each role. Users can browse the active catalog, search and filter products, inspect product details, view supported policies, and receive non-personalized discovery content. Registration and sign-in are required for account-specific operations such as favorites, cart persistence, checkout, order tracking, support history, membership, notifications, personalized recommendations, and virtual try-on jobs.

The Web Admin Panel supports permission-scoped store operations. Administrators and authorized staff manage customer and internal accounts, products and catalog references, inventory, orders, payments, shipping, promotions, loyalty, reviews, support, storefront information, reporting, and virtual try-on operations. Analytics monitor search and recommendation behavior rather than an undocumented rule editor.

1.1.2. Product Functions

The fashion e-commerce system provides core functionalities to meet the needs of unregistered customers, registered customers, administrators, and staff across the Mobile Application and Web Admin Panel. These functions support product discovery, virtual outfit creation, personalized recommendations, online purchasing, order processing, inventory management, and system administration, ensuring that shopping and store management processes are convenient, secure, and efficient.

Unregistered and Registered Customer Functions (Mobile Application):

- Sign Up: Enables new customers to create an account using personal information and a password.
- Sign In: Provides secure access for registered users via email or phone number and password.
- Log Out: Allows registered customers to end the current session safely and return to guest mode.
- View Product List: Enables customers to browse the catalog of available fashion products in the mobile application.
- View Product Detail: Displays current product images, variants, prices, stock, descriptions, reviews, ratings, and similar-product recommendations.
- Search Product: Allows customers to search active products by keyword, use suggestions and history, and open paginated results.
- Filter Products: Helps users filter product listings based on specific criteria such as category, brand, price, color, and size.
- Use Virtual Try-On: Allows customers to select a personal photo and valid garment variants, generate try-on images and optional video, view try-on history, and add result items to the cart.
- View Recommendations: Displays personalized Home recommendations, similar products on Product Detail, and complementary products in Cart.
- Manage Favorite Products: Allows customers to add, search, view, and remove favorite products.
- Manage Shopping Cart: Helps customers add single products or complete outfits to the cart, update quantities, change sizes or colors, select items for checkout, and remove unwanted items.
- View Coupons / Vouchers: Allows customers to view eligible vouchers and validate or apply a voucher during checkout.
- Manage Payment Method: Allows customers to view and manage stored bank accounts used for eligible refund flows and choose a default verified account.
- Place Order and Make Payment: Allows registered customers to enter delivery information, preview totals, apply vouchers, create an order, and complete COD or VNPay payment.
- Track Orders: Allows customers to view real-time delivery status of their orders.
- View Order History: Allows customers to review past orders, payment details, shipping information, and purchased items.

- Request Return: Allows customers to submit a return request for eligible orders with a reason and photos.
- Review and Rate Products: Allows eligible customers to rate and review purchased products, upload review images, edit or delete their reviews, and mark public reviews as helpful.
- Contact Support: Allows customers to create support tickets, attach evidence, exchange messages, and track ticket status.
- View Application Policies: Allows users to read policies related to privacy, payment, shipping, returns, and terms of use.
- Manage Personal Information: Allows signed-in customers to update name, phone, gender, date of birth, avatar, delivery addresses, and password; the registered email is displayed read-only.
- View Membership Information: Allows customers to view their current tier, point balance, progress to the next tier, and tier benefits.
- Manage Notifications: Allows customers to view and mark notifications as read, receive push alerts, and configure supported notification preferences.

Administrator and Staff Functions (Web Admin):

- Manage Customer Accounts: Enables authorized staff to view customer profiles, block or unblock accounts, and review customer order history.
- Manage Roles and Permissions: Enables administrators to assign roles and permissions to staff members, controlling access to admin features.
- Manage Categories: Supports creating, updating, viewing, hiding, or deleting product categories.
- Manage Products: Enables authorized staff to add, update, view, or hide products, including images, colors, sizes, prices, descriptions, and stock availability.
- Manage Brands: Supports creating, updating, viewing, hiding, or deleting product brands.
- Manage Inventory: Enables authorized staff to view stock, monitor low-stock items, create and confirm import receipts, and adjust quantities with recorded reasons.
- Manage Orders: Allows staff to view, search, confirm, process, update status, cancel, and track customer orders.
- Manage Virtual Try-On: Enables authorized staff to monitor try-on requests and media, retry, cancel, or hide eligible jobs, configure limits, manage prompt-policy rules and violations, and lock or unlock customer try-on access.

- Search & Recommendation Analytics: Allows authorized staff to inspect customer search demand, zero-result gaps, recommendation funnels, attribution, coverage, diversity, algorithm/context segments, and top products.
- Manage Promotions: Supports voucher and campaign creation, updates, duplication, scheduling, status control, eligibility/scope configuration, usage limits, and analytics.
- Manage Loyalty and Membership: Allows authorized staff to manage membership tiers, tier benefits, point-earning rules, customer balances, manual adjustments, and point history.
- Manage Customer Support: Enables staff to receive, respond to, assign, track, and close customer support tickets and complaints.
- Manage Reviews and Ratings: Allows authorized staff to check, approve, reject, reply to, or remove customer product reviews.
- Manage Storefront Settings: Allows administrators to configure public store identity, avatar, legal/contact information, support hours, address, map link, and social links.
- View Reports and Statistics: Provides permission-scoped dashboard insights into paid sales, orders, customers, top products, order health, and inventory risk.

1.1.3. User Classes and Characteristics

Customers (Registered Users)

- Characteristics: Registered online shoppers with basic smartphone skills who use the Mobile Application to search for fashion products, try out virtual outfit combinations, manage shopping activities, place orders, and track purchases.
- Needs: An intuitive mobile interface, efficient product search and filtering, virtual fitting room (outfit images and videos), personalized product recommendations, favorite product and outfit management, smooth cart and checkout processes, secure online payment, real-time order tracking, and access to membership benefits and order history.

Administrators

- Characteristics: System managers responsible for configuring, supervising, and maintaining the overall operation of the fashion e-commerce platform via the Web Admin Panel. They have good knowledge of system administration and online store management.
- Needs: Tools to manage customer and staff accounts, roles and permissions, catalog and inventory, orders and shipping, promotions and loyalty, reviews and support, storefront information, virtual try-on

operations, and permission-scoped business/search/recommendation reports.

Staff

- Characteristics: Store employees responsible for handling daily store operations via the Web Admin Panel, such as updating products, controlling inventory, processing orders, coordinating shipping, and supporting customers.
- Needs: Access to authorized management functions in the Web Admin Panel, clear operational workflows, accurate product and stock details, efficient order processing, review moderation, and tools to respond to customer support requests.

Guests (Unregistered Users)

- Characteristics: Visitors who download and open the Mobile Application without creating an account or logging in. They expect a quick, simple, and engaging browsing experience.
- Needs: Ability to browse, search, filter, view product details, receive basic discovery content, and read store policies without mandatory registration. Registration and sign-in are required for virtual try-on jobs, saved favorites, persistent cart/checkout, orders, and other account-specific functions.

1.2. Functional Requirements

1.2.1. Use Case Diagram

1.2.1.1. Customer

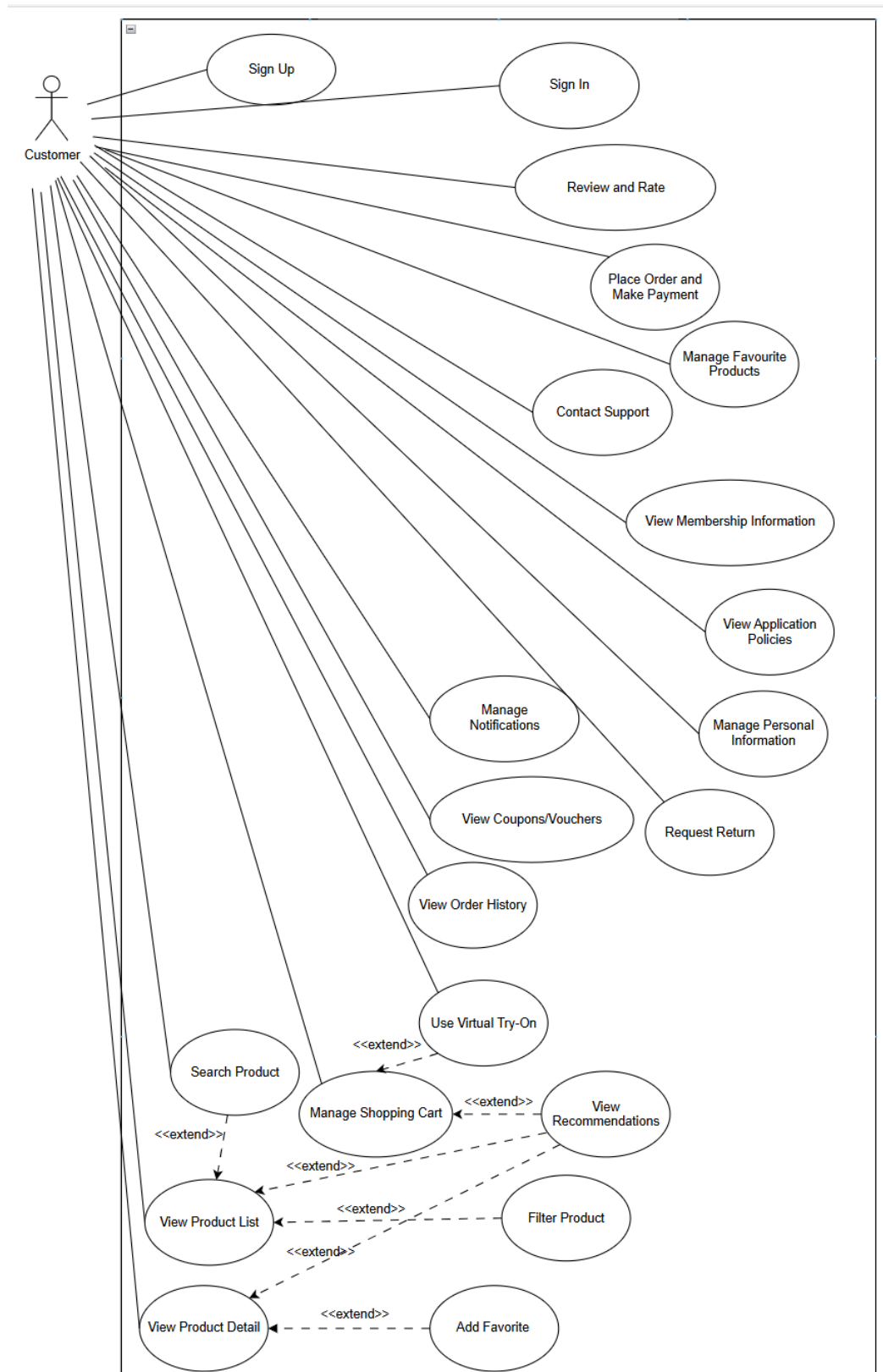


Figure 1.1 Customer use case diagram

1.2.1.2. Admin & Staff

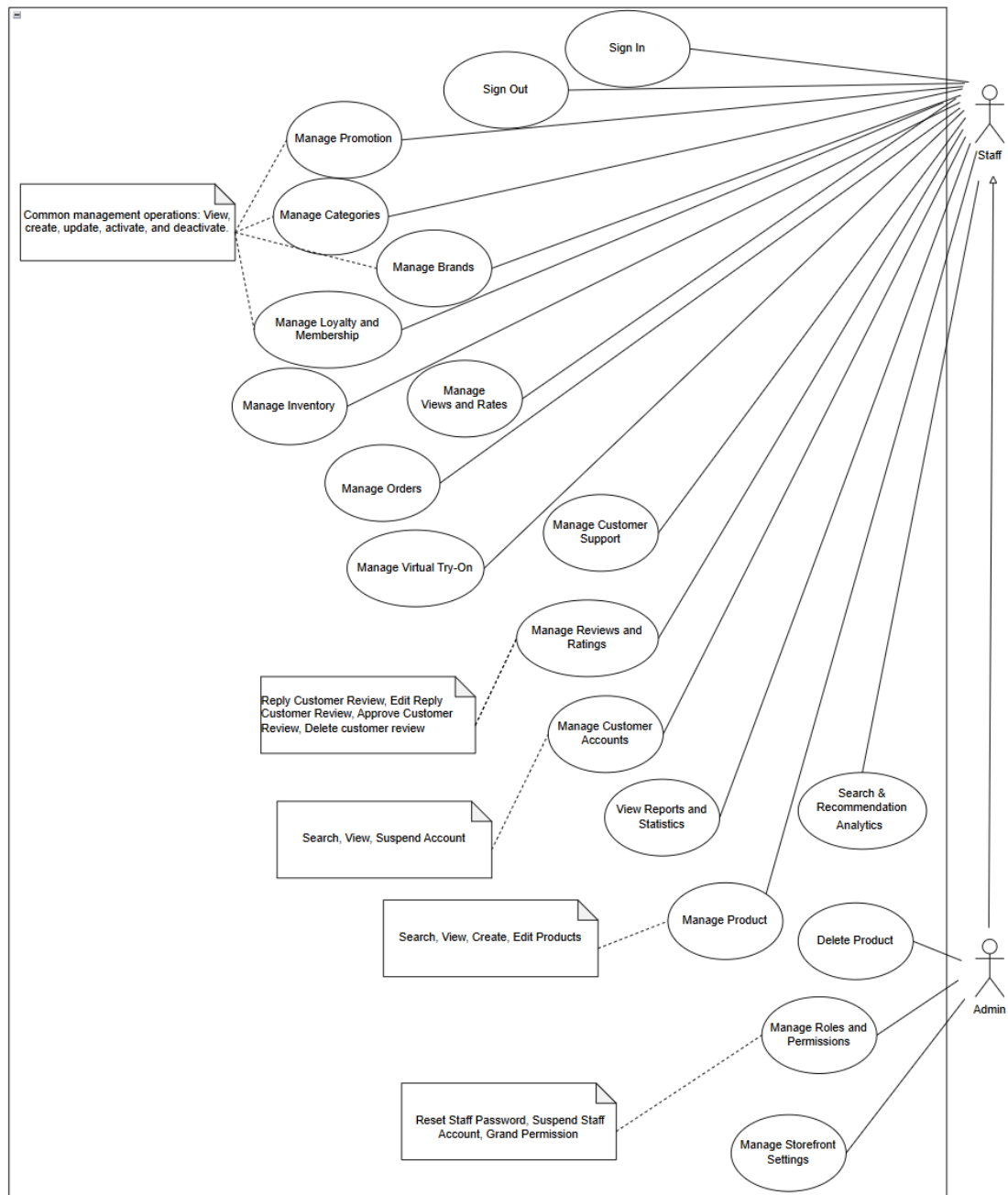


Figure 1.2 Admin and staff use case diagram.

1.2.1.3. Guest

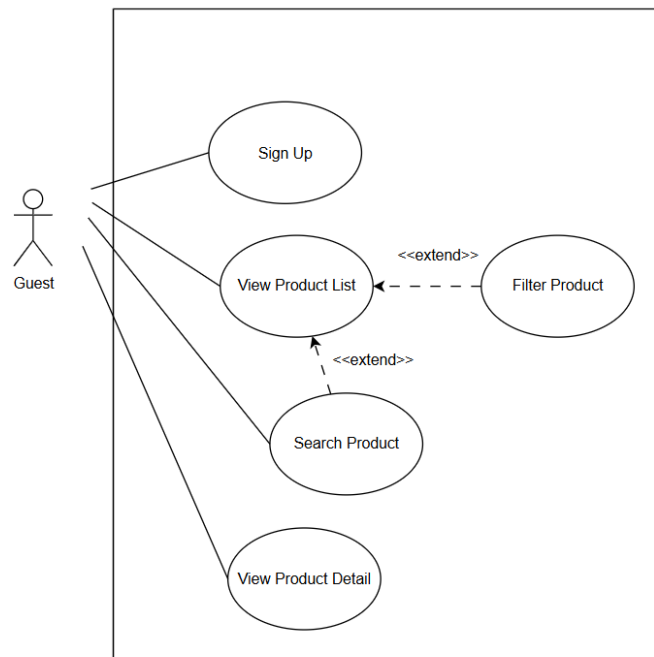


Figure 1.3 Guest use case diagram

1.2.2. Use Case Specification

1.2.2.1. Sign In

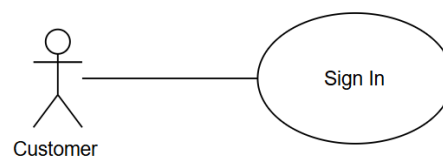


Figure 1.4 Sign-in use case diagram

Table 1.1 Sign In use case specification

Function name: Sign In	ID: UC-MOB-01
Main actor: Customer	Essential: Important
	Classification: High
Brief description: Allows a registered customer to authenticate and access protected shopping and account functions.	

Relationships: + Combination: None + Includes: None + Extend: None
Precondition: 1. The customer has a registered account. 2. The account is active, and the system is available.
Normal flow: 1. The customer opens the Sign In screen. 2. The system displays fields for an email address or phone number and a password. 3. The customer enters the credentials and submits the form. 4. The mobile application validates the required fields. 5. The authentication service verifies the credentials, account status, and temporary lock state. 6. The system stores the access and refresh tokens for the authenticated session. 7. The system opens Force Change Password if required, Edit Profile if the profile is incomplete, or Home otherwise.
Subflows: 1. Account unlock: a temporarily locked customer selects email or SMS, requests a six-digit OTP, verifies it, and then attempts sign-in again.
Alternate/Exception Flow: AL-01 - Missing Credentials: At step 4, if the identifier or password is empty, the application identifies the missing field and returns to step 3. AL-02 - Incorrect Credentials: At step 5, if the credentials are incorrect, the service rejects the request without indicating whether the account exists and returns to step 3. AL-03 - Temporarily Locked Account: At step 5, if the account is temporarily locked, the application displays the available unlock channels and starts subflow S1 when selected. After a successful unlock, the use case returns to step 3. AL-04 - Invalid or Expired Unlock OTP: During subflow S1, if the OTP is incomplete, invalid, or expired, the account remains locked. The customer may enter another code or request a new one. AL-05 - Inactive Account: At step 5, if the account is inactive, the system denies access, and the use case ends.

EX-01 - Authentication Request Failure: At step 5, if authentication cannot be completed, the application displays the returned error, keeps the customer on Sign In, and then returns to step 3.

1.2.2.2. Sign Up

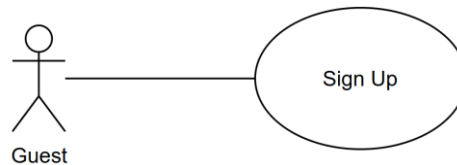


Figure 1.5 Sign-up use case diagram

Table 1.2 Sign Up use case specification

Function name: Sign Up	ID: UC-MOB-02
Main actor: Guest	Essential: Important
	Classification: High
Brief description: Allows a guest to create a customer account after verifying a phone number and accepting the current legal policies.	
Relationships: + Combination: None + Includes: None + Extend: None	
Precondition: 1. The guest is not signed in. 2. The email address and phone number are not associated with another account.	
Normal flow: 1. The guest opens the Sign Up screen. 2. The system loads province/city and ward/commune options and displays the registration form. 3. The guest enters a valid Vietnamese phone number and requests an OTP. 4. The system sends the OTP; the guest enters the code and requests verification. 5. The system verifies the OTP and binds the verification token to the current phone number.	

6. The guest enters name, email, gender, date of birth, delivery address, password, and password confirmation.
7. The guest accepts the current Terms of Use and Privacy Policy and submits the form.
8. The system validates the form, OTP token, password rules, policy version, address, and the uniqueness of the email and phone number.
9. The system creates the customer account and its first default delivery address.
10. The application displays successful registration and opens Sign In.

Subflows:

1. Resend OTP: After the required waiting period, the guest requests a new OTP. Changing the phone number cancels the previous verification.
2. Manual ward information: When permitted, the guest enters ward information not listed in the displayed location list.

Alternate/Exception Flow:

AL-01 - Invalid Registration Data: At step 8, if a required personal, contact, date-of-birth, or address field is invalid, the application identifies the field. The use case returns to step 6.

AL-02 - Weak or Mismatched Password: At step 8, if the password is fewer than eight characters, lacks a letter or number, or does not match its confirmation, the application rejects registration. The use case returns to step 6.

AL-03 - Phone Not Verified: At step 8, if the OTP token is missing or associated with another phone number, the application requires completion of steps 3-5.

AL-04 - Invalid or Expired OTP: At step 5, if the OTP is invalid or expired, the phone remains unverified. The guest may request another code after the waiting period.

AL-05 - Policies Not Accepted: At step 8, if the current policies are not accepted, the system does not create the account. The use case returns to step 7.

AL-06 - Duplicate Email or Phone: At step 8, if the email address or phone number already exists, the system reports the conflicting field. The use case returns to step 6.

EX-01 - Registration Service Failure: At steps 2, 4, 5, or 9, if a required service fails, the application displays an error and preserves entered data when possible. The guest may retry the failed step. E2. Account creation cannot be completed: no partial customer account is retained, and the guest may try again.

1.2.2.3. View Product List

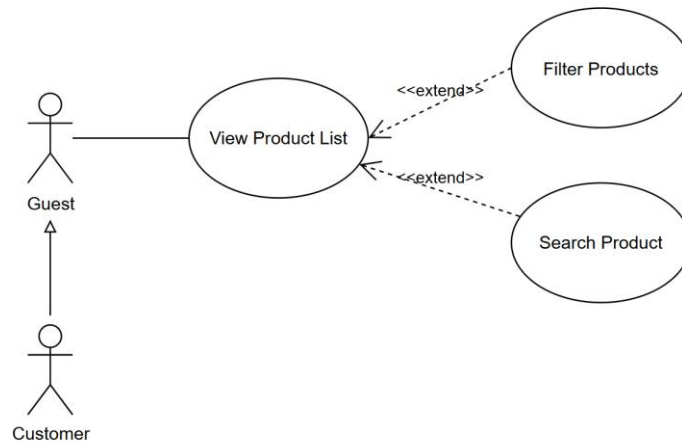


Figure 1.6 Product listing, filtering and searching use case diagram

Table 1.3 View Product List use case specification

Function name: View Product List	ID: UC-MOB-03
Main actor: Guest, Customer	Essential: Important
	Classification: Medium
Brief description: Displays active fashion products with pagination, pricing, availability, and sorting information.	
Relationships: + Combination: None + Includes: None + Extend: None	
Precondition: 1. The catalog service is available.	
Normal flow: 1. The actor opens the home screen or a product collection. 2. The system requests active products and the applicable catalog metadata. 3. The system resolves current prices, discounts, images, ratings, and stock availability. 4. The system displays the first page of products. 5. The actor scrolls or changes sorting, and the system requests the next result set. 6. The actor selects a product to open its details.	

Subflows: 1. Authenticated browsing: Home may display a separate personalized recommendation section when the customer is signed in; failure of that section does not block the catalog list.
Alternate/Exception Flow: AL-01 - No Matching Products At step 4, if the system finds no matching active products, it displays an empty-state message. The use case returns to step 3. EX-01 - Product Loading Failure At step 2, if the product information is temporarily unavailable, the system offers a retry action and retains any previously displayed results. The use case returns to step 1 when the actor retries.

1.2.2.4. Search Product

Table 1.4 Search Product use case specification

Function name: Search Product	ID: UC-MOB-04
Main actor: Guest, Customer	Essential: Important
	Classification: Medium
Brief description: Allows guests and signed-in customers to find active products by keyword using search history, debounced keyword/product/category suggestions, and a paginated result list.	
Relationships: + Combination: None + Includes: None + Extend: View Product List	
Pre-condition: 1. The catalog service is available. 2. The actor can open the Search screen.	
Normal flow: 1. The actor opens Search from Home or Product List. 2. The system focuses the search field and displays local history; for a signed-in customer, local and server histories are synchronized. 3. The actor enters at least two non-space characters.	

4. After a 300 ms debounce, the application cancels any stale suggest request and calls the suggest endpoint. 5. The backend normalizes Vietnamese text and material/gender tokens, then returns active product, category, and keyword suggestions. 6. The application merges matching history with server keywords and displays keyword, product, and category suggestions. 7. The actor submits a keyword or selects a keyword or history suggestion. 8. The application stores the keyword, creates a search event/source, and opens Product List with the keyword. 9. The catalog service returns active matching products, pagination, and filter metadata; the application displays the results and loads additional pages when requested. 10. Selecting a product opens Product Detail and records the search-result interaction.
Subflows: 1. Product suggestion: selecting a suggested product opens Product Detail directly. 2. Category suggestion: selecting a category opens Product List scoped to that category. 3. Empty submission: the application opens all products sorted by newest. 4. History management: the actor removes one term or clears all terms; authenticated deletions are also sent to the server. 5. Result refinement: the actor applies catalog filters or sorting while the search keyword remains active.
Alternate/Exception Flow: AL-01 - Query shorter than two characters: The system does not call the suggest endpoint and keeps history or the empty-search guidance visible. AL-02 - Suggestion service unavailable: The system displays a suggestion error but still allows exact keyword submission. AL-03 - No suggestions or products: The system displays the corresponding empty state and allows the actor to adjust or submit the keyword. AL-04 - Network or catalog error: The system keeps the current usable state and displays the available retry/fallback action.

1.2.2.5. Filter Products

Table 1.5 Filter Products use case specification

Function name: Filter Products	ID: UC-MOB-05
---------------------------------------	----------------------

Main actor: Guest, Customer	Essential: Important
	Classification: Medium
Brief description: Narrows and sorts the catalog by keyword and fashion-specific criteria.	
Relationships: + Combination: None + Includes: None + Extend: View Product List	
Precondition: 1. Catalog categories, brands, and filter metadata are available.	
Normal flow: 1. The actor opens Search or the filter panel on Product List. 2. The actor enters a keyword or selects the filters currently exposed by mobile: gender, category, brand, price range, sale state, new-product state, and sorting order. 3. The application normalizes the selected values and submits the query. 4. The catalog service searches active products and returns a paginated result and available filter metadata. 5. The application displays active filter chips and matching products. 6. The actor changes, removes, or clears criteria; the application resets to page one and refreshes the result.	
Subflows: 1. Search history: the system records the keyword, source, result count, and selected result; signed-in history can be available on the customer's signed-in devices.	
Alternate/Exception Flow: AL-01 - No Matching Result: At step 4, if no matching result is found, the system displays an empty state and search suggestions where available. The use case returns to step 3. AL-02 - Invalid Filter Value: At step 3, if a filter value is invalid, the system ignores or rejects the unsupported value and returns a validation message. The use case returns to step 2. EX-01 - Request Failure: At step 4, if the request fails, the previous result remains available and the actor can retry. The use case returns to step 3 when the actor retries.	

1.2.2.6. View Product Detail

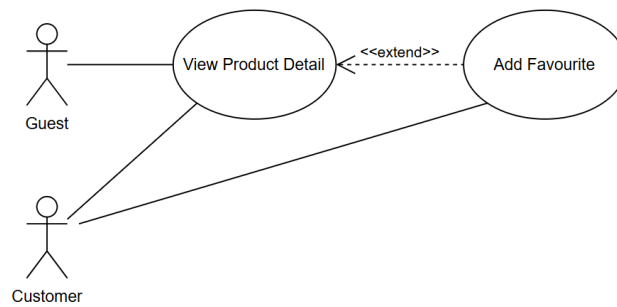


Figure 1.7 Product details and favorites use case diagram

Table 1.6 View Product Detail use case specification

Function name: View Product Detail	ID: UC-MOB-06
Main actor: Guest, Customer	Essential: Important
	Classification: Medium
Brief description: Displays current product information, selectable variants, inventory availability, public reviews, and similar-product recommendations, with entry points to favorite, cart, buy-now, and virtual try-on actions.	
Relationships: + Combination: None + Includes: None + Extend: None	
Precondition: 1. The actor selects a product that exists in the catalog.	
Normal flow: 1. The actor selects a product from a catalog, search results, favorites, recommendations, or notifications. 2. The system loads the active product and separately requests public reviews and similar-product recommendations. 3. The application displays images, price, discount, description, category, brand, fit/variant options, colors, sizes, measurements, and current availability. 4. The actor selects a valid fit/variant, color, size, and quantity for purchase or try-on. 5. The actor may toggle Favorite, add the item to Cart, choose Buy Now, or open a similar product. 6. The system validates the selected options, performs the requested action, and records supported view or recommendation interactions without blocking the core display.	

Subflows: 1. Interaction tracking: the system records product views and recommendation clicks for ranking and analytics without blocking display.
Alternate/Exception Flow: AL-01 - Product Not Found or Inactive: At step 2, if the product is missing or inactive, the application displays the returned error and offers navigation back to the catalog. The use case ends. AL-02 - Required Option Not Selected: At steps 4-6, if a required variant, color, or size is missing, the application opens the selection control and asks the actor to complete it. The use case returns to step 4. AL-03 - Selected Option Out of Stock: At steps 4-6, if the selected combination has no available inventory, quantity and commerce actions are disabled or rejected. The use case returns to step 4. AL-04 - Protected Action by Guest: At step 5, if a guest selects Favorite, Cart, Buy Now, or the application requests sign-in. The actor may return to the product after authentication. EX-01 - Secondary Content Failure: At step 2, if reviews or similar recommendations fail, core product detail remains usable, and the unavailable secondary section is omitted or shows its local error state.

1.2.2.7. Add Favorite

Table 1.7 Add Favorite use case specification

Function name: Add Favorite	ID: UC-MOB-07
Main actor: Customer	Essential: Important
	Classification: Medium
Brief description: Allows an authenticated customer to add a fashion product to their favorite list.	
Relationships: + Combination: None + Includes: None + Extend: View Product Detail	
Precondition: 1. The actor is viewing Product Detail. 2. The selected product exists.	

Normal flow:

1. The actor selects the outlined heart on the Product Detail page.
2. If the actor is not authenticated, the application prompts to sign in; otherwise, it continues.
3. The application immediately updates the heart to the favorited state and disables repeated taps while the request is in progress.
4. The favorites service verifies that the product exists and is active.
5. The service creates the customer-product favorite association; an existing association is preserved without duplication.
6. The application retains the filled-heart state and invalidates cached favorite lists/status for later refresh.

Subflows:

1. Idempotent add: if the association already exists, the service returns the favorited state without creating a duplicate record.

Alternate/Exception Flow:**AL-01 - Guest Selects Favorite**

At step 2, if the actor is not signed in, the application displays a sign-in prompt. If the actor postpones sign-in, the product remains unchanged, and the use case ends.

AL-02 - Product Not Available

At step 4, if the product is missing or inactive, the service rejects the request. The application restores the outlined heart and returns to Product Detail.

AL-03 - Request Already Running

At step 3, if the favorite control is already processing, the app ignores additional taps until the current request completes.

EX-01 - Favorite Update Failure

At steps 4-5, if the request fails or authentication can no longer be refreshed, the application restores the previous heart state and displays the returned error. The use case ends.

1.2.2.8. Manage Shopping Cart

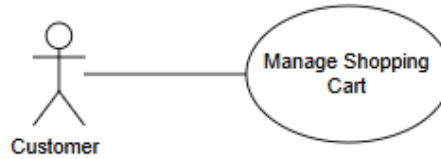


Figure 1.8 Shopping cart use case diagram

Table 1.8 Manage Shopping Cart use case specification

Function name: Manage Shopping Cart	ID: UC-MOB-08
Main actor: Customer	Essential: Important
	Classification: High
Brief description: Maintains purchasable product variants and the subset selected for checkout.	
Relationships: + Combination: None + Includes: None + Extend: None	
Precondition: 1. The customer is signed in. 2. The selected product variant, color, size, and inventory item exist.	
Normal flow: 1. The customer selects an active product and chooses an available variant, color, size, and quantity. 2. The customer selects Add to Cart. 3. The system verifies the selected product options, current selling price, and available quantity. 4. The system creates a cart item or combines the quantity with an existing equivalent item. 5. The system updates the cart item count and confirms the change. 6. The customer opens the Shopping Cart. 7. The system displays the current cart items, availability, selection state, individual totals, and the cart subtotal. 8. The customer changes quantities, selects individual items, selects all eligible items, or removes items. 9. The system validates each change and recalculates the cart totals. 10. The system enables checkout when at least one valid item is selected.	

Subflows:

1. Remove item: the customer confirms removal and the system deletes the selected cart item.
2. Select all: the system selects or clears all eligible items and recalculates the subtotal.
3. Add from virtual try-on: the customer adds one or more available garments from a completed virtual try-on result.
4. Cart recommendation: the customer selects a complementary recommended product and adds an available option to the cart.

Alternate/Exception Flow:

AL-01 - Inactive Account

At step 3, if the customer is not signed in or the account is inactive, the system denies cart management. The use case returns to step 2.

AL-02 - Product Or Selected Option Is Unavailable

At step 3, if the product or selected option is unavailable, the system does not add or update the item. The use case returns to step 2.

AL-03 - Invalid or Excessive Quantity

At step 9, if the requested quantity is invalid or exceeds available stock, the system reports the current available quantity. The use case returns to step 8.

AL-04 - Cart Information Changed

At step 9, if price or availability changed after the cart was displayed, the system refreshes the cart using current values. The use case returns to step 8.

AL-05 - Cart Item No Longer Exists

At step 9, if the selected cart item no longer exists, the system refreshes the cart and does not apply the requested change. The use case returns to step 8.

EX-01 - Cart Update Failure

At step 9, if the cart cannot be updated, the system keeps the last confirmed cart state and informs the customer. The use case ends.

1.2.2.9. Place Order and Make Payment

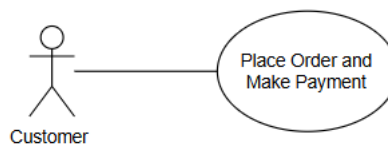


Figure 1.9 Checkout and payment use case diagram

Table 1.9 Place Order and Make Payment use case specification

Function name: Place Order and Make Payment	ID: UC-MOB-09
Main actor: Customer	Essential: Important

	Classification: High
Brief description: Creates an order from selected cart items using an authoritative checkout quote and completes COD or VNPay payment handling.	
Relationships: + Combination: None + Includes: None + Extend: None	
Pre-condition: 1. The customer is signed in. 2. At least one valid cart item is selected. 3. A deliverable address is provided or selected.	
Normal flow: 1. The customer proceeds to checkout with one or more items in the cart. 2. The system displays the selected products and available delivery addresses. 3. The customer selects or enters a delivery address. 4. The customer selects COD or VNPay as the payment method and may apply an eligible voucher. 5. The system verifies product availability, current prices, voucher eligibility, membership benefits, delivery availability, and shipping fees. 6. The system displays the complete order summary and final payable amount. 7. The customer reviews the information and confirms the order. 8. The system verifies that the displayed checkout information is still current. 9. The system records the order, updates inventory, records the use of applicable promotions, and removes the purchased items from the cart. 10. For COD, the system confirms order creation and displays the Order Success screen. 11. For VNPay, the system redirects the customer to VNPay to complete payment. 12. The system records the confirmed payment result and displays the order in Order History.	
Subflows: 1. Saved address: the customer selects an existing address or adds a new one before confirming checkout. 2. Voucher: the system rechecks the selected voucher against the exact checkout items before applying the discount. 3. VNPay retry: the customer retries payment for an eligible unpaid order before the payment deadline. 4. Delivery quotation fallback: when the delivery provider cannot return a live quotation, the system clearly identifies any configured fallback quotation before confirmation.	

Alternate/Exception Flow:

AL-01 - No Valid Checkout Item

At step 1, if no valid cart item is selected, the system does not proceed with checkout. The use case returns to step 1.

AL-02 - Invalid Delivery Address

At step 5, if the address is incomplete or outside the supported delivery area, the customer must correct or replace it. The use case returns to step 4.

AL-03 - Voucher No Longer Eligible

At step 5, if the voucher is no longer eligible, the system removes the benefit and recalculates the order summary. The use case returns to step 4.

AL-04 - Checkout Information Changed

At step 8, if the product price, stock, discount, or shipping information changes after review, the system requires the customer to review an updated summary. The use case returns to step 7.

AL-05 - Insufficient Inventory

At step 8, if inventory is insufficient, the system does not create the order and identifies the affected item. The use case returns to step 7.

AL-06 - VNPay Payment Failure or Cancellation

At step 11, if VNPay payment is canceled, fails, or cannot be initiated, the order remains unpaid and may be retried while eligible. The use case returns to step 10.

EX-01 - Order Creation Failure

At step 9, if order creation cannot be completed, the system does not leave a partial order or an incorrect inventory change. The use case ends.

1.2.2.10. View Order History

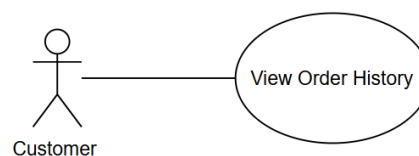


Figure 1.10 Order history use case diagram

Table 1.10 View Order History use case specification

Function name: View Order History	ID: UC-MOB-10
Main actor: Customer	Essential: Important
	Classification: High
Brief description: Displays the customer's orders, payment state, fulfillment timeline, and available post-order actions.	

Relationships: + Combination: None + Includes: None + Extend: None
Precondition: 1. The customer is signed in.
Normal flow: 1. The customer opens My Orders. 2. The system displays the customer's orders and a summary of their current statuses. 3. The customer searches, filters, refreshes, or loads additional orders. 4. The system displays only orders belonging to the signed-in customer. 5. The customer selects an order. 6. The system displays the ordered items, prices, delivery address, payment status, shipping information, status timeline, and return information. 7. When the order status changes, the system refreshes the displayed information.
Subflows: 1. Available actions include canceling an eligible new order, retrying VNPay, confirming carrier-delivered receipt, requesting return, reviewing an item, or contacting support.
Alternate/Exception Flow: AL-01 - Order Number Belongs To Another Customer At step 4, if the order number belongs to another customer, access is denied, and the use case returns to step 3. AL-02 - No Matching Orders At step 3, if no order matches the filters, the system displays an empty state and returns to step 2. EX-01 - Order Loading Failure At step 2, if the current order information is temporarily unavailable, the system informs the customer and allows a later retry. The use case returns to step 1 when the actor retries.

1.2.2.11. Review and Rate Products

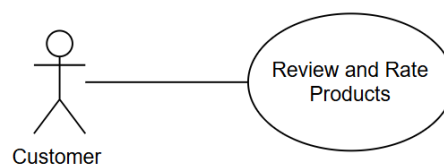


Figure 1.11 Product review and rating use case diagram

Table 1.11 Review and Rate Products use case specification

Function name: Review and Rate Products	ID: UC-MOB-11
Main actor: Customer	Essential: Important
	Classification: Medium
Brief description: Creates, updates, or removes a verified review for a product purchased in a completed, paid order.	
Relationships: Combination: None Includes: None Extend: None	
Precondition: 1. The customer owns a completed and paid order item. 2. The order item does not already have another review.	
Normal flow: 1. The customer opens an eligible order item or My Reviews. 2. The system verifies review eligibility for the specific order item. 3. The customer selects a rating, enters review content, and optionally attaches images. 4. The system submits the review. 5. The system creates a verified-purchase review and evaluates moderation rules. 6. The system updates the public product rating when the review is visible. 7. The customer can later edit or delete the owned review.	
Subflows: 1. Helpful vote: another signed-in customer may toggle a helpful vote on a visible review. 2. Moderation: content matching offensive-language rules is stored as Pending and excluded from public ratings until reviewed.	

Alternate/Exception Flow:

AL-01 - Review Not Eligible

At step 2, if the order item is not eligible, the order is unpaid, or a review already exists for that order item, creation is rejected. The use case returns to step 1.

AL-02 - Invalid Review Data

At step 4, if the rating, required content, or image upload is invalid, validation fails. The use case returns to step 3.

AL-03 - Unauthorized Review Modification

At step 7, if the customer attempts to modify another customer's review, access is denied. The use case returns to step 6.

AL-04 - Review Hidden During Voting

At step 7, if the review becomes hidden during a helpful-vote request, the vote is rolled back. The use case returns to step 6.

1.2.2.12. Contact Support

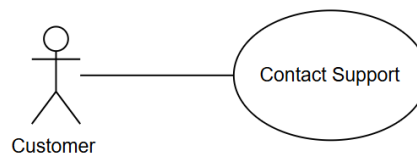


Figure 1.12 Customer support use case diagram

Table 1.12 Contact Support use case specification

Function name: Contact Support	ID: UC-MOB-12
Main actor: Guest, Customer	Essential: Important
	Classification: High
Brief description: Provides public FAQ and feedback functions and authenticated support-ticket conversations.	
Relationships: Combination: None Includes: None Extend: None	
Precondition: P1. The support service is available.	

Normal flow:

1. The actor opens the Support Center and browses or searches the public FAQs.
2. A signed-in customer selects Create Ticket, chooses a category, and enters a subject and message.
3. The customer may associate an order and attach up to three supported files.
4. The system validates the request and creates a support ticket.
5. The system displays the ticket in the customer's support history.
6. The customer and authorized support staff exchange messages in the ticket conversation.
7. The customer may mark messages as read and, when the ticket's status allows, close or reopen the ticket.

Subflows:

1. Guest feedback: a guest submits the public feedback form and receives a verification link without opening an authenticated ticket.
2. FAQ vote: a signed-in customer marks a public FAQ as helpful or not helpful.

Alternate/Exception Flow:

AL-01 - Invalid Support Ticket Data

At step 4, if required ticket information is missing or invalid, the system rejects the request. The use case returns to step 3.

AL-02 - Invalid Support Attachment

At step 3, if an attachment uses an unsupported format, exceeds the size limit, or exceeds the allowed count, the system rejects the attachment. The use case returns to step 2.

AL-03 - Support Rate Limit

At step 4, if the customer submits requests or messages too frequently, the system requires the customer to wait. The use case returns to step 3.

AL-04 - Unauthorized Ticket Access

At step 6, if the selected ticket belongs to another customer, the system denies access. The use case returns to step 5.

AL-05 - Invalid Ticket State

At step 7, if the ticket status does not permit the requested action, the system explains the allowed next action. The use case returns to step 6.

EX-01 - Message Delivery Failure

At step 6, if the system cannot deliver a message, it retains the last confirmed conversation and allows the customer to retry. The use case returns to step 5 when the customer retries.

1.2.2.13. View Recommendations

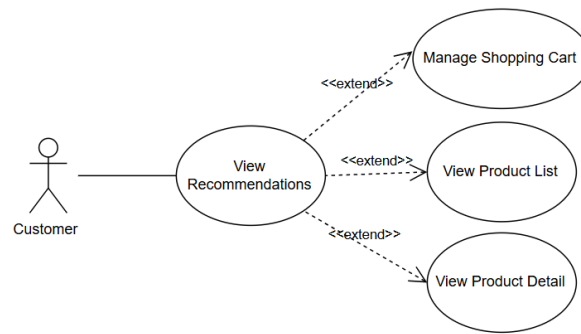


Figure 1.13 Product recommendations use case diagram

Table 1.13 View Recommendations use case specification

Function name: View Recommendations	ID: UC-MOB-13
Main actor: Customer	Essential: Important
	Classification: High
Brief description: Displays relevant products for general discovery, similar-product browsing, and complementary cart suggestions.	
Relationships: Combination: None Includes: None Extend: View Product List; View Product Detail; Manage Shopping Cart	
Precondition: 1. The current screen supplies the context required by its recommendation endpoint. 2. The surrounding shopping screen is available independently of recommendations.	
Normal flow: 1. The actor opens Home, Product Detail, or Cart. 2. The application identifies the interface-specific context. 3. Home requests personal or session-based recommendations; Product Detail requests items similar to the viewed product; Cart requests items complementary to the current cart. 4. The service filters ineligible products, ranks the remaining candidates for that context, and returns the ordered list with request metadata, when available. 5. The application displays the recommendation rail without blocking the primary screen. 6. When a recommendation is displayed, the application records an impression on a best-effort basis. 7. The actor selects a recommendation; the application records the click and opens Product Detail.	
Subflows: 1. Home personal recommendation: signed-in behavior history or anonymous session context supports broad personal relevance; when history is insufficient, the service's general fallback is used.	

2. Product Detail similar recommendation: the viewed product identifier is the anchor, so candidates are ranked by product similarity rather than by the customer's complete profile. 3. Cart complementary recommendation: the selected cart composition is the anchor, so candidates are ranked for outfit/order completion rather than for substitution.
Alternate/Exception Flow: AL-01 - Guest Context At step 3 on Home or Product Detail, if the actor is a guest, the application requests recommendations without a customer access token and uses the available anonymous session/context. AL-02 - No Eligible Recommendation At step 4, if no eligible items are returned, the application omits the rail or leaves it empty while the primary screen remains usable. AL-03 - Candidate Becomes Unavailable At steps 4-5, if a product is no longer active or displayable, the service or application excludes it from the displayed list. EX-01 - Recommendation Request Failure At steps 3-4, if the request fails, the interface-specific rail shows no items or enters its local error state. Home, Product Detail, or Cart remains usable. EX-02 - Interaction Tracking Failure At steps 6-7, if impression or click tracking fails, navigation and shopping continue because tracking is non-blocking.

1.2.2.14. Use Virtual Try-On

The customer use case begins before generation, with source and garment selection, and ends only when the result has been persisted and can be reused in the shopping journey. Validation, submission, progress tracking, history, retry, and commerce handoff are consequently treated as parts of a single coherent function.

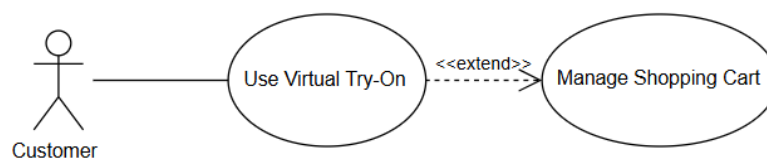


Figure 1.14 Virtual try-on use case diagram

Table 1.14 Use Virtual Try-On use case specification

Function name: Use Virtual Try-On	ID: UC-MOB-14
Main actor: Customer	Essential: Important
	Classification: High

Brief description: Allows a signed-in customer to orchestrate virtual try-on image and optional video generation through the integrated external workflow/model, then manage the resulting assets and commerce handoff.
Relationships: Combination: None Includes: None Extend: Manage Shopping Cart
Precondition: 1. The customer is signed in with an active account. 2. Virtual try-on is enabled, and the customer is within concurrency and daily limits.
Normal flow: 1. The customer opens Virtual Try-On directly or via seeded Product Detail/Cart items. 2. The application loads capabilities, image assets, recent jobs, and any seeded garments. 3. The customer uploads/captures or selects one personal image. 4. The customer chooses Single Item or Full Set and selects exact active variants, colors, sizes, and garment roles; up to four items are active, while mobile may retain up to eight queued items. 5. The system validates the personal image for the selected roles, including person detection, visible regions, pose, quality, and safety information. 6. The customer selects a context preset or an allowed custom context and chooses image-only or image-and-video output; video duration is 5-12 seconds when selected. 7. The customer confirms; the application submits an idempotent asynchronous job. 8. The backend validates capacity, source suitability, garment selection, and prompt policy, then maps the inputs into the configured external workflow/model. 9. The application displays real-time progress and polls every 2.5 seconds as a fallback until the job succeeds, fails, or is canceled. 10. After image success, the system displays one or more persistent try-on images and the optional video/status. A video-only failure does not discard valid images. 11. The customer may save/share media, retry eligible image or video work, review/delete history, return to the builder, or add the selected-sized garments to Cart/Checkout.
Subflows: 1. Soft image warning: for a non-blocking validation warning, the application explains the risk and asks whether to continue or select another image. 2. Retry or cancel: queued or processing work may be canceled; eligible failed or canceled image work or failed optional video work may be retried. 3. Commerce handoff: the application verifies that every selected garment has a size, adds or replaces the items with a quantity of one, selects them, and opens Cart or Checkout.
Alternate/Exception Flow: AL-01 - Missing Source Image At step 7, if no source asset is selected, the application returns to the photo/library step. The use case returns to step 3. AL-02 - Invalid Garment Selection At steps 4 or 8, if the item count, outfit mode, role, variant, size, or availability is invalid, the application/backend rejects creation and returns to step 4.

AL-03 - Image Validation Incomplete or Hard-Blocked

At steps 5-7, if validation is still running, submission waits. If no person is detected or another hard safety/capability block applies, creation is blocked, and the customer must choose another image.

AL-04 - Soft Image Warning

At steps 5-7, if the issue is non-blocking, the application runs subflow S1; the customer may continue or replace the image.

AL-05 - Invalid Context or Usage Limit

At step 8, if the custom context violates policy or the account exceeds current daily/concurrency rules, the backend rejects the job. The customer must revise the input or wait.

AL-06 - Optional Video Failure

At step 10, if image generation succeeds but video generation fails, images remain available, the overall image result is kept, and the customer may retry only the video.

AL-07 - Missing Size Before Commerce Handoff

During subflow S3, if any garment lacks a size, the application identifies the product and keeps the customer on the result/builder.

EX-01 - Job Creation or Image Processing Failure

At steps 7-10, if the service cannot create or finish image generation, the job records the failure and the application offers retry or return-to-builder when eligible.

EX-02 - Realtime Connection Failure

At step 9, if Socket.IO updates are unavailable, periodic API polling continues to refresh the job state.

1.2.2.15. Manage Personal Information

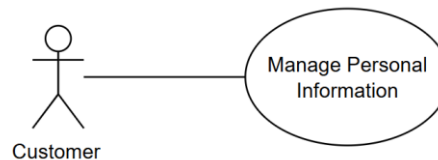


Figure 1.15 Manage personal information use case diagram

Table 1.15 Manage Personal Information use case specification

Function name: Manage Personal Information	ID: UC-MOB-15
Main actor: Customer	Essential: Important
	Classification: High
Brief description: Allows a signed-in customer to view and maintain profile details, avatar, delivery addresses, and password through the protected account services.	

Relationships: Combination: None Includes: None Extend: None
Precondition: 1. The customer is signed in with an active customer account. 2. The user and location services are available.
Normal flow: 1. The customer opens Personal Information from Profile. 2. The application loads /users/me and the address list in parallel and loads province/ward options. 3. The system displays Profile, Address, and Security tabs. 4. In Profile, the customer edits name, phone, gender, and date of birth; email is read-only. 5. The application validates required values and submits the profile update. 6. The backend validates name length, Vietnamese phone format/uniqueness, gender, and age from 13 to 100, then saves the user and recomputes profile completion. 7. The application updates the stored session user and confirms success. 8. The customer may manage avatar, addresses, or password through the corresponding subflow.
Subflows: 1. Avatar: choose JPG/PNG, upload it to managed media storage, or remove the current avatar. 2. Address: add, edit, delete, or set a default recipient address using province/ward data; the backend keeps at most five addresses and one default when the list is non-empty. 3. Password: submit current, new, and confirmation values; the backend verifies the current password, hashes the new password, clears the refresh token, and revokes device/realtime access. 4. Profile completion: completing phone, gender, date of birth, and at least one address updates profileCompleted.
Alternate/Exception Flow: AL-01 - Session invalid A 401 response clears the local session and redirects the customer to Sign In. AL-02 - Invalid or duplicate profile data The system displays field- or server-side validation; a phone number already used by another account returns a conflict. AL-03 - Location data unavailable The current address remains visible; manual ward entry is allowed only when the location service explicitly permits it. AL-04 - Invalid avatar Unsupported formats, unreadable base64, or images exceeding the backend 3 MB limit are rejected. AL-05 - Address rule violation The system rejects a sixth address, an unknown address, or invalid recipient/location fields. AL-06 - Password failure

The system rejects an incorrect current password, a mismatched confirmation, a weak password, or a new password equal to the current password.

1.2.2.16. Manage Customer Accounts

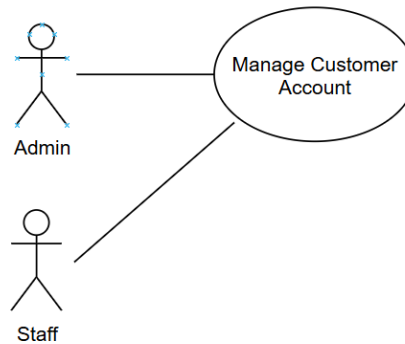


Figure 1.16 Customer account management use case diagram

Table 1.16 Manage Customer Accounts use case specification

Function name: Manage Customer Accounts	ID: UC-ADM-01
Main actor: Admin, Staff	Essential: Required
	Classification: High
Brief description: Allows an administrator or authorized staff member to search and inspect customer accounts, review profile, address, order, and activity information, maintain internal notes, activate or deactivate access, force a password reset, and perform administrator-only role changes.	
Relationships: + Combination: None + Includes: None + Extend: Manage Roles and Permissions for administrator-only role changes	
Precondition: 1. The actor is signed in to the Web Admin Panel with an active Admin or Staff account. 2. Viewing requires customers.read permission. 3. Notes, status changes, and forced resets require customers.manage; role changes require Admin.	
Normal 1. The actor opens Customer Management. 2. The system verifies the actor and loads summary cards plus the first customer page. 3. The actor searches by name, email, or phone and may filter by account state. 4. The system returns the matching authorized customer records. 5. The actor opens a customer drawer and reviews overview, addresses, orders, activity, notes, and administration data. 6. The actor selects an allowed note, status, password-reset, or role action. 7. The system validates the target, actor, current state, and submitted data. 8. The system saves the change, applies security consequences, records administrative activity, and refreshes the customer view.	Flow:

Subflows:

S1 - Customer insights: aggregate recent orders, spend, status, activity, and loyalty information for the selected customer.

S2 - Internal notes: create, edit, or delete a 1-2,000-character staff-only note.

S3 - Account status/reset: activate or deactivate the customer, or issue a forced password reset and revoke affected sessions.

S4 - Role change: an administrator changes only an eligible target role through the protected administrator endpoint.

Alternate/Exception Flow:

AL-01 - No matching customer: display an empty result while preserving filters.

AL-02 - No order or activity data: display a clear empty state in the selected drawer tab.

EX-01 - Insufficient permission: deny the page or mutation and retain stored data.

EX-02 - Invalid or protected target: reject missing users, protected roles, stale notes, or unsupported state changes.

EX-03 - Invalid note or reset delivery failure: identify the failed action without saving partial data.

EX-04 - Service failure: keep the last confirmed view where possible and offer retry.

1.2.2.17. Manage Roles and Permissions

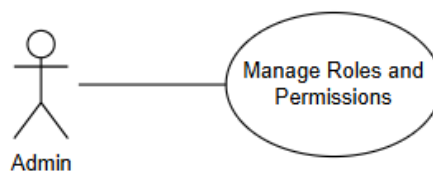


Figure 1.17 Role and permission management use case diagram

Table 1.17 Manage Roles and Permissions use case specification

Function name: Manage Roles and Permissions	ID: UC-ADM-02
Main actor: Admin	Essential: Required
	Classification: High
Brief description: Allows an administrator to list internal accounts, create Staff accounts, assign supported permissions, activate or deactivate Staff access, and set a temporary password that must be changed at the next sign-in.	
Relationships: + Combination: None + Includes: None + Extend: None	
Pre-condition: 1. The actor is signed in with an active Admin account. 2. The internal-account service is available. 3. The target is a Staff account for permission, status, or temporary-password mutations.	

Normal Flow:

1. The administrator opens Staff and Permission Management.
2. The system loads internal accounts and summary counts.
3. The administrator searches or filters by keyword, role, or status.
4. The administrator selects Create Staff or an allowed action on an existing Staff account.
5. The system displays the staff form, permission editor, status confirmation, or temporary-password form.
6. The administrator submits the required identity, credential, permission, or state data.
7. The backend validates the Admin actor, target role, uniqueness, password rules, and supported permission set.
8. The system expands required implied permissions, saves the change, revokes access when necessary, and refreshes the list.

Subflows:

- S1 - Create Staff: enter name, email, phone, temporary password, and least-privilege permissions; flag the new account to change its password.
- S2 - Update permissions: add or remove supported permissions; implied read/reply permissions are expanded consistently.
- S3 - Activate/deactivate: change Staff access without permitting modification of an Admin target.
- S4 - Reset temporary password: validate and hash the replacement, revoke existing Staff sessions, and require a password change.

Alternate/Exception Flow:

- AL-01 - No matching account: display an empty internal-account list.
- EX-01 - Duplicate email or phone: reject Staff creation and return field feedback.
- EX-02 - Invalid identity, password, or permission: reject short names, malformed contact data, weak credentials, unknown permissions, or incompatible selections.
- EX-03 - Protected target: deny attempts to demote, disable, reset, or re-permission an Admin through Staff actions.
- EX-04 - Missing or stale Staff account: stop the mutation and reload the current list.
- EX-05 - Persistence failure: do not expose a partial account or permission update.

1.2.2.18. Manage Orders

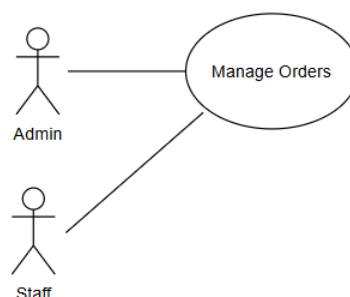


Figure 1.18 Order management use case diagram

Table 1.18 Manage Orders use case specification

Function name: Manage Orders	ID: UC-ADM-03
Main actor: Admin, Staff	Essential: Required

	Classification: High
Brief description: Allows an administrator or authorized staff member to search, inspect, export, and process orders across lifecycle, payment, invoice, shipping, return, and bulk-operation branches while preserving inventory and transaction consistency.	
Relationships: + Combination: None + Includes: None + Extend: None	
Precondition: 1. The actor is signed in to the Web Admin Panel. 2. Listing, detail, transaction, and export operations require orders.read. 3. Lifecycle, return, payment, shipping, and bulk mutations require orders.update. 4. Provider-specific actions additionally require a compatible order state and configuration.	
Normal Flow: 1. The actor opens Order Operations or Order Lookup. 2. The system loads the selected operational queue or the paginated all-order results. 3. The actor searches, filters, sorts, selects columns, or exports the authorized result set. 4. The actor opens an order and reviews overview, items, payment, shipping, return, invoice, transaction, and history. 5. The actor selects an eligible lifecycle, payment/refund, shipping/GHN, return-review, or bulk action and enters a reason or required data. 6. The system rechecks permissions, order state, payment state, shipping state, inventory effects, and action-specific limits. 7. The backend performs the action, records provider and transaction outcomes, and applies inventory or notification side effects atomically where required. 8. The system refreshes the queue/detail view and displays the confirmed result; bulk results identify each success and failure.	
Subflows: S1 - Lookup/export: find by order, invoice, customer, product, payment, date, or shipping data and export CSV/Excel. S2 - Lifecycle: confirm, pack, hand off, cancel, deliver, complete, or reject an invalid transition with an explanation. S3 - Payment/refund: inspect transactions, reconcile VNPay, record eligible adjustments, and refund through VNPay or a saved verified bank account as applicable. S4 - Shipping: resolve GHN area mapping, create/sync/cancel a shipment, update permitted shipping data, or process a simulated webhook. S5 - Return review: inspect reason/evidence, approve to the return workflow, or reject and restore the previous delivered/completed state. S6 - Bulk operation: compute the action shared by all selected orders, process each independently, and retain failures for correction.	

Alternate/Exception Flow:

AL-01 - Empty queue or result: display a usable empty state and keep filters.
 AL-02 - Provider fallback: retain the order with an explicit fallback/partial shipping state when a live quote or mapping is unavailable.
 EX-01 - Insufficient permission: deny the requested read or mutation.
 EX-02 - Invalid or stale transition: reject an action whose order, payment, return, or shipping state changed after loading.
 EX-03 - Provider or reconciliation failure: record the confirmed provider outcome and avoid claiming success.
 EX-04 - Inventory or transaction failure: roll back the protected operation without a partial order write.
 EX-05 - Partial bulk failure: return per-order results, remove successes from the selection, and retain failed orders with reasons.

1.2.2.19. Manage Customer Support

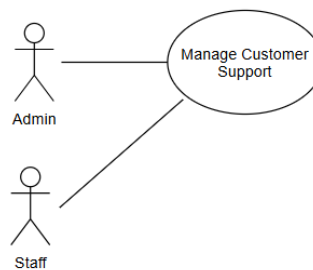


Figure 1.19 Customer support management use case diagram

Table 1.19 Manage Customer Support use case specification

Function name: Manage Customer Support	ID: UC-ADM-04
Main actor: Admin, Staff	Essential: Important
	Classification: High
Brief description: Allows authorized administration users to triage verified customer tickets, inspect context, assign and update workflow state, send public replies or internal notes, and maintain canned responses, FAQs, and support analytics.	
Relationships: + Combination: None + Includes: None + Extend: None	
Pre-condition: 1. The actor is signed in with an active Admin or Staff account. 2. Ticket queue and reply operations require support.reply. 3. FAQs, canned responses, and analytics require support.manage.	

Normal Flow:

1. The actor opens Support Management.
2. The system loads summary counts, filters, and the prioritized verified ticket queue.
3. The actor opens a ticket and reviews customer, order, category, priority, assignment, attachments, and conversation.
4. The actor assigns the ticket, changes priority/status, or marks customer messages read.
5. The actor enters a public reply or internal note and may select a canned response and up to three allowed attachments.
6. The backend validates permission, lifecycle, message, rate limit, and media.
7. The system stores the message and ticket updates; a public reply triggers customer notification and real-time delivery.
8. With support.manage, the actor maintains FAQs/canned responses or reviews analytics.

Subflows:

- S1 - Filter and triage: combine status, type, category, priority, assignee, reply need, order, and date filters.
- S2 - Public reply: send customer-visible text/attachments, update response timestamps, and emit email/push/realtime notifications.
- S3 - Internal note: store a staff-only message without exposing it to the customer.
- S4 - Knowledge management: create, edit, reorder, enable, or delete FAQs and reusable responses.
- S5 - Analytics: aggregate volume, response, resolution, category, and staff workload for the selected period.

Alternate/Exception Flow:

- AL-01 - No matching ticket: display an empty queue with filters retained.
- AL-02 - Closed ticket: require an allowed reopen transition before incompatible updates.
- EX-01 - Insufficient reply/manage permission: deny the selected action.
- EX-02 - Invalid message or attachment: reject empty/oversized input, unsupported media, or more than three files without storing a partial message.
- EX-03 - Rate limit or stale lifecycle: preserve the current conversation and request retry or refresh.
- EX-04 - Notification delivery failure: retain the confirmed ticket reply while recording or reporting the delivery problem.

1.2.2.20. Search & Recommendation Analytics

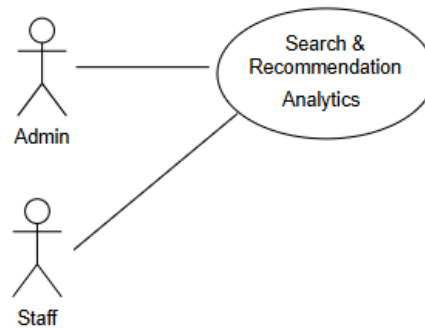


Figure 1.20 Search and recommendation analytics use case diagram

Table 1.20 Search & Recommendation Analytics use case specification

Function name: Search & Recommendation Analytics	ID: UC-ADM-05
Main actor: Admin, Staff	Essential: Required
	Classification: High
Brief description: Allows authorized administration users to analyze customer search demand and result gaps together with recommendation exposure, funnel conversion, attribution, fallback, coverage, diversity, product, context, and algorithm performance.	
Relationships: + Combination: None + Includes: None + Extend: None	
Pre-condition: 1. The actor is signed in with an active Admin or Staff account. 2. The actor has reportsread permission. 3. Recommendation-request/event and search-history data may be empty but must follow the report contract.	
Normal Flow: 1. The actor opens Search & Discovery reports. 2. The system displays the supported date range and report controls. 3. The actor selects a valid range and, for recommendation analysis, optional context and algorithm-version filters. 4. The interface normalizes date-only parameters and requests recommendation analytics. 5. The backend verifies reports.read and aggregates recommendation requests, issued candidates, events, and search history. 6. The interface displays search volume, zero-result rate, average results, product opens, query trends, and popular/gap keywords. 7. It displays recommendation impressions, clicks, CTR, cart/order/payment funnel, attributed revenue, fallback, coverage, diversity, trends, segments, products, and recent requests. 8. The actor switches report focus, changes filters, inspects details, or requests a diagnostic preview.	

Subflows:

S1 - Search analysis: group keyword/image searches, result counts, product opens, trends, and zero-result queries.

S2 - Recommendation analysis: filter Home, Product Detail Similar, Cart, or all contexts and an optional algorithm version.

S3 - Insight guard: show a segment conclusion only when the configured minimum sample is met; otherwise show raw metrics and an insufficiency note.

S4 - Preview recommendations: submit supported product/cart/customer context and return a diagnostic list without recording a purchase action.

Alternate/Exception Flow:

AL-01 - Empty report: render explicit zero denominators and empty sections.

AL-02 - Insufficient sample: withhold the best/worst conclusion while preserving raw metrics.

EX-01 - Invalid date, context, or version: reject the filter before issuing or aggregating the request.

EX-02 - Insufficient reports.read permission: deny the analytics and preview endpoints.

EX-03 - Malformed or partial payload: reject the response contract and show loading/error state rather than misleading charts.

EX-04 - Aggregation failure: provide retry and do not label partial results as complete.

1.2.2.21. Manage Virtual Try-On

The administration use case complements the customer flow. It does not generate outfits on behalf of customers; instead, it provides controlled visibility and intervention when jobs fail, prompt rules are violated, accounts must be restricted, or provider and quota settings need to change. Permissions are evaluated for every view and mutation because the available evidence may contain customer and provider information.

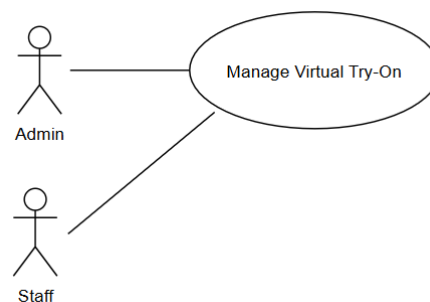


Figure 1.21 Virtual try-on management use case diagram

Table 1.21 Manage Virtual Try-On use case specification

Function name: Manage Virtual Try-On	ID: UC-ADM-06
Main actor: Admin, Staff	Essential: Important
	Classification: High

Brief description: Allows permission-scoped administration users to monitor virtual try-on jobs and outputs, retry/cancel/hide eligible work, review prompt-policy violations, manage account locks, test prompts, and maintain versioned runtime settings and rules.
Relationships: + Combination: None + Includes: Use Virtual Try-On + Extend: None
Pre-condition: 1. The actor is signed in with an active Admin or Staff account. 2. Summary and job listing require virtual_try_on.read. 3. Job, violation, and account-lock actions require virtual_try_on.manage. 4. Prompt rules, prompt testing, settings update, and rollback require virtual_try_on.settings.
Normal Flow: 1. The actor opens Virtual Try-On Management. 2. The system verifies the applicable permission and loads the job-health summary plus the selected tab. 3. The actor filters jobs or switches among jobs, violations, prompt rules, account locks, and settings. 4. The actor inspects customer, products, provider, status, progress, outputs, error, policy, and timing information. 5. The actor selects an eligible retry-image, retry-video, cancel, hide, rule, lock, prompt-test, settings, or rollback action. 6. The system validates permission, target, current job state, safety closure, rule data, limits, and expected settings version. 7. The backend records the reason and applies the mutation; settings rollback creates a new active revision rather than rewriting history. 8. The interface refreshes the affected summary, list, media, policy, lock, or settings state.
Subflows: S1 - Job operation: retry only eligible failed work, retry video only with a valid image result, cancel active work, or hide an eligible stored job. S2 - Policy management: review violations and create, edit, or delete additional prohibited-description rules. S3 - Account restriction: lock a customer's try-on access with a reason or remove the current lock. S4 - Settings: test the production prompt policy, update provider/feature/limit settings using an expected version, or roll back by creating a controlled new revision. S5 - Policy-closed failure: block ordinary retry when safety policy requires a closed terminal outcome.

Alternate/Exception Flow:

AL-01 - Empty jobs/violations/locks: show an explicit empty operational state.

EX-01 - Insufficient read/manage/settings permission: deny only the affected tab or mutation.

EX-02 - Ineligible or stale job: reject retry, video retry, cancel, or hide after the state has changed.

EX-03 - Invalid/duplicate prompt rule or unknown customer: reject the mutation without partial policy or lock data.

EX-04 - Concurrent settings update or invalid limit/provider: reject the stale revision and reload current settings.

EX-05 - Provider operation failure: retain confirmed job/settings state and display the recorded error and retry eligibility.

1.3. Non-functional Requirements

1.3.1. Performance Requirements

- Response Time: Common system requests (browsing products, viewing details, cart updates) should complete within 3 seconds under normal network conditions. Heavy operations, such as Virtual Fitting Room image and video generation, should run asynchronously in the background with real-time progress updates.
- • Transaction Processing: Operations such as checkout, payment processing, and order creation should be processed quickly, securely, and accurately.
- • Scalability: The system architecture should support future growth in users, product catalogs, orders, recommendation logs, and fitting room job requests.
- • Load Capacity: The system should support multiple concurrent mobile application users and web admin staff without noticeable performance degradation.

1.3.2. Safety Requirements

- Data Backup: System data and database records should be backed up regularly before maintenance, upgrades, or major configuration changes.
- Data Integrity: The system must maintain data accuracy and operational stability as product data, user interaction logs, and try-on history volumes grow.
- Content Validation: Personal photos uploaded to the Virtual Fitting Room must be validated (file format, image size, single-person pose check, safety policies) to prevent invalid uploads or malicious files.
- System Protection: The mobile application, web admin panel, and backend services must be protected against malicious files, unsafe links, and unauthorized API requests.

1.3.3. Operational Requirements

- **Connectivity:** The mobile app and web admin panel require an active Internet connection to communicate with backend APIs and cloud services.
- **Request Processing:** The system should process user requests accurately, provide real-time updates on background tasks, and display clear feedback messages.
- **User Interface:** The user interface should present information clearly, supporting touch interactions on mobile devices and desktop navigation in web browsers.
- **Fault Tolerance:** The system should handle software errors gracefully without crashing the mobile app or significantly disrupting the user experience.

1.3.4. Interface Requirements

- **Mobile UI Design:** The mobile application interface should be modern, clean, visually appealing, and optimized for mobile fashion shopping.
- **Web Admin UI Design:** The Web Admin Panel should feature a structured, intuitive dashboard designed for efficient store management by admins and staff.
- **Responsiveness:** The mobile app must adapt cleanly to different mobile screen resolutions (Android), while the Web Admin Panel must display correctly on standard desktop monitors.
- **Smooth Interaction:** Screen transitions, touch gestures on mobile, and table navigation on web admin should be smooth and responsive.

1.3.5. Security Requirements

- **Data Protection:** Sensitive data, including passwords, authentication tokens, payment details, and personal information, must be encrypted and protected. Passwords shall never be stored or displayed in plain text.
- **Authentication and Authorization:** The system enforces Role-Based Access Control (RBAC), separating Customers, Staff, and Administrators. Customers access mobile features, while Staff and Admins access Web Admin management functions according to their granted permissions.

1.4. Operating Environment

- **Client Side (Mobile Application):** Smartphones running Android with an active Internet connection.
- **Client Side (Web Admin Panel):** Personal computers (Windows, macOS, or Linux) running modern web browsers (Google Chrome, Microsoft Edge, Firefox, or Safari).
- **Server Side:** A backend server environment running API services, database storage (MongoDB), Virtual Fitting Room job processors, Product

Recommendation engines, cloud media storage (Cloudinary), and real-time communication servers (Socket.IO).

1.5. Implementation and Design Constraints

1.5.1. Implementation Constraints

- The system consists of a mobile application for customers and a web-based administration system for administrators and staff.
- The system is developed using the technologies and tools defined within the scope of the thesis.
- System components and third-party services are integrated through RESTful APIs and Socket.IO communication.
- Payment, media storage, shipping, and virtual try-on functions depend on the technical requirements and limitations of external services.
- Virtual try-on results depend on the quality of input images and the capabilities of the selected AI models and hardware.

1.5.2. Design Constraints

- The system follows a modular client–server architecture that separates user interfaces, business logic, data storage, recommendation processing, and virtual try-on services.
- The interfaces must be consistent, user-friendly, and suitable for mobile and web users.
- Administration functions must be controlled according to the roles and permissions of administrators and staff.
- The database must support fashion products, variants, inventory, user interactions, orders, recommendations, and virtual try-on records.
- System components should be modular and maintainable to support future improvements.

CHAPTER 2. THEORETICAL BACKGROUND

This chapter outlines the theoretical background, foundational concepts, and core technologies used to design, implement, and evaluate the proposed fashion e-commerce system. Building a modern mobile fashion platform that integrates virtual fitting capabilities and personalized product recommendations requires a strong foundation in mobile software engineering, recommendation algorithms, computer vision, generative artificial intelligence (AI), and systematic software testing.

2.1. Mobile Fashion E-commerce and System Architecture

- Definitions, Significance, and Standard Workflow: E-commerce encompasses the production, distribution, marketing, sale, or delivery of goods and services via electronic means. A mobile fashion workflow typically includes discovery, variant selection, cart, checkout, payment, fulfillment, and post-purchase support. A client-server architecture separates the mobile interface from business logic, data, and external services.[7]
- State-of-the-art Tools & Frameworks: React Native 0.81 [8], Expo SDK 54 [9], TypeScript , Express 5, MongoDB [10], Cloudinary, RESTful HTTP APIs, Socket.IO [11], Redis [12], BullMQ, and Docker Compose [13].

2.2. Product Recommendation Systems

2.2.1. Recommendation principles

The module is a context-aware weighted hybrid recommender. Context determines the ranking intent: Home models accumulated user or session preference, Product Detail retrieves similar items, and Shopping Cart estimates outfit-completion value. This separation aligns with context-aware and hybrid recommendation principles. [1], [14]; content evidence supports similarity [15], while fashion compatibility motivates the Cart context [16].

CONTEXT-AWARE HYBRID RANKING

Combines behavioral preference, content similarity, and outfit compatibility.
Popularity & merchandising are fallback signals.

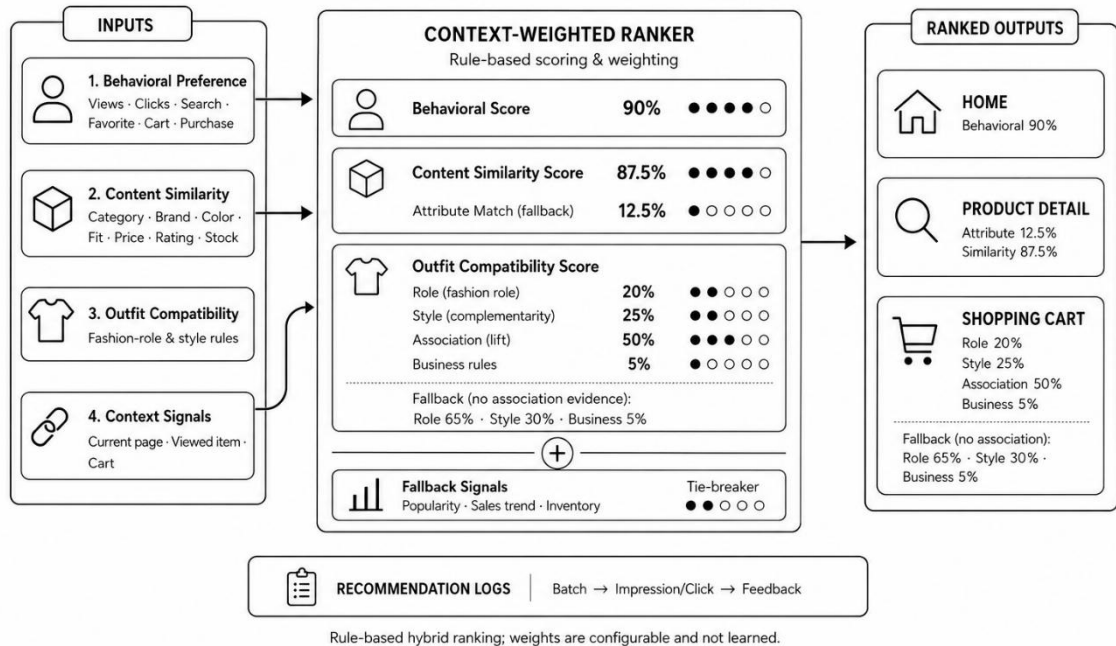


Figure 2.1 Conceptual architecture of the context-aware hybrid ranker.

2.2.2. Signals and evaluation datasets

Preference evidence comes from implicit events: views, clicks, searches, favorites, recommendation clicks, virtual try-ons, add-to-carts, and purchases. Stronger actions receive higher base weights, and older events receive a smaller recency multiplier. This follows confidence-weighted implicit feedback [17] and time-aware weighting [18], while the concrete values below are project-specific.

Table 2.1 Base interaction weights for recommendation events

Tracked action	Base weight	Interpretation
view, click	1	Passive and therefore weak evidence.
search, search_result_click	2	Active exploration of a need.
favorite, recommendation_click	3	Explicit positive interest.
try_on	4	Additional effort spent evaluating the product.
add_to_cart	5	Strong purchase intention.
purchase	10	Confirmed transaction outcome.

Time-aware recommendation approaches typically assign greater weight to recent interactions because user preferences may change over time [18]. Based on this principle, the effective weight of interaction event i is defined in this study as:

$$w_i = \omega(a_i) \cdot d(\Delta t_i) \quad (2.1)$$

Here, a_i is the action type, $\omega(a_i)$ is its base weight from the table, Δt_i is the event age in days, and $d(\Delta t_i)$ is the recency factor. The current implementation uses $d = 1.0$ for fewer than 7 days, 0.7 for 7-29 days, 0.4 for 30-89 days, and 0.2 for 90 days or more. Repeated views are also subject to a 45-minute cooldown.

$$d(\Delta t_i) = \begin{cases} 1.0 & 0 \leq \Delta t_i < 7 \\ 0.7 & 7 \leq \Delta t_i < 30 \\ 0.4 & 30 \leq \Delta t_i < 90 \\ 0.2 & \Delta t_i \geq 90 \end{cases} \quad (2.2)$$

The weighting scheme in Equations (2.1)-(2.2) is a project-specific design motivated by implicit-feedback confidence [17] and time-aware recommendation [18]. The action weights in Table 2.1, the recency thresholds of 7, 30, and 90 days, and the corresponding multipliers are configuration choices of this system.

Offline formula selection uses the Amazon Reviews 2023 - Amazon Fashion controlled subset for Home and Product Detail, and the Polyvore Dataset for Learning Fashion Compatibility with Bidirectional LSTMs for Cart outfit completion [19]. Appendix E lists dataset provenance, exact links, and evaluation notebooks.

2.2.3. Deployed scoring functions

All signals are normalized to [0,1]. $P(u, i)$ is the preference match across category, brand, color, gender, and price bucket; $A(q, i)$ is the weighted attribute similarity across category, gender, brand, color, fit, and price; $C(q, i)$ is the cosine similarity of a binary metadata vector; $R(c, i)$ and $T(c, i)$ are the Cart role and style compatibility; $L(c, i)$ is the normalized association lift; and $B(i)$ is the sale/new-arrival business score.

The following scoring functions are project-specific weighted combinations motivated by hybrid and context-aware recommendation principles [1], [14]. Content-based recommendation motivates the similarity signals [15], while fashion compatibility motivates the Cart context [16], [19]. Coefficient selection and the distinction between validation-selected and production configurations are described in Section 3.6 and Table 4.3.

For Home, behavioral preference remains dominant while a small business term preserves current sale and new-arrival priorities:

$$S_{\text{home}}(u, i) = 0.90P(u, i) + 0.10B(i) \quad (2.3)$$

For Product Detail, grid search combines the explainable weighted-attribute score with the independent cosine signal [15]:

$$S_{\text{detail}}(q, i) = 0.125A(q, i) + 0.875C(q, i) \quad (2.4)$$

For Shopping Cart, the deployed score combines fashion-domain rules, training-only association evidence, and a small business term [19]:

$$S_{\text{cart}}(c, i) = 0.20R(c, i) + 0.25T(c, i) + 0.50L(c, i) + 0.05B(i) \quad (2.5)$$

If training baskets provide no eligible association evidence, Cart uses the rule-only fallback $0.65R + 0.30T + 0.05B$. Amazon Fashion does not expose an equivalent business signal, so that term is set to zero during offline tuning only. The formula-selection procedure is presented in Section 3.6, comparison results in Section 4.7, and auxiliary equations in Appendix E.

2.2.4. Interpretation and scope

The deployed module is deterministic and explainable, unlike a newly trained recommendation model. Its contribution is the context-specific fusion of behavioral, metadata, fashion-rule, association, and business signals. Offline ranking evidence supports coefficient selection but does not, by itself, establish gains in click-through, conversion, revenue, or satisfaction.

2.3. AI-Based Virtual Try-On System

2.3.1. Scope and Technical Position of the VTO Module

A virtual fitting room is useful only when the generated image resolves the customer's practical question: how would the selected items look on this person in this photograph? Consequently, the problem extends beyond producing an aesthetically pleasing image. The system must preserve the source person's visible identity, pose, body proportions, skin tone, and camera framing while keeping the selected catalog garments recognizable. This requirement defines the complete processing pipeline described in this section.

This thesis integrates generative models instead of training or fine-tuning them. It transforms diverse inputs like customer photos, catalog images, product metadata, outfit roles, and scene requests into a validated asynchronous job for safe mobile app use. The pipeline includes image validation, garment prep, prompt construction, workflow mapping, execution, media saving, progress updates, and admin controls.

The current image workflow is executed through ComfyUI with the model identifier `gemini-3-pro-image-preview`; the corresponding node is labeled Nano Banana Pro (Google Gemini Image). When a short video is requested, a successful try-on image becomes the first frame of a second ComfyUI workflow using `klimg-v3-omni` (Kling 3.0 Omni image-to-video). Because provider identifiers remain configuration values, the model can later be replaced without changing the mobile request contract or the persisted job model.

2.3.2. Inputs, Outputs, and Constraints

Before discussing the models, it is necessary to identify what enters and leaves the system clearly. A VTO request combines customer-supplied data, authoritative catalog data resolved by the backend, information derived during validation, and provider configuration. Separating these groups prevents the prompt from becoming the source of business truth: product identity, color, variant, size, and image references always come from the catalog and persisted assets.

Table 2.2 Runtime inputs and outputs of the VTO pipeline

Group	Field	Required	Use in the pipeline
Person image	sourceAssetId	Yes	References one active customer-owned upload or camera asset. Its URL is snapshotted into the job.
Selection	outfitMode	Yes	single, top_bottom, or full_set; determines compatibility checks and prompt blocks.
Catalog items	selectedItems[1..4]	Yes	Each item contains productId, variantId, colorVariantId, optional size, and one role: top, bottom, dress, shoes, outerwear, or accessory.
Scene	contextPreset	Optional	none, work, casual, party, travel, sport, date, or custom; controls only background/occasion context.
Scene	contextPrompt	Optional	Maximum 200 characters; validated by prompt policy and prohibited from overriding garment references.
Output	outputMode	Optional	image or image_and_video.
Video	videoDurationSeconds	Conditional	Required for video output; accepted range is 5-12 seconds.
Request	Idempotency-Key	Recommended	Prevents duplicate job creation for the same customer request.
Derived	sourceImageProfile	System	Visible regions, supported modes, recommended mode, body visibility, and validation reason.
Generated output	image/video assets	System	Up to four persisted 3:4 image results and one optional MP4 video, plus status, progress, timing, and provider metadata.

2.3.3. Models and Responsibilities

No single model performs the entire function. Instead, the system assigns a narrow responsibility to each component so it can detect errors before submitting an expensive generation request. The validation and segmentation models establish whether the inputs are usable; only the final generative model synthesizes the try-on result.

Table 2.3 Models and techniques used by the implemented system

Stage	Model/technique	Exact configuration	Responsibility
Image quality	OpenCV	Laplacian variance and grayscale mean	Checks resolution, blur, and brightness before provider submission.
Person/pose	YOLO11 Pose	yolo11n-pose.pt; default inference size 960	Detects the principal person, pose keypoints, bounding-box coverage, and visible upper/hip/leg/foot regions.
Garment detection	Grounding DINO	IDEA-Research/grounding-dino-tiny	Uses the selected garment role as a text-guided query to propose the relevant catalog-image box.
Garment segmentation	SAM 2.1	facebook/sam2.1-hiera-tiny	Refines and validates the detected garment region. The mask determines a rectangular crop; the returned crop keeps original pixels and texture.
Image synthesis	Google Gemini Image through ComfyUI	gemini-3-pro-image-preview; workflow title Nano Banana Pro	Combines the person and selected garment references, applies the prompt, and generates a 2x2 try-on grid that is cropped into four images.
Video synthesis	Kling 3.0 Omni through ComfyUI	kling-v3-omni, 720p, 5-12 s, audio disabled	Animates one successful try-on image while preserving identity, outfit, background, lighting, and framing.

This division of responsibility is important for interpreting the result. OpenCV, YOLO11 Pose, Grounding DINO, and SAM 2.1 are preprocessing and validation components; they do not synthesize the final try-on image. Gemini performs image generation, whereas Kling is invoked only after at least one usable image result is available. Therefore, a video failure does not invalidate a successful image job.

2.4. Software Testing and System Evaluation

- Definitions, Significance, and Standard Workflow: Software testing verifies whether a system meets its functional and non-functional requirements. The workflow includes test planning, test-case and test-data preparation, unit testing, integration testing, end-to-end testing, Android device testing, and result recording and defect analysis. This thesis also evaluates recommendation quality and virtual try-on performance. [20]

- State-of-the-art Tools & Frameworks: Jest and ts-jest for backend testing; Jest and jest-expo for mobile logic; pytest for image-validation and garment-processing services; Android emulators or physical devices for complete workflow testing; system logs and evaluation scripts for recommendation and virtual try-on analysis.

CHAPTER 3. DESIGN AND IMPLEMENTATION

3.1. System Overview

The project aims to develop an intelligent fashion e-commerce system that delivers a personalized shopping experience for customers while enabling administrators and staff to manage store operations efficiently.

The technology stack includes a customer-facing mobile application built with React Native and Expo, and an administration website built with React, TypeScript, and Vite. The backend server uses Node.js, Express, TypeScript, and Mongoose, with MongoDB Atlas storing and managing system data. Image and video generation capabilities are implemented through ComfyUI and exposed to the backend via a FastAPI-based service.

The system follows a client-server architecture. Customers primarily interact through the mobile application, while operational staff use the web-based administration portal. Both clients communicate with the backend server via HTTP-based APIs. The backend handles authentication, product and inventory management, orders, payments, promotions, customer information, recommendations, and other e-commerce business logic.

The backend communicates with MongoDB Atlas via Mongoose to store and retrieve system data. It also integrates with several third-party services, including Cloudinary for media storage and management, VNPAY for online payments, GHN for shipping and delivery services, and Firebase for authentication, notifications, and other mobile-related functionality. The system processes responses and asynchronous updates from these services via APIs and webhooks.

The system serves four main user groups: guests, customers, administrators, and operational staff. Guests can browse products and view basic product information without signing in. Customers can manage their accounts, explore products, receive personalized recommendations, add items to their carts, place orders, make payments, and track delivery progress. Administrators manage the overall system, including products, categories, inventory, users, orders, promotions, permissions, and reports. Operational staff assist with order processing, inventory management, customer support, and other tasks according to the permissions assigned by administrators.

One of the system's key advanced features is the virtual styling room. Customers can select fashion products and generate visual outfit representations to preview how different items can be combined. The backend sends requests to the

ComfyUI image and video processing workflow via the FastAPI service and receives the results via API responses or webhooks.

The system also provides intelligent product recommendations based on customer preferences, behaviors, interactions, and selected fashion items. These recommendations help customers discover relevant products, create more suitable outfits, and reduce the time spent searching through the product catalog. Together, the virtual styling room and recommendation features deliver a better interactive, personalized, and engaging fashion shopping experience.

3.2. System Architecture

Figure 3.1 presents the system as a layered client-server architecture in which the mobile application and Web Administration Portal share a common backend. This arrangement keeps presentation concerns in the client applications while centralizing authentication, authorization, validation, business rules, and external-service coordination in the server layer.

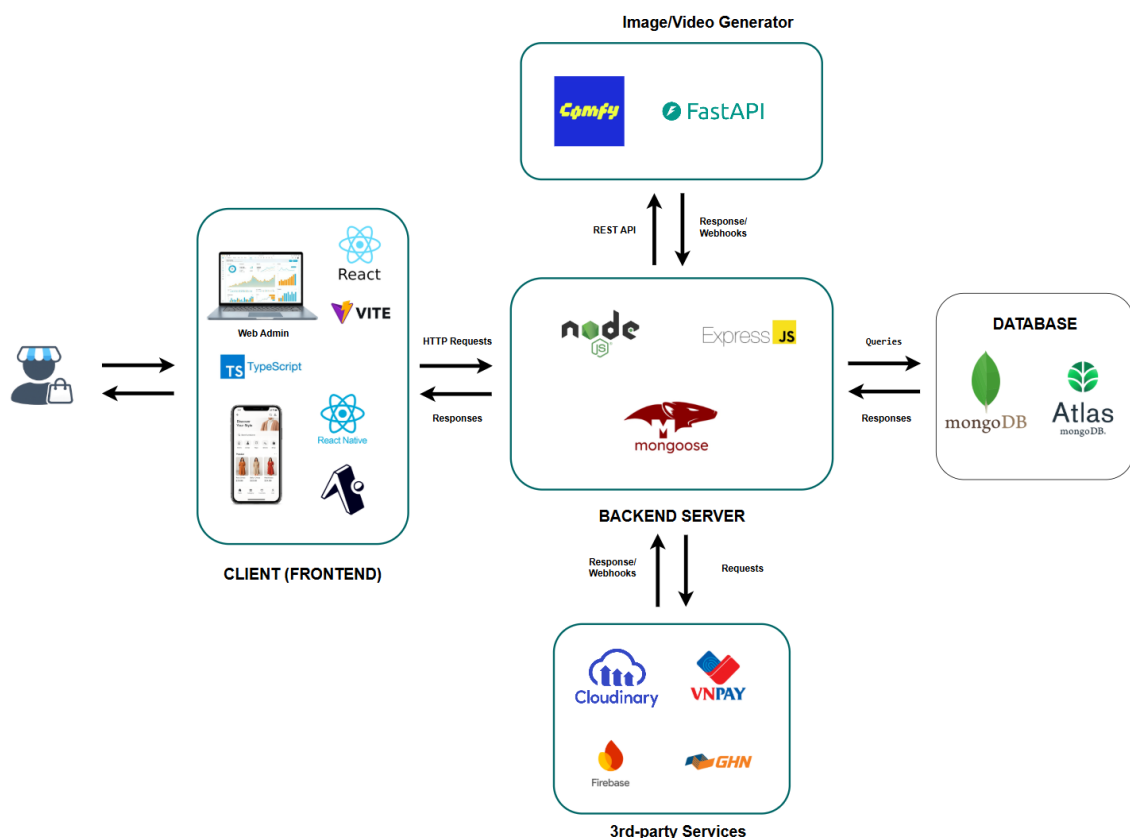


Figure 3.1 Overall system architecture

At the presentation layer, the React Native mobile application supports the customer shopping process, while the React and Vite web application provides permission-controlled operational interfaces for administrators and staff. Both clients exchange HTTP requests and responses with the Node.js and Express backend. For long-running virtual try-on operations, the backend also publishes asynchronous progress and result updates so clients can continue to display the latest job state without coupling the user interface to the generation provider.

The backend serves as the system's integration boundary. Its functional modules enforce domain rules and use Mongoose to persist data in MongoDB Atlas. Media objects are stored in Cloudinary, VNPay handles online payment flows, GHN provides shipping services, and Firebase supports mobile-oriented services such as notifications. Image and video generation is isolated behind a FastAPI service that submits configured workflows to ComfyUI and returns results via REST responses or webhooks. This separation allows each external integration to be replaced, tested, or degraded independently while the core shopping workflow remains consistent.

3.3. System Decomposition

The decomposition diagram refines the architecture from the perspective of the four principal actor groups. Rather than treating the system as a single application, it separates functions that can be used anonymously from those that require a customer account or an authorized operational role.

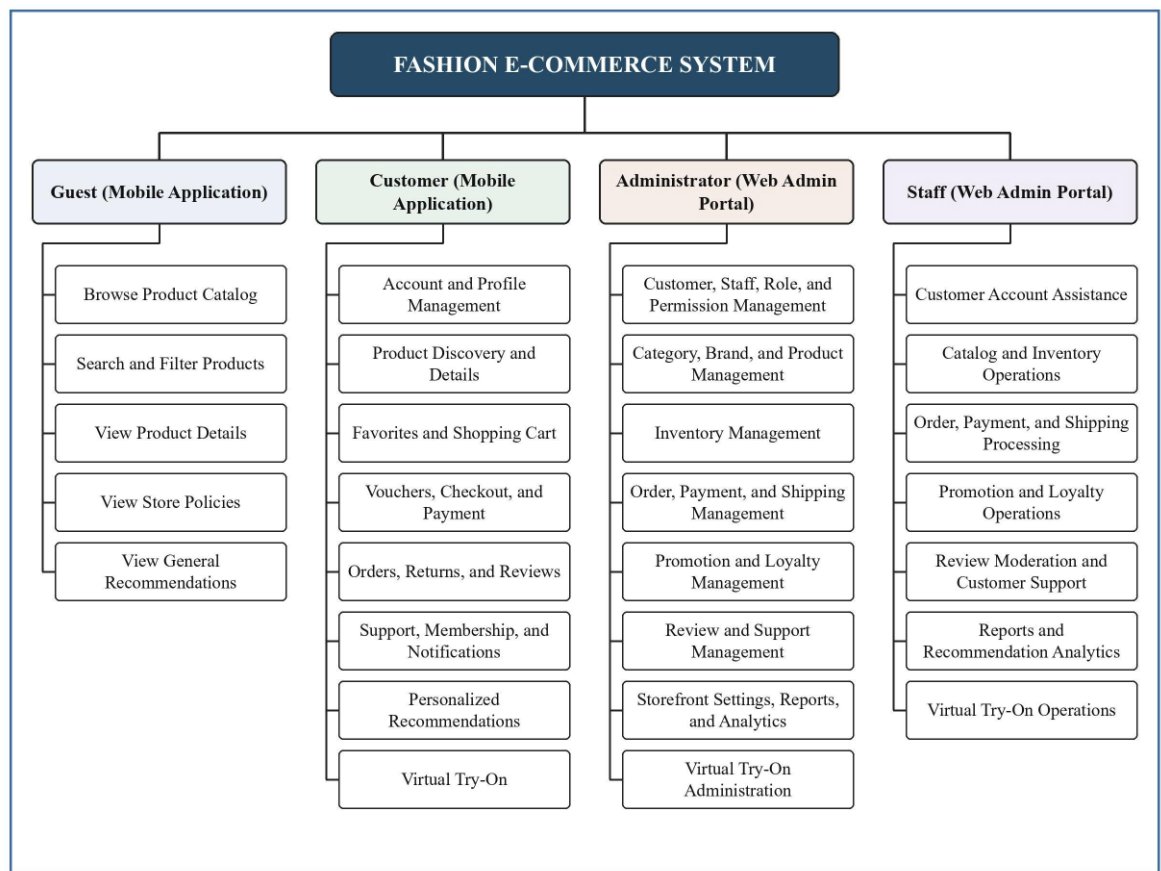


Figure 3.2 Decomposition diagram

Guests are limited to product discovery, product details, public policies, and general recommendations. After authentication, customers gain access to profile management, favorites, cart and checkout, order and return tracking, reviews, support, membership information, notifications, personalized recommendations, and virtual try-on. Administrators control customer and staff accounts, permissions, catalog data, inventory, orders, promotions, loyalty programs, support, reporting, and intelligent feature configuration. Staff members use the same administration portal, but the available functions are constrained by the permissions assigned to their accounts.

Although actor groups the functions in the diagram, they reuse common backend modules and persistent collections. For example, product and inventory services support browsing, checkout, order fulfillment, recommendations, and virtual try-on, while the account and authorization services protect both customer and administrative workflows. This modular decomposition reduces duplicated logic and provides a clear boundary for testing each business capability.

3.4. Data Design

The persistence layer uses MongoDB because the system combines stable business entities with document-shaped data that varies by workflow. Mongoose schemas define stored attributes, validation constraints, references, indexes, default values, timestamps, and embedded subdocuments. The collection relationship diagram therefore focuses on the business core: users, products, categories, brands, carts, orders, transactions, inventory, stock receipts and reservations, reviews, and virtual try-on records.

References are used when a record has an independent lifecycle or must be queried across domains. Typical examples include links from an order to a user, from a product to its category and brand, and from a transaction to its order. Embedded documents are used when the data belongs to the parent aggregate and should be read or updated with it. Product variants and colors, cart items, order-item snapshots, shipping addresses, return information, support context, and virtual try-on selections follow this pattern.

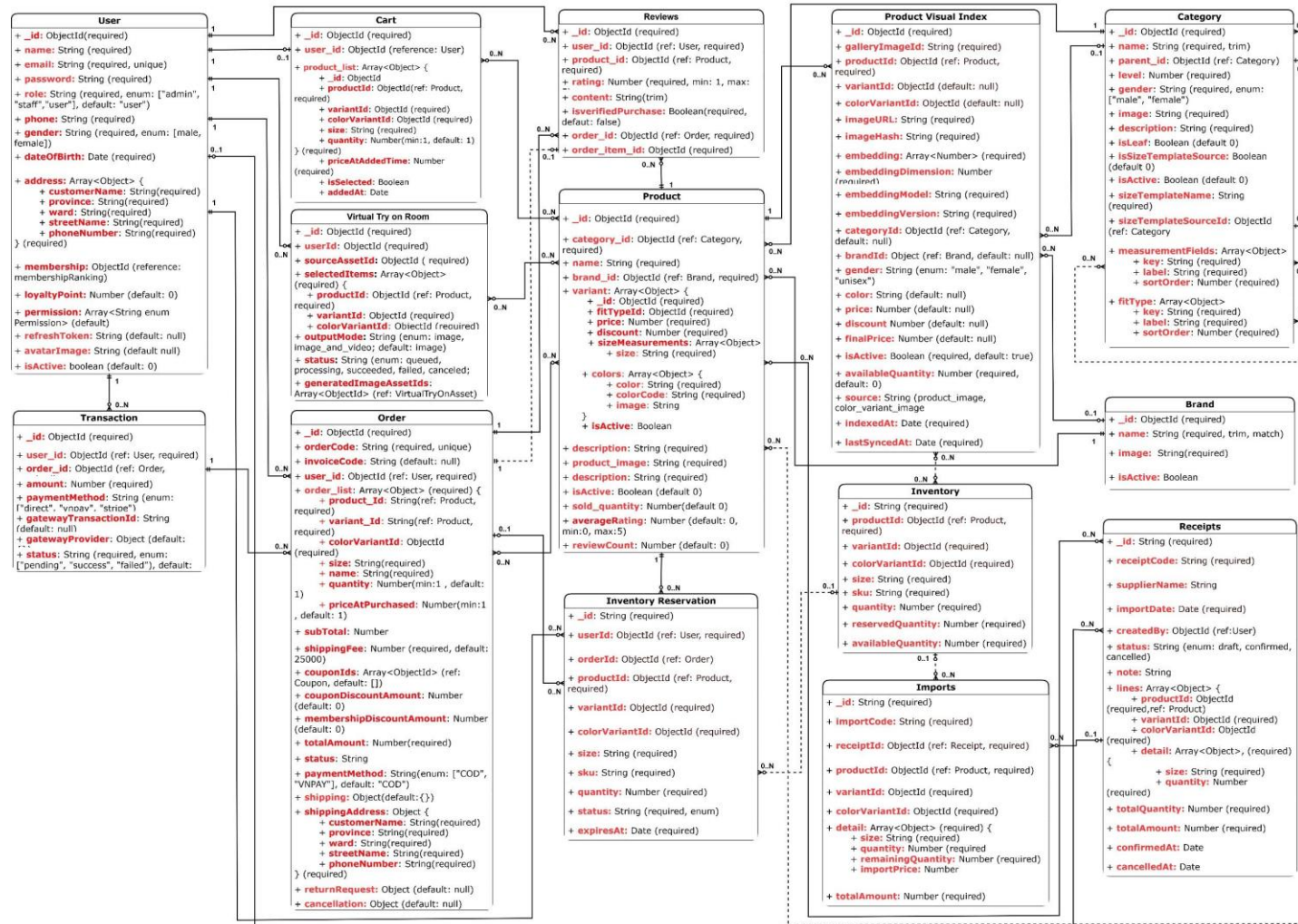


Figure 3.3 MongoDB collection relationship diagram

The user collection stores identity, profile, address, permission, membership, and consent data. The catalog is organized by category and brand references, and each product embeds the variants, colors, size measurements, and pricing information required by the customer interfaces. Inventory records use the product identifier along with embedded variant and color identifiers so that stock can be controlled at the sellable SKU level.

The order collection serves as the principal commercial aggregate. It embeds snapshots of purchased items, addresses, shipping, loyalty rules, returns, and cancellations so that later catalog or configuration changes do not alter the historical meaning of a completed transaction. Payment transactions, coupon usage, inventory reservations, customer reviews, and support records remain separate collections because they have their own status transitions, audit requirements, or query patterns.

3.5. Functional Design

3.5.1. Sign In

- **Purpose:** Authenticate a customer and establish a secure application session.
- **Users:** Registered customers with an active account.
- **Interface:**

The image shows a mobile application sign-in screen. At the top, there is a blue header with a back arrow and the 'D SHOP' logo. The main title is 'ĐĂNG NHẬP TÀI KHOẢN' (Sign In Account). Below this, there are two input fields: 'Số điện thoại hoặc Email' (Phone number or Email) and 'Mật khẩu' (Password). The first field has a placeholder 'Nhập số điện thoại hoặc email'. The second field has a placeholder 'Nhập mật khẩu' and a toggle icon. Below the password field is a link 'Quên mật khẩu?' (Forgot password?). At the bottom, there is a large black button labeled 'Đăng nhập' (Sign In). Below the button, there is a link 'Chưa có tài khoản? Đăng ký ngay' (Don't have an account? Sign up now).

Figure 3.4 Mobile sign-in interface

Table 3.1 summarizes the main interface components.

Table 3.1 Main interface components for sign-in

No.	Control	Purpose	Default value
1	Text input	Enter email address or phone number	Empty

No.	Control	Purpose	Default value
2	Password input	Enter and show or hide the password	Hidden
3	Link	Open password recovery or registration	-
4	Button	Submit credentials	Sign In
5	OTP panel	Request and verify an unlock code after temporary lockout	Hidden until needed

Table 3.2 summarizes the principal data used by the function.

Table 3.2 Data used by sign-in

No.	Data/model	Operations	Role in the function
1	User	Read, update	Credentials, role, active state, profile state, token version
2	LoginAttempt	Create, read, update, delete	Failed-attempt count and temporary lock
3	OtpVerification	Create, read, update	Login-unlock verification

Processing: The app validates the identifier and password, and the backend verifies the account, active status, lock status, and password hash. A successful request returns access and refresh tokens and routes the customer to profile completion or the home screen. Failed attempts are recorded; a temporarily locked account can be unlocked with a six-digit OTP.

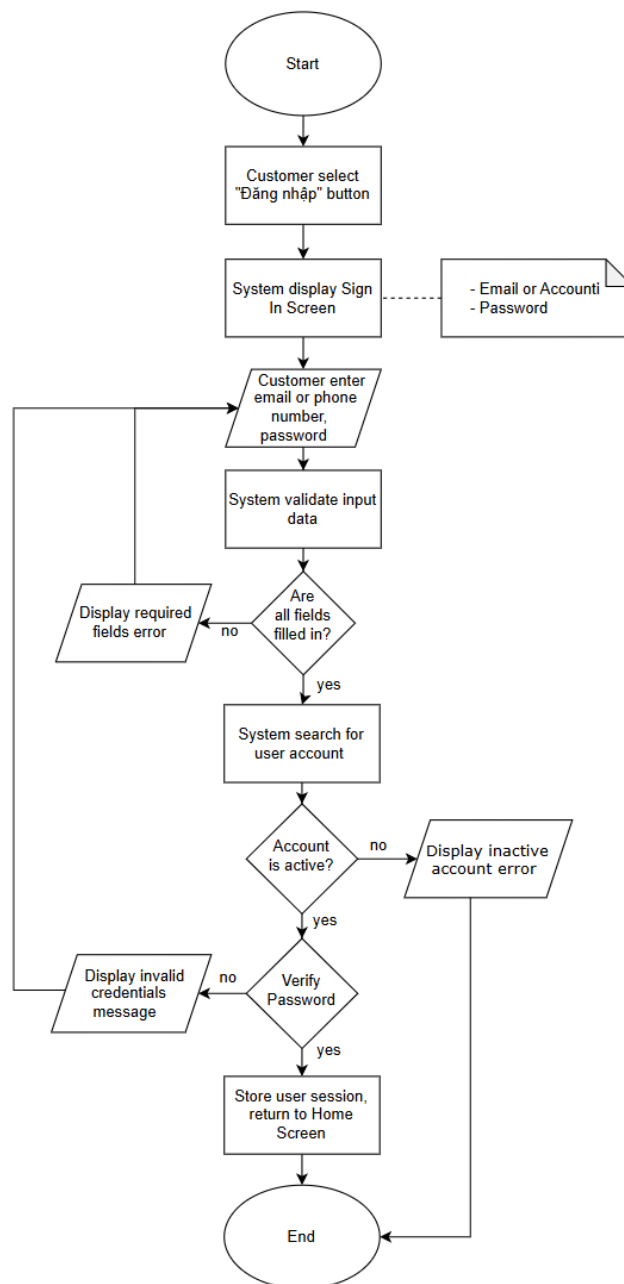


Figure 3.5 Mobile sign-in processing flow

3.5.2. Sign Up

- **Purpose:** Create a customer account with verified contact and delivery-profile information.
- **Users:** Customers who do not yet have an account.
- **Interface:**

Figure 3.6 Mobile account-registration interface

Table 3.3 summarizes the main interface components.

Table 3.3 Main interface components for sign up

No.	Control	Purpose	Default value
1	Text input	Enter name, phone, email, street address, and password	Empty
2	OTP controls	Send and verify the phone OTP	Unverified
3	Select	Choose gender, date of birth, province, and ward	Not selected
4	Checkbox	Accept the terms of use and privacy policy	Cleared
5	Button	Create the account	Sign Up

Table 3.4 summarizes the principal data used by the function.

Table 3.4 Data used by sign up

No.	Data/model	Operations	Role in the function
1	User	Create, read	Identity, contact, address, password hash, and customer profile
2	OtpVerification	Create, read, update	Phone ownership verification
3	Location data	Read	Province and ward options and codes

Processing: The customer requests and verifies a phone OTP, completes the required fields, accepts the policies, and submits the form. The app validates the password confirmation and the age range (13-100 years). The backend checks uniqueness and the OTP token, creates the user and address, hashes the password, and returns the new session.

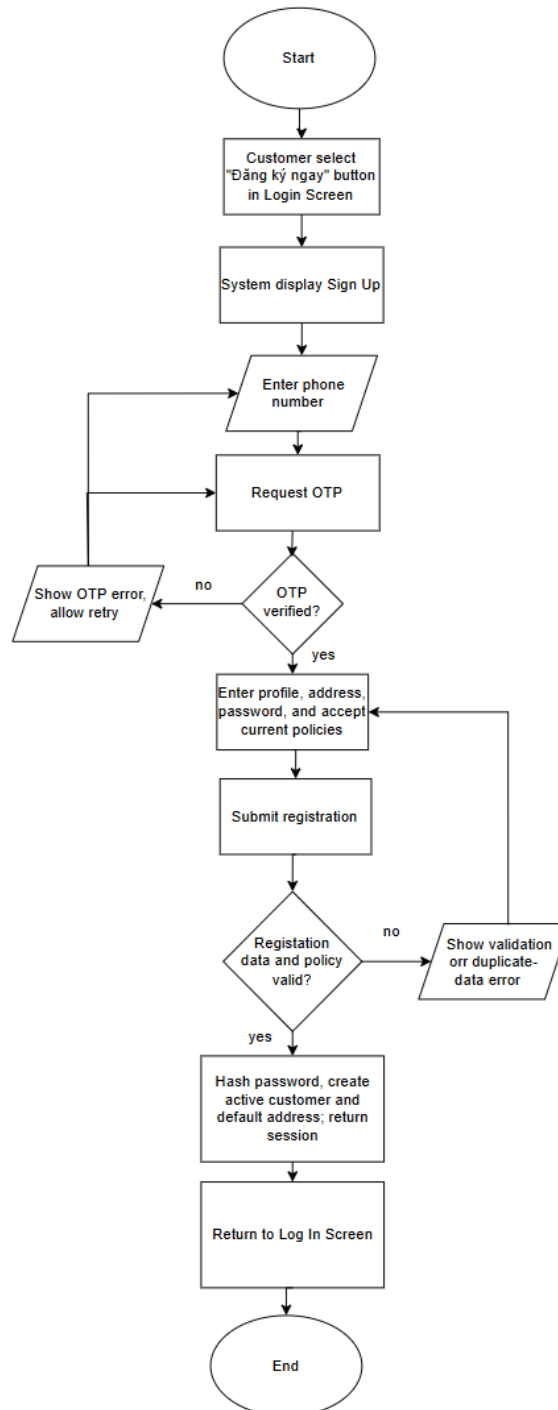


Figure 3.7 Mobile account-registration processing flow

3.5.3. View Product List

- **Purpose:** Help customers discover available products and open the desired product quickly.
- **Users:** Guests and signed-in customers.
- **Interface:**

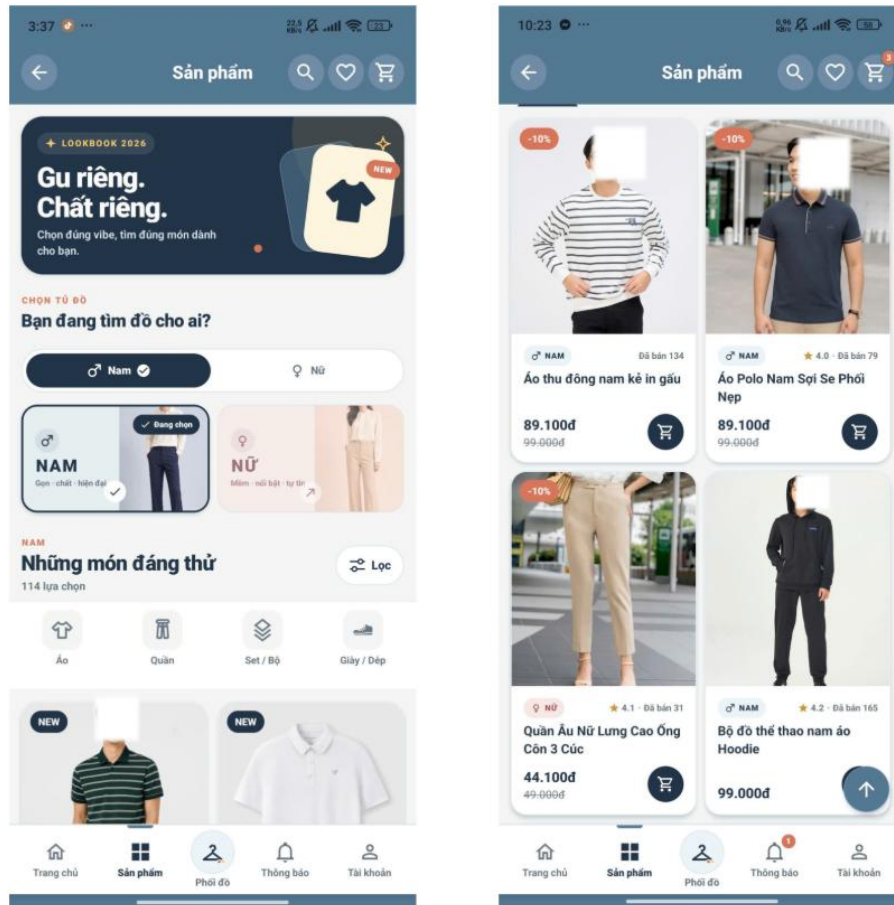


Figure 3.8 Mobile product-discovery and product-list interface

Table 3.5 summarizes the main interface components.

Table 3.5 Main interface components for viewing product list

No.	Control	Purpose	Default value
1	Search	Open product search	Empty
2	Gender/category controls	Select discovery scope	All
3	Product grid	Show image, name, price, discount, and rating	Newest products
4	Product card actions	Open details, favorites, or cart guidance	-
5	Pagination	Load additional results	Page 1

Table 3.6 summarizes the principal data used by the function.

Table 3.6 Data used by view product list

No.	Data/model	Operations	Role in the function
1	Product	Read	Product card, price, sale, new, and availability data
2	Category; Brand	Read	Discovery grouping and product labels
3	Inventory	Read	Variant availability
4	Review	Read	Average rating and review count

Processing: The client sends paging, sort, keyword, gender, category, and other active filters. The backend returns only eligible products, resolves effective prices and availability, and includes filter metadata. The app appends unique results and records product-list or search-result interactions when a card is opened.

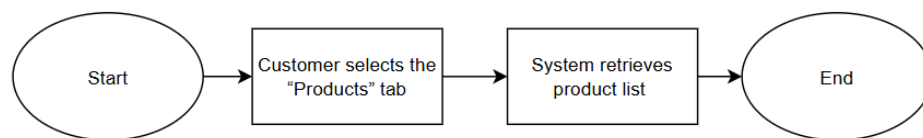


Figure 3.9 Mobile view product list flow

3.5.4. Search Product

- **Purpose:** Help guests and customers find relevant active products quickly while retaining a usable search path when suggestions are unavailable.
- **Users:** Guests and signed-in customers.
- **Interface:**

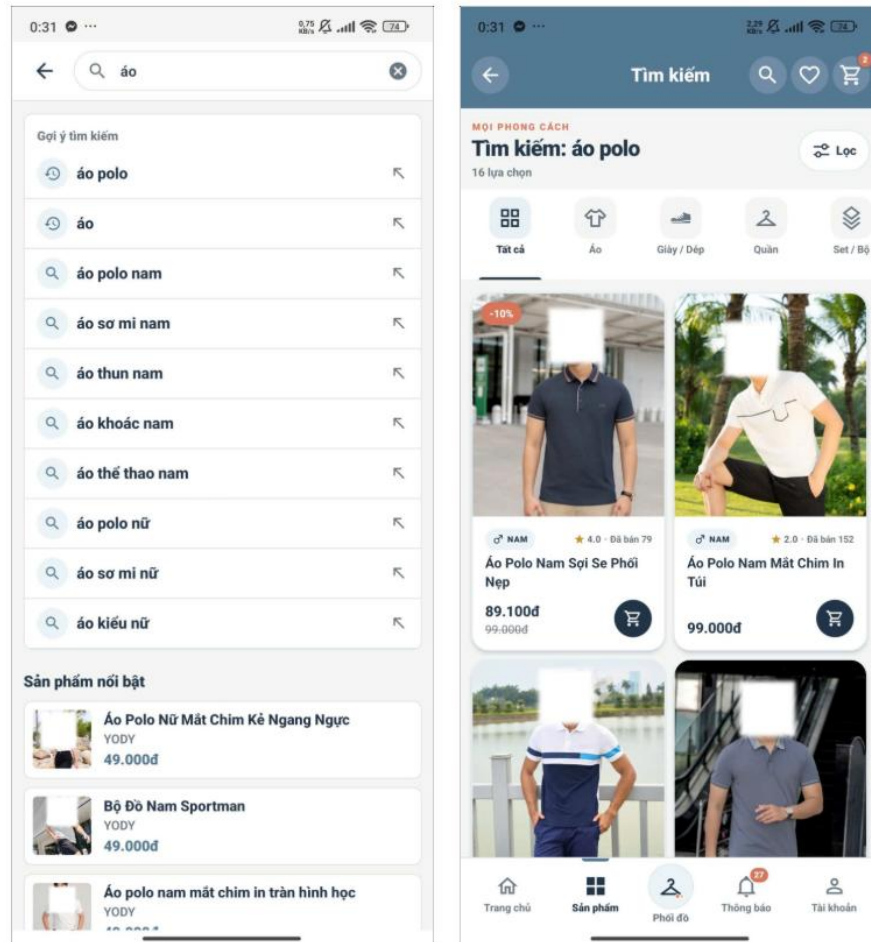


Figure 3.10 Mobile product-search suggestions and results interface

Table 3.7 summarizes the main interface components.

Table 3.7 Main interface components for search product

No.	Control	Purpose	Default value
1	Search input	Enter a keyword; submitting an empty value opens all active products	Empty and focused
2	History chips	Reuse, remove, or clear up to ten recent search terms	Local history; server history when signed in
3	Keyword suggestions	Select merged local/server suggestions after the minimum query length	Hidden until two characters
4	Suggested products	Preview price/brand/image and open Product Detail	Up to three shown from five returned
5	Suggested categories	Open Product List scoped to a matching category	Matching active categories

No.	Control	Purpose	Default value
6	Search and clear actions	Submit the current keyword or clear the input	Search icon; clear hidden when empty
7	Result list and filter entry	Show total, product cards, paging, filters, sorting, and retry/empty states.	Page 1 with current keyword

Table 3.8 summarizes the principal data used by the function.

Table 3.8 Data used by search product

No.	Data/model	Operations	Role in the function
1	Product	Read	Active suggestion and result cards, prices, discounts, images, sales/rating metadata
2	Category; Brand	Read	Suggestion matching, labels, gender inference, and filter metadata
3	Inventory	Read	Availability and purchasable variant information in result cards
4	SearchHistory	Create, read, update, delete	Keyword, source, session/user ownership, result count/products, deduplication, and sync
5	UserProductInteraction	Create	Search-result click telemetry used by analytics and recommendation signals.

Processing: Search starts locally with recent terms and syncs server history for signed-in users. At two characters, the client waits 300 ms, aborts stale work, and requests up to five product suggestions, categories, and keywords. The backend matches tokens, ignoring accents and material details; it may infer gender and briefly cache responses. Submitting a keyword logs an event, stores history, and opens Product List, applying variants, availability, filters, sorting, and pagination. Selections navigate directly; even on failure, exact search remains available.

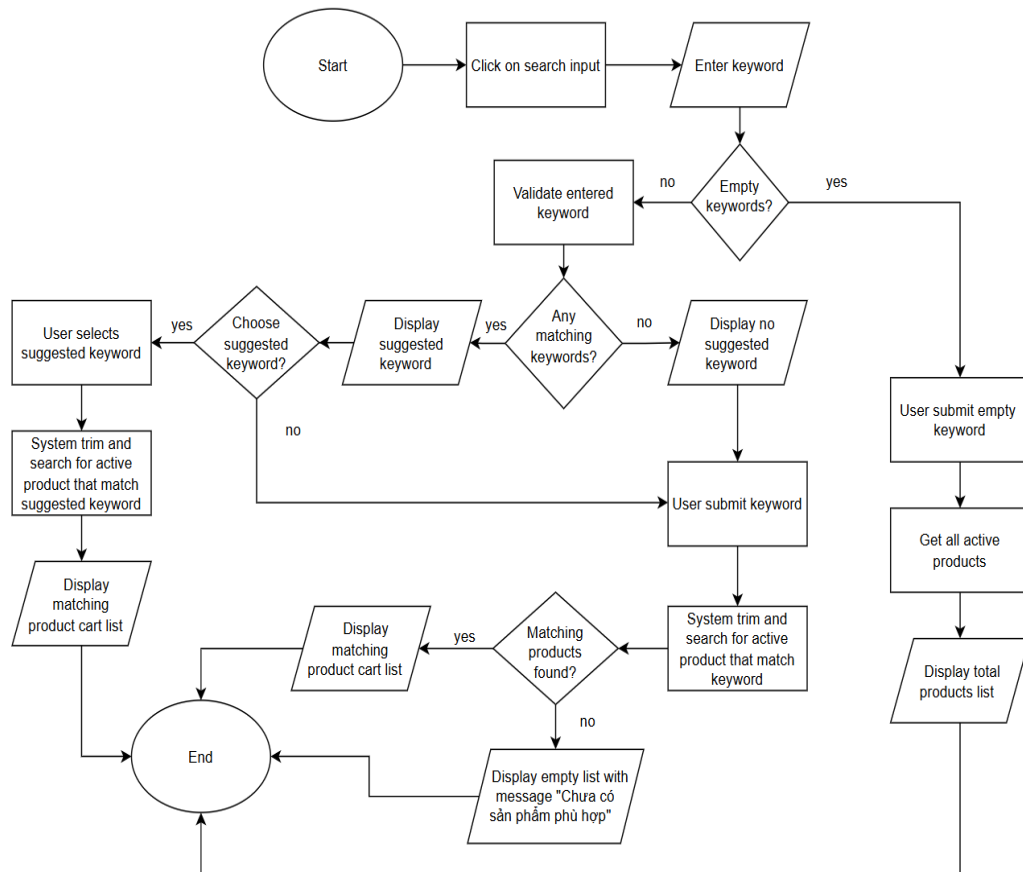


Figure 3.11 Mobile product-search and result-navigation flow

3.5.5. Filter Products

- **Purpose:** Narrow and order the catalog using product attributes and price conditions.
- **Users:** Guests and signed-in customers.
- **Interface:**

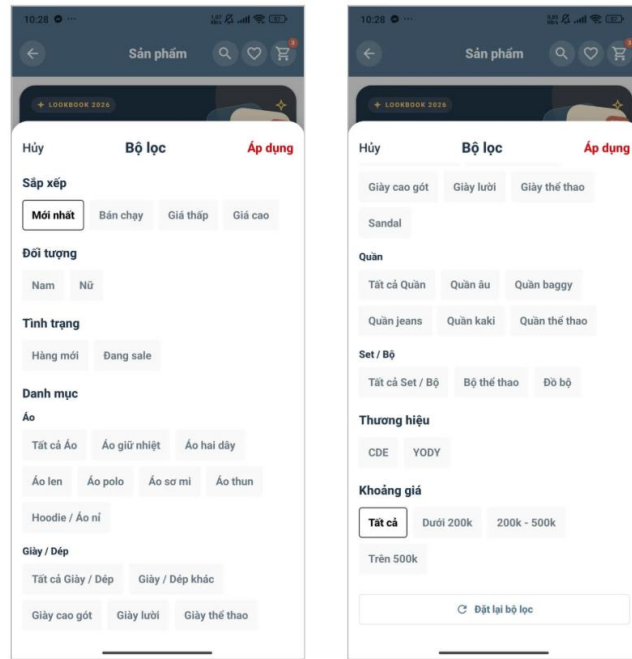


Figure 3.12 Mobile product-filter interface

Table 3.9 summarizes the main interface components.

Table 3.9 Main interface components for filter products

No.	Control	Purpose	Default value
1	Sort options	Choose newest, best-selling, rating, or price order	Newest
2	Choice chips	Filter gender, sale/new state, category, brand, color, fit, and size	All
3	Price presets	Limit the price range	All prices
4	Button	Clear or apply filters	Apply

Table 3.10 summarizes the principal data used by the function.

Table 3.10 Data used by filter products

No.	Data/model	Operations	Role in the function
1	Product	Read	Filterable product fields and effective price
2	Category	Read	Hierarchy, descendants, and gender
3	Brand	Read	Brand options
4	Inventory	Read	Available colors and sizes

Processing: Selected values are normalized into query arrays and a price range. The backend resolves descendant categories, matches variants and inventory

attributes, applies availability and price rules, sorts the result, and returns the matching page together with currently valid filter options.

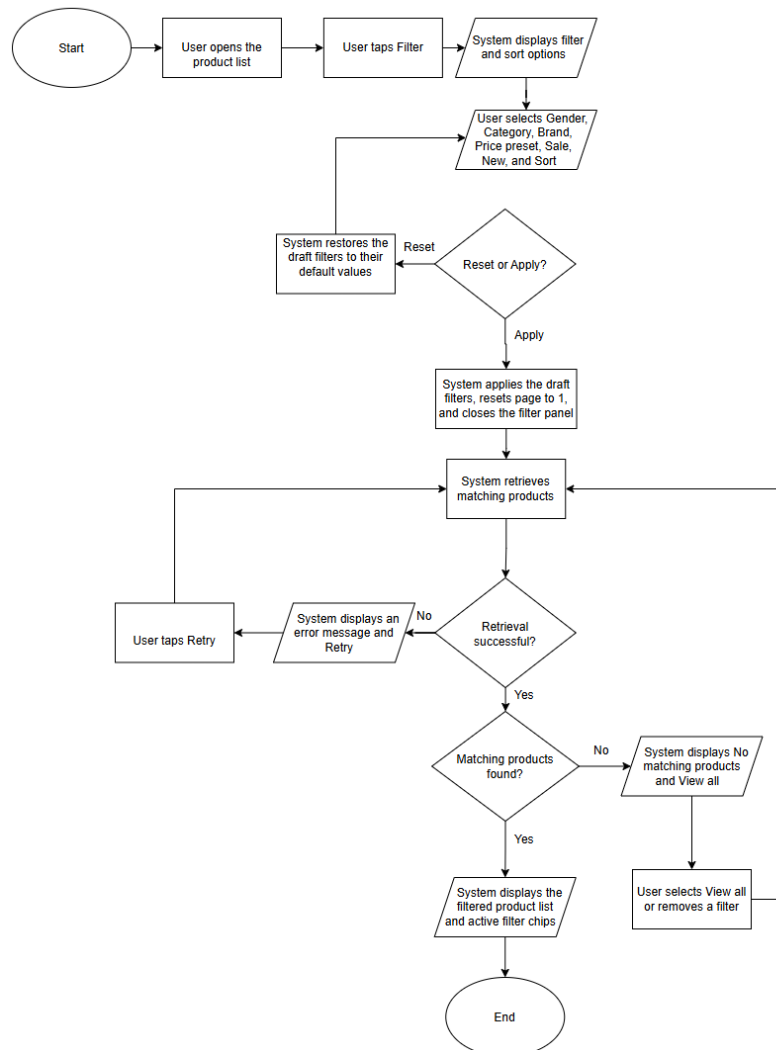


Figure 3.13 Mobile Product Filtering and Retrieval Flow

3.5.6. View Product Detail

- **Purpose:** Present complete product, variant, stock, review, and purchasing information.
- **Users:** Guests and signed-in customers; authentication is required for protected actions.
- **Interface:**

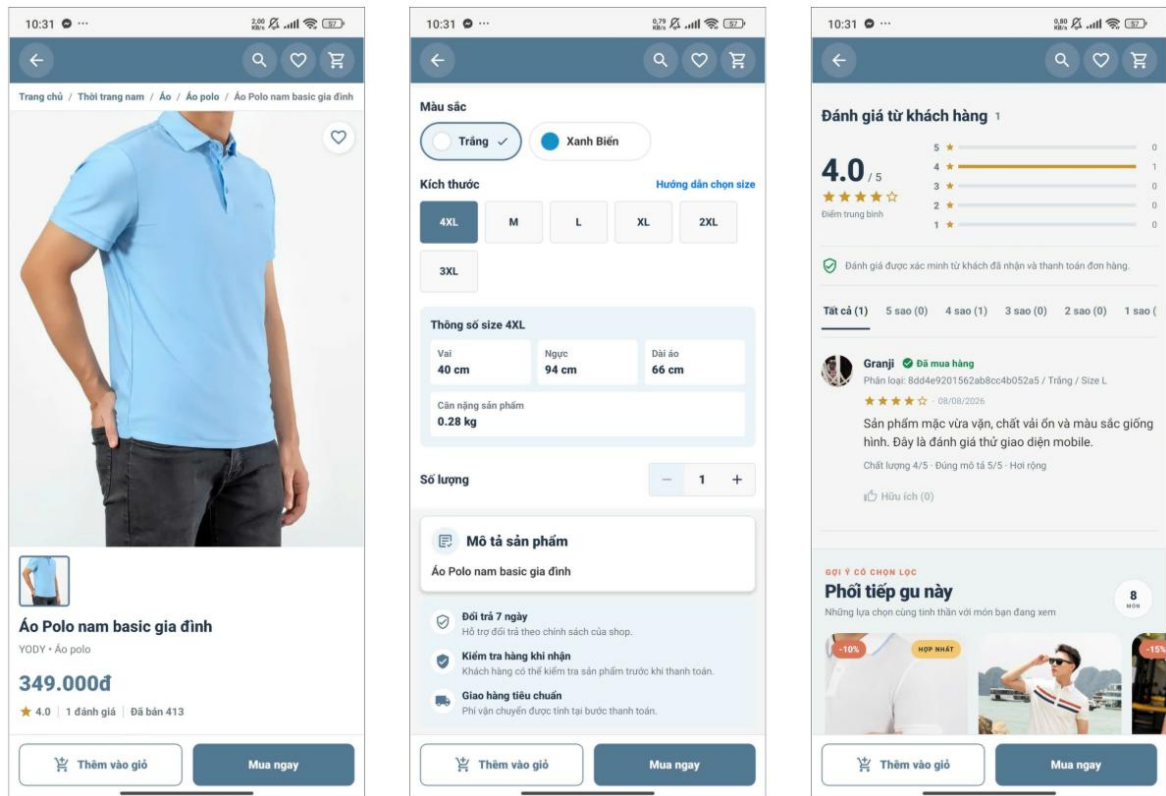


Figure 3.14 Mobile product-detail interface

Table 3.11 summarizes the main interface components.

Table 3.11 Main interface components for viewing product details

No.	Control	Purpose	Default value
1	Gallery	View the main image and product gallery	First image
2	Variant selectors	Choose fit, color, and size	First valid option
3	Quantity stepper	Set purchase quantity within available stock	1
4	Action buttons	Add to cart, buy now, or toggle favorite	-
5	Sections	View size data, description, rating summary, reviews, and related products.	Expanded as designed

Table 3.12 summarizes the principal data used by the function.

Table 3.12 Data used by view product detail

No.	Data/model	Operations	Role in the function
1	Product; Category; Brand	Read	Core product and classification information
2	Inventory	Read	SKU and available quantity by variant, color, and size

No.	Data/model	Operations	Role in the function
3	Review	Read	Rating distribution and customer reviews
4	Favorites; UserProductInteraction	Read, create, delete	Favorite state and behavioral events

Processing: The backend loads the product with variants, inventory, breadcrumb, prices, policies, and review summary. The app keeps only compatible selections and blocks unavailable combinations. Add-to-cart and buy-now actions send the selected variant, color, size, and quantity; view and recommendation events are recorded for analytics.

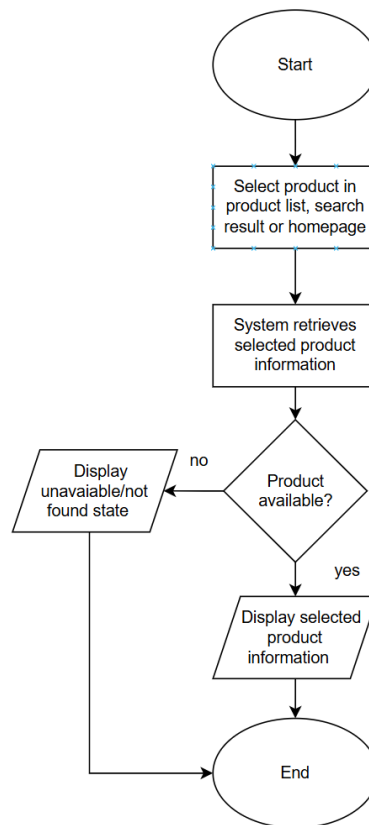


Figure 3.15 Mobile product-detail retrieval flow

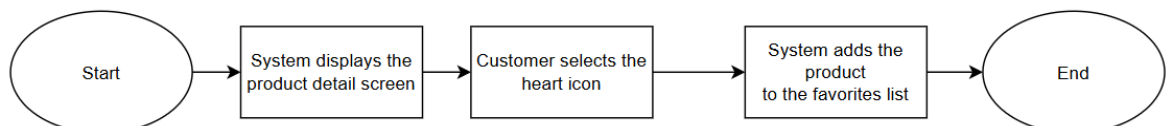


Figure 3.16 Mobile add-to-favorites flow

3.5.7. Manage Shopping Cart

- **Purpose:** Maintain selected product variants and prepare the items to be checked out.
- **Users:** Signed-in customers.
- **Interface:**

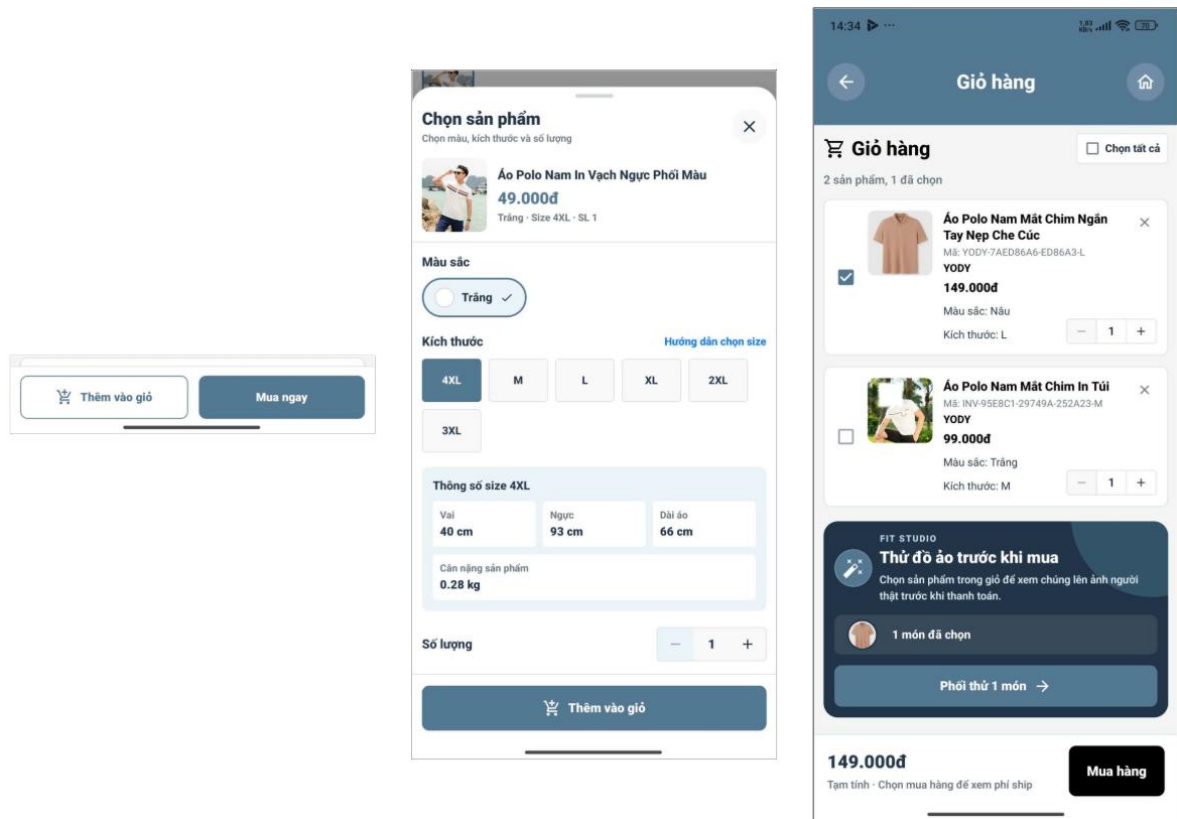


Figure 3.17 Mobile shopping-cart management interface

Table 3.13 summarizes the main interface components.

Table 3.13 Main interface components for managing the shopping cart

No.	Control	Purpose	Default value
1	Checkbox	Select one item or all items for checkout	Saved selection
2	Variant controls	Change color or size	Current variant
3	Quantity stepper	Increase or decrease quantity	Current quantity
4	Remove action	Delete a cart line	-
5	Summary/button	Show selected subtotal and continue to checkout	Checkout
6	Recommendation rail	Show complementary products	Contextual

Table 3.14 summarizes the principal data used by the function.

Table 3.14 Data used by manage shopping cart

No.	Data/model	Operations	Role in the function
1	Cart	Create, read, update, delete	Variant lines, quantity, selection, and added price
2	Product; Inventory	Read	Current price and available quantity
3	RecommendationRequest; RecommendationEvent	Create, read	Complementary recommendations and events

Processing: Every cart mutation validates ownership, variant identity, size, quantity, and current stock. The backend recalculates line totals and the selected subtotal and supports select-all. The app refreshes cart state after each mutation and requests outfit-complement recommendations from the selected cart context.

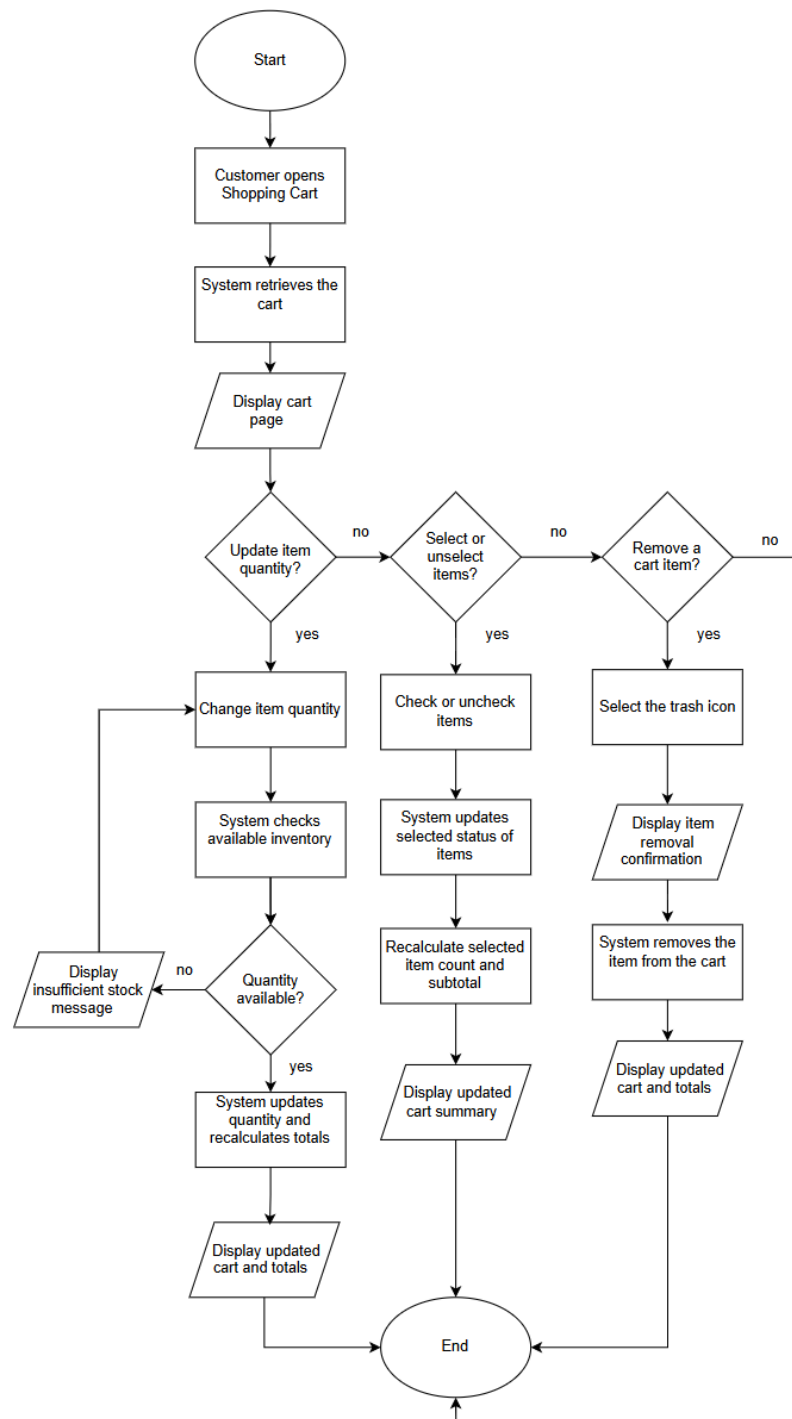


Figure 3.18 Mobile shopping-cart update flow

3.5.8. Place Order and Make Payment

- **Purpose:** Convert selected cart items into a consistent order and complete COD or VNPay payment initiation.
- **Users:** Signed-in customers with selected cart items and a valid delivery address.

- Interface:

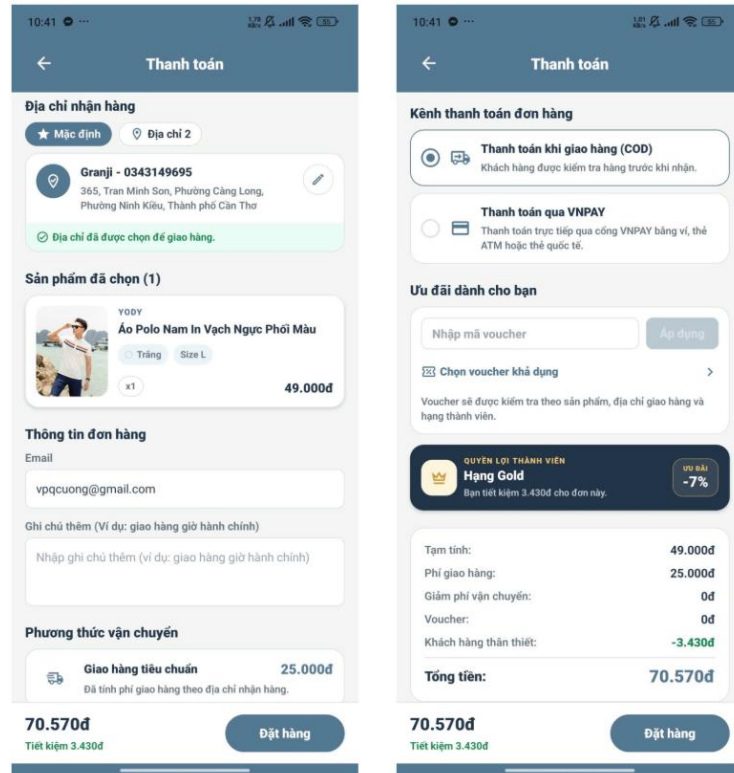


Figure 3.19 Mobile checkout and payment interface

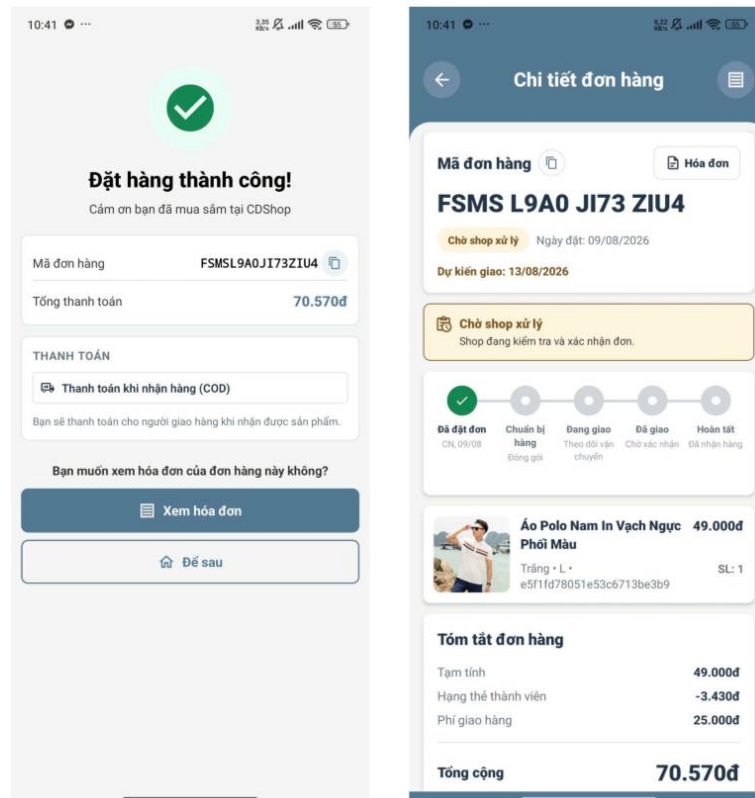


Figure 3.20 Mobile order-success and order-detail interface

Table 3.15 summarizes the main interface components.

Table 3.15 Main interface components for place order and make payment

No.	Control	Purpose	Default value
1	Address selector	Choose or enter delivery information	Default address
2	Order summary	Review products, quantities, discounts, shipping, and total	Current quote
3	Text input	Enter contact email and an optional order note	Profile email / empty
4	Shipping controls	Review the selected or recommended shipping option	Recommended option
5	Payment options	Choose COD or VNPay	COD
6	Voucher/membership controls	Apply eligible benefits	Automatic membership
7	Button	Create the order and show its result	Place Order

Table 3.16 summarizes the principal data used by the function.

Table 3.16 Data used by place order and make payment

No.	Data/model	Operations	Role in the function
1	Cart; User	Read, update	Selected lines, contact, and address
2	Order	Create, read	Snapshot of items, totals, status, payment, and shipping
3	Inventory	Read, update	Stock validation and reservation/commit
4	Coupon; MembershipRanking; LoyaltyRule	Read, update	Discount and benefit calculation
5	Transaction; PaymentMethod	Create, read, update	Payment initiation and status
6	ShippingAreaMapping	Read	GHN mapping, quote, and fallback

Processing: Checkout first requests a server quote and receives a quoteVersion covering products, promotions, membership, tax, and shipping. Order creation is idempotent and revalidates the quote, account, cart, stock, and address within the order workflow. COD creates a pending payment order; VNPay also creates the payment transaction and redirect flow. Selected cart lines are cleared after successful creation and the order-success screen displays the code and summary.

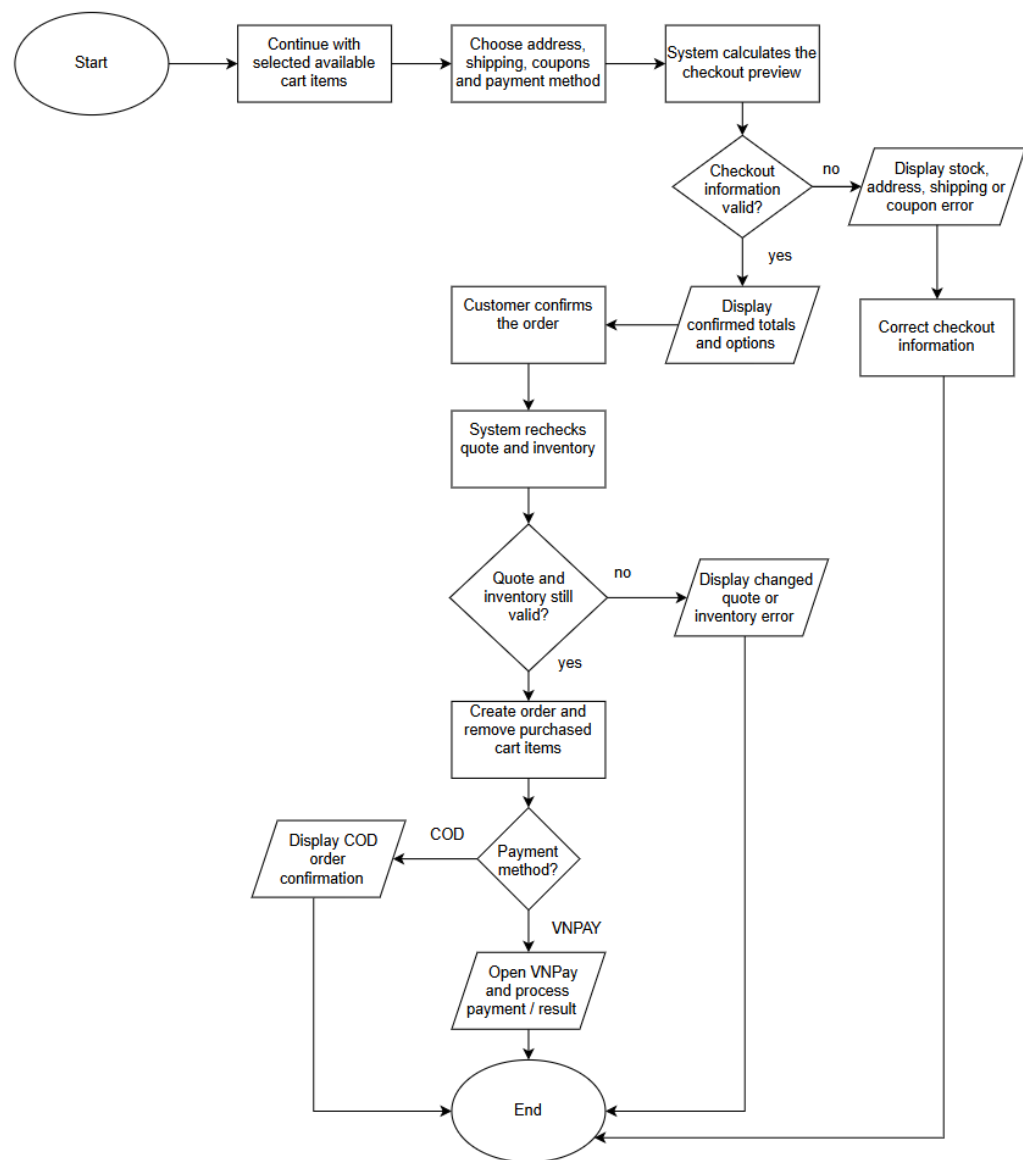


Figure 3.21 Mobile checkout and payment processing flow

3.5.9. View Order History

- **Purpose:** Track past and active purchases and open their current fulfillment details.
- **Users:** Signed-in customers.
- **Interface:**

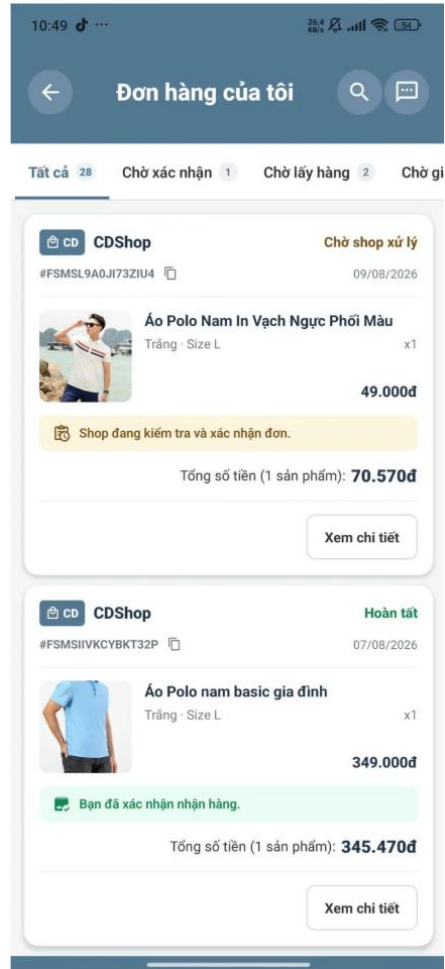


Figure 3.22 Mobile order-history interface

Table 3.17 summarizes the main interface components.

Table 3.17 Main interface components for view order history

No.	Control	Purpose	Default value
1	Status tabs	Filter orders by lifecycle state	All
2	Search	Find an order	Empty
3	Order card	Show code, products, total, status, and action	-
4	Detail action	Open timeline, payment, shipping, and item details	View Details
5	Support action	Open related support communication	-

Table 3.18 summarizes the principal data used by the function.

Table 3.18 Data used by view order history

No.	Data/model	Operations	Role in the function
1	Order	Read	Customer order snapshots and lifecycle state

No.	Data/model	Operations	Role in the function
2	Product	Read	Current product references for navigation
3	Transaction	Read	Payment status and transaction history

Processing: The backend returns only orders owned by the authenticated customer and supports status and paging filters. The app renders the normalized lifecycle timeline and refreshes when order events arrive. Opening a card loads the authoritative order details and enables only actions valid for the current state.

3.5.10. Review and Rate Products

- **Purpose:** Collect verified product ratings and experience feedback after purchase.
- **Users:** Signed-in customers with an eligible completed order item.
- **Interface:**

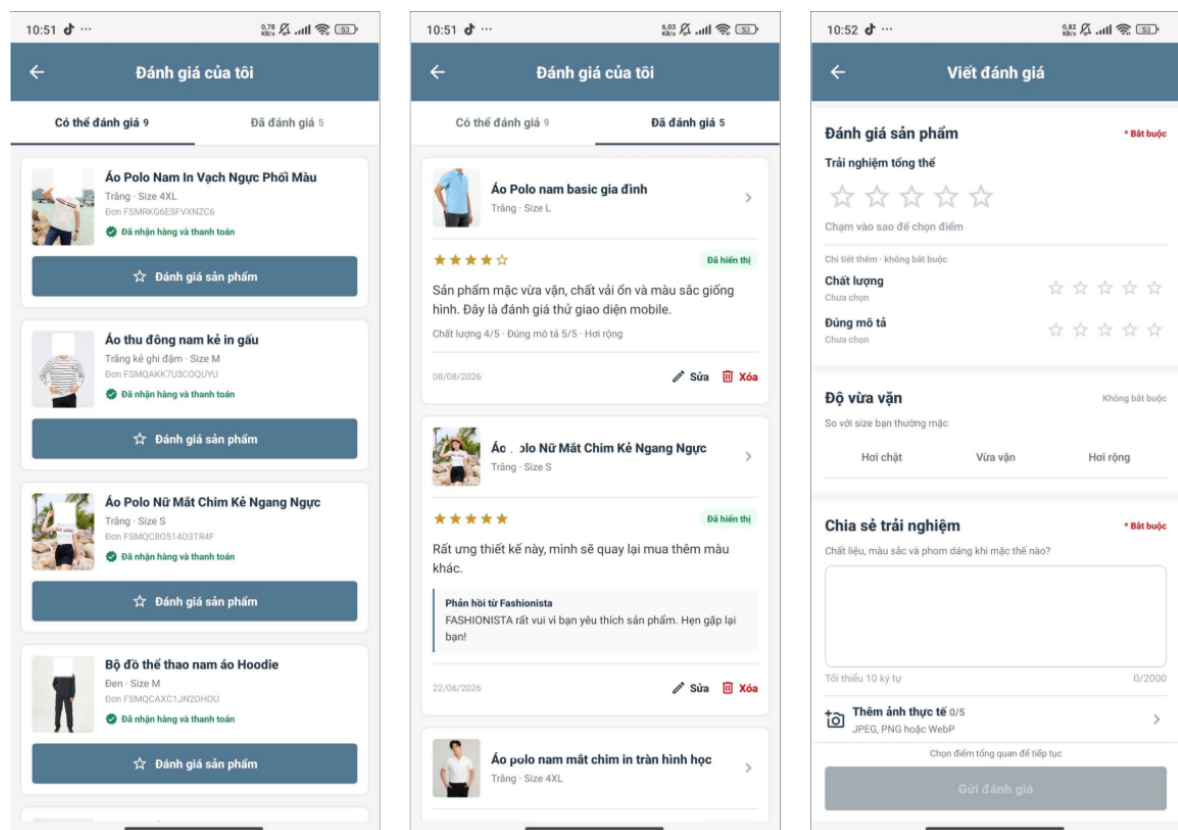


Figure 3.23 Mobile product-review and rating interface

Table 3.19 summarizes the main interface components.

Table 3.19 Main interface components for reviewing and rating products

No.	Control	Purpose	Default value
1	Tabs/list	Show products that can be reviewed and reviews already submitted	Eligible items
2	Star control	Choose the overall rating	Not selected
3	Criterion controls	Rate quality, fit, color, and value when applicable	Not selected
4	Text area	Write an experience comment	Empty
5	Media picker	Attach permitted review images	No attachment
6	Button/actions	Submit, edit, or delete the review	Submit Review

Table 3.20 summarizes the principal data used by the function.

Table 3.20 Data used by review and rate products

No.	Data/model	Operations	Role in the function
1	Review	Create, read, update, delete	Rating, criteria, comment, media, moderation, and reply
2	Order	Read	Verified-purchase eligibility
3	Product	Read, update	Reviewed item and aggregate rating
4	ReviewHelpfulVote	Create, read, delete	Helpful-vote state

Processing: The backend confirms that the order item is eligible and prevents duplicate active reviews. It validates rating values, content, and image limits, then saves the review with verified-purchase context and updates product aggregates. Customers can edit or delete their own review within service rules; authorized staff handle moderation, and shop replies separately.

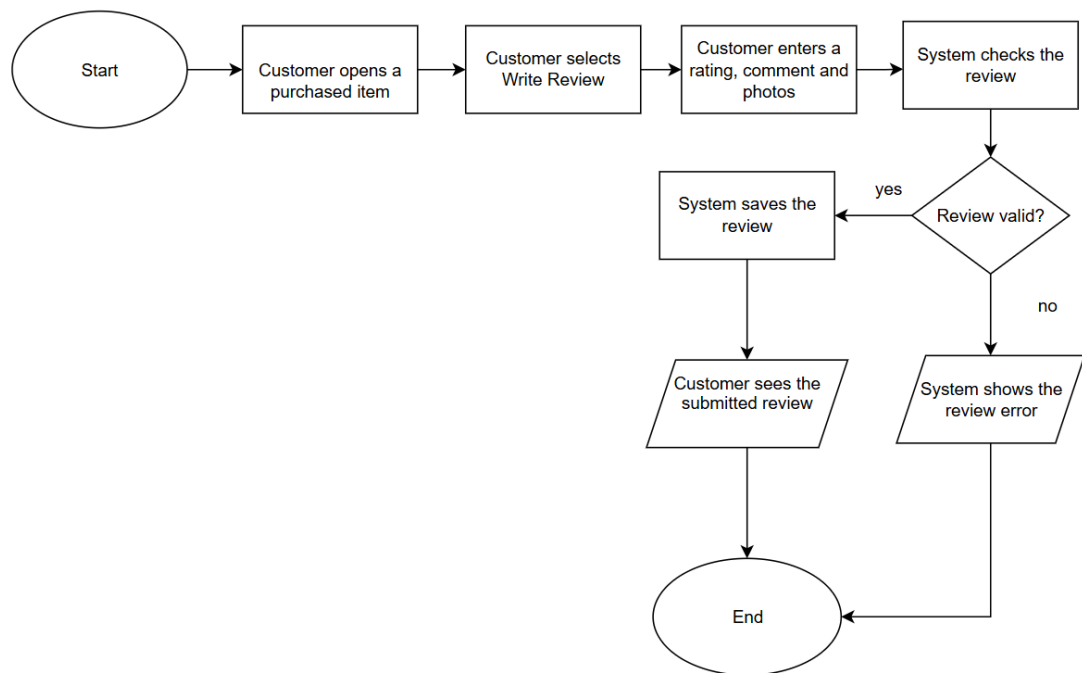


Figure 3.24 Mobile product-review submission flow

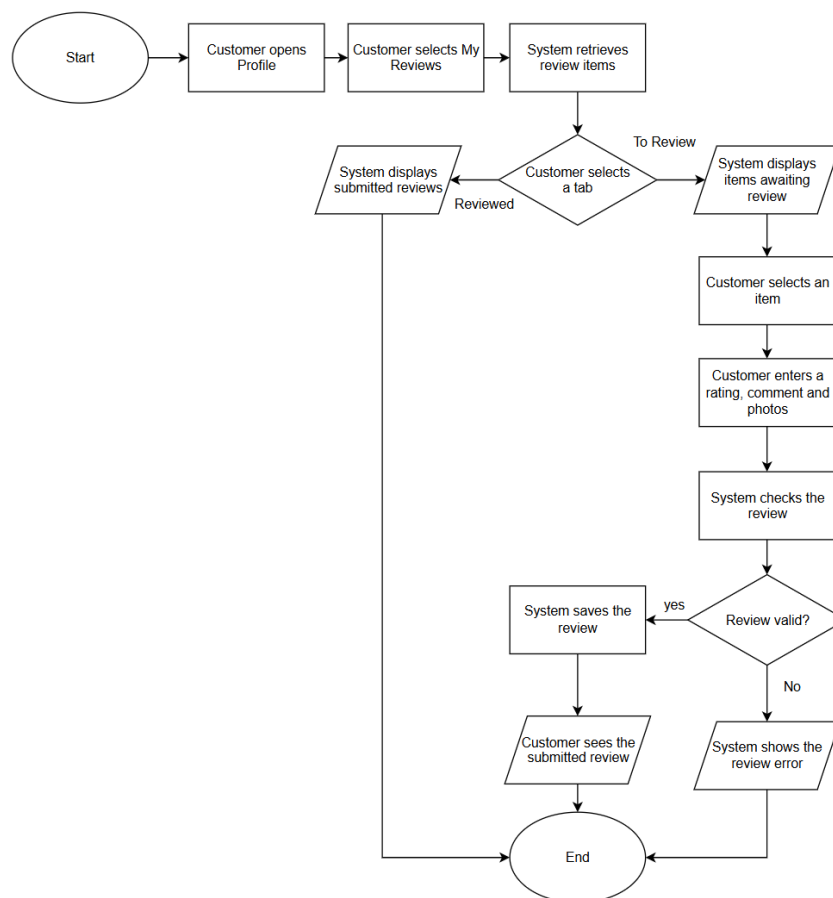


Figure 3.25 Mobile review-list and review-submission flow

3.5.11. Contact Support

- **Purpose:** Provide self-service answers and a traceable communication channel with shop support.
- **Users:** Guests for public FAQs/feedback and signed-in customers for support tickets.
- **Interface:**

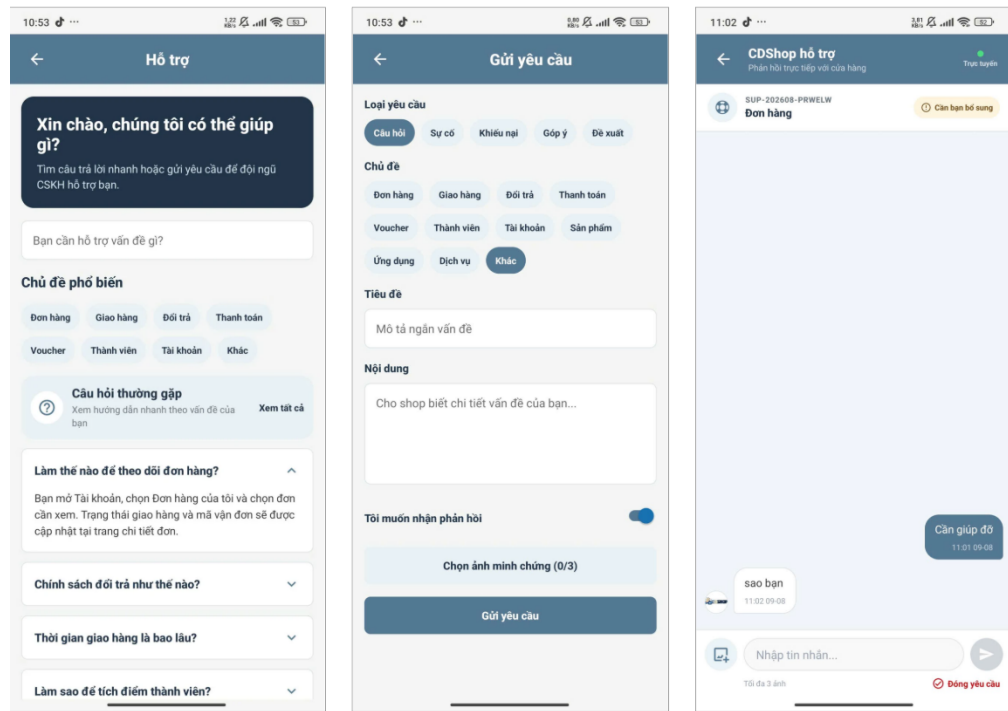


Figure 3.26 Mobile customer-support interface

Table 3.21 summarizes the main interface components.

Table 3.21 Main interface components for contact support

No.	Control	Purpose	Default value
1	Search/category controls	Find FAQ guidance	Popular topics
2	Request form	Choose type/category and enter subject and content	Empty
3	Evidence picker	Attach supporting files	No attachment
4	Ticket list/detail	Track status and read the conversation	Newest tickets
5	Message input	Reply to an open ticket	Empty

Table 3.22 summarizes the principal data used by the function.

Table 3.22 Data used by contact support

No.	Data/model	Operations	Role in the function
1	FaqArticle; FaqVote	Read, create, update	Public guidance and helpful votes
2	SupportTicket	Create, read, update	Request metadata, assignment, priority, and state
3	SupportMessage	Create, read	Customer and staff conversation
4	PushToken	Create, delete	Support-reply notifications

Processing: The customer can search FAQs before creating a ticket. Ticket creation validates the category, subject, content, rate limit, and attachments, then generates a ticket code and support state. Messages, read state, close, and reopen operations are ownership-protected and synchronized through real-time events; staff replies can also trigger email and push notifications.

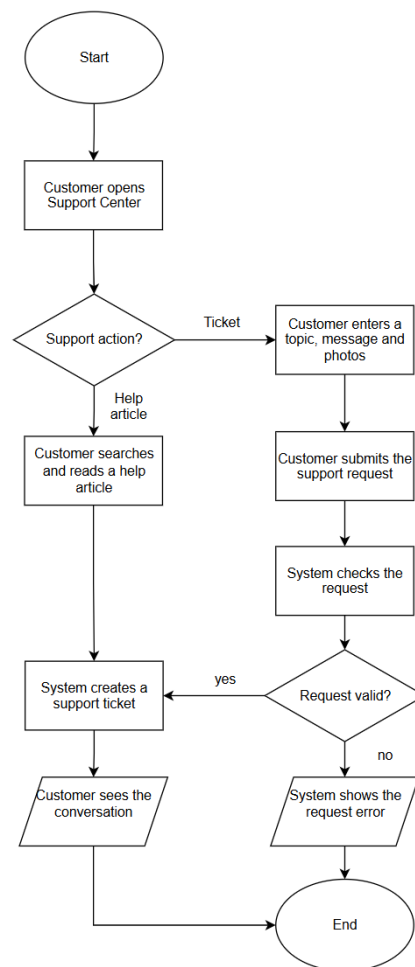


Figure 3.27 Mobile customer-support FAQ and ticket flow

3.5.12. Manage Notifications

- **Purpose:** Read, classify, and clear customer notifications generated by system events.
- **Users:** Signed-in customers.
- **Interface:**



Figure 3.28 Mobile notification-management interface

Table 3.23 summarizes the main interface components.

Table 3.23 Main interface components for managing notifications

No.	Control	Purpose	Default value
1	Category tabs	Filter order, promotion, virtual try-on, and other notifications	All
2	Notification card	Show title, message, time, and unread state	Newest first
3	Action	Open the related order, ticket, or try-on result	Contextual
4	Button	Mark all notifications as read	Mark All Read

Table 3.24 summarizes the principal data used by the function.

Table 3.24 Data used by manage notifications

No.	Data/model	Operations	Role in the function
1	CustomerNotification	Read, update	Notification payload, category, read state, and deep link
2	PushToken	Create, delete	Registered device token
3	User	Read	Notification ownership and active state

Processing: The backend returns notifications owned by the active customer and an unread summary. Opening an item marks it read and uses its validated target to navigate to the related feature. Mark-all updates all unread records, while push-token registration and removal keep device delivery synchronized with the signed-in session.

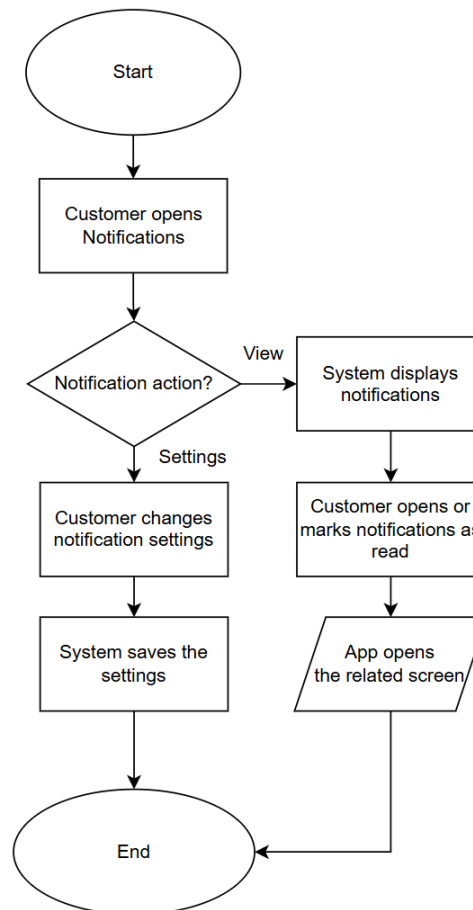


Figure 3.29 Mobile notification viewing and settings flow

3.5.13. View Recommendations

- **Purpose:** Show relevant products for personal discovery, product similarity, and outfit completion.

- **Users:** Guests and signed-in customers; personal history is used when available.
- **Interface:**

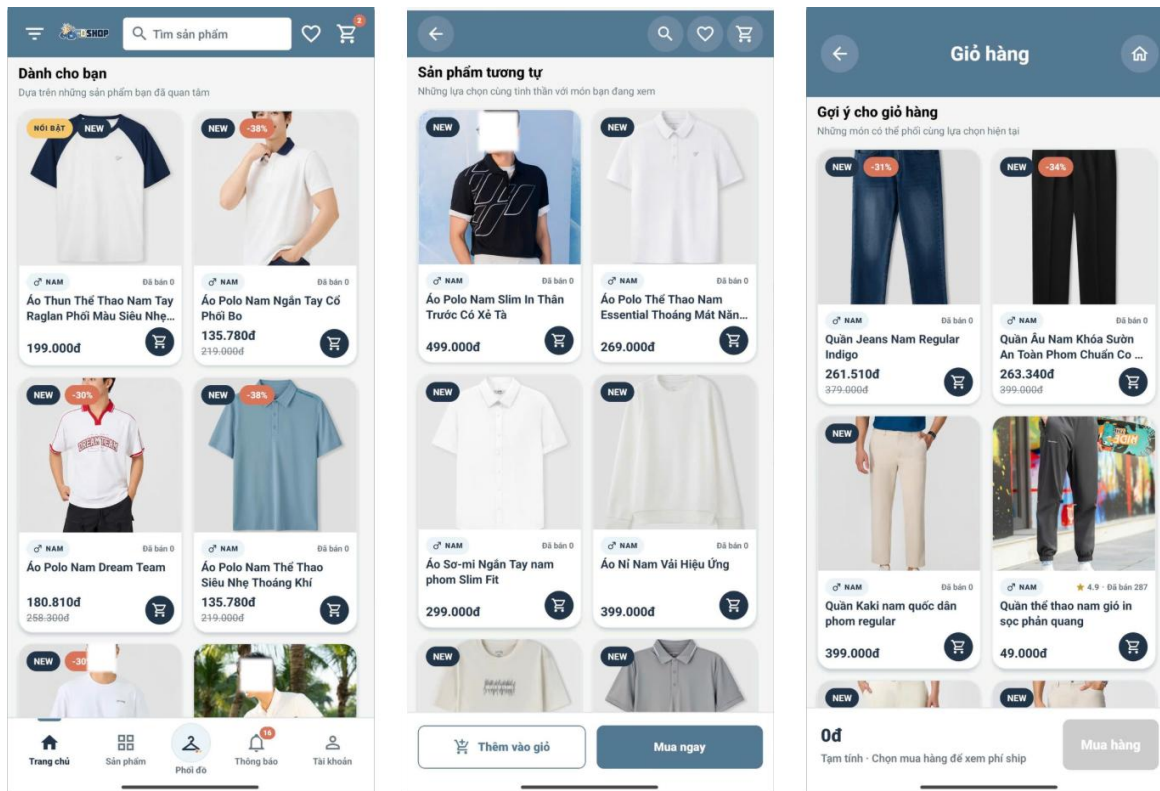


Figure 3.30 Mobile recommendation interfaces on home, product detail, and cart

Table 3.25 summarizes the main interface components.

Table 3.25 Main interface components for view recommendations

No.	Control	Purpose	Default value
1	Recommendation rail	Display ranked product cards	Context-specific list
2	Product card	Open the recommended product	-
3	Cart/detail actions	Continue to variant selection or purchase	-

Table 3.26 summarizes the principal data used by the function.

Table 3.26 Data used by view recommendations

No.	Data/model	Operations	Role in the function
1	RecommendationRequest	Create, read	Context, algorithm version, candidates, and response
2	RecommendationEvent	Create	Impression, click, add-to-cart, and purchase attribution

No.	Data/model	Operations	Role in the function
3	UserProductInteraction	Create, read	Weighted behavioral preference signals
4	Product; Inventory	Read	Candidate attributes, price, popularity, and availability

Processing: Home recommendations score behavioral preference with recency and business signals; product-detail recommendations score content similarity; cart recommendations emphasize complementary outfit roles. Unavailable and excluded products are removed, diversity and fallback rules are applied, and request, impression, click, cart, and purchase events are recorded for evaluation.

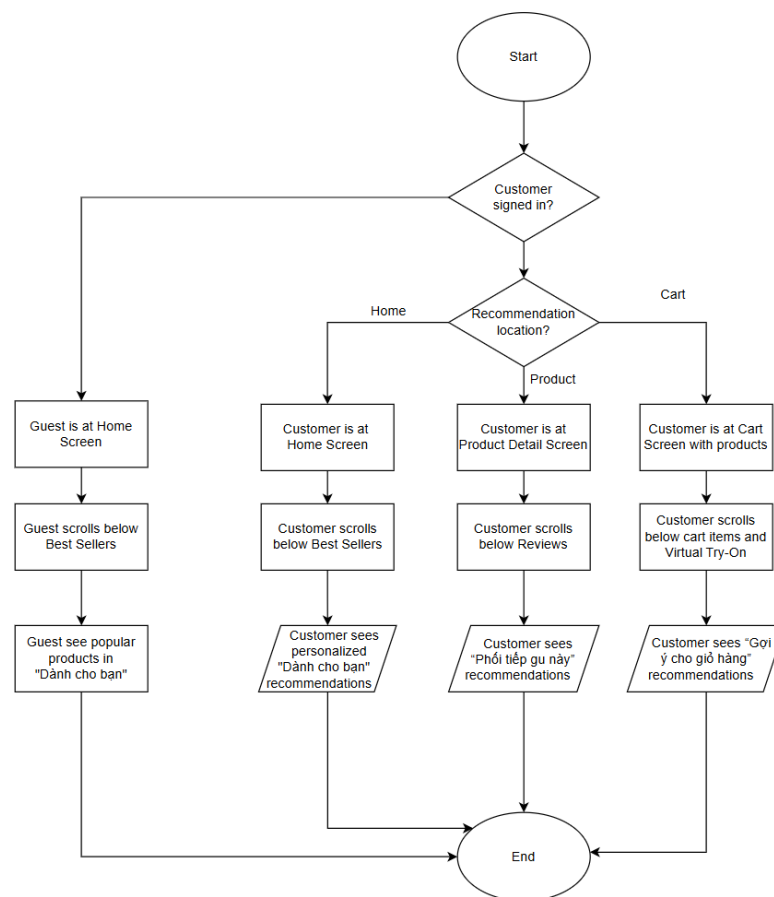


Figure 3.31 Mobile recommendation display flow by context

3.5.14. Use Virtual Try-On

- **Purpose:** Generate image and optional short-video previews of selected garments on a customer image.
- **Users:** Signed-in, active customers who are not restricted from virtual try-on.

- **Interface:**

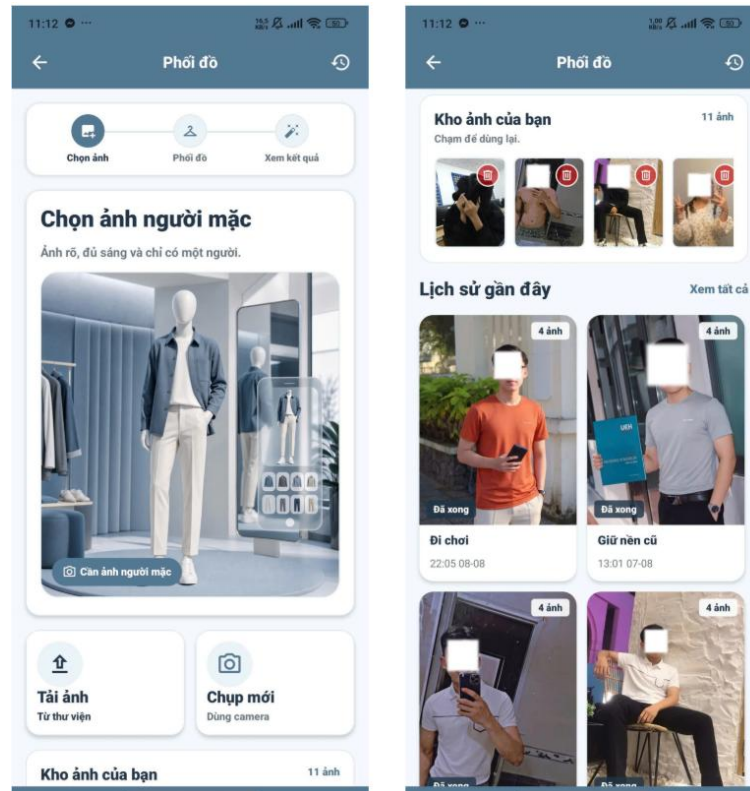


Figure 3.32 Mobile virtual try-on source-selection and history interface

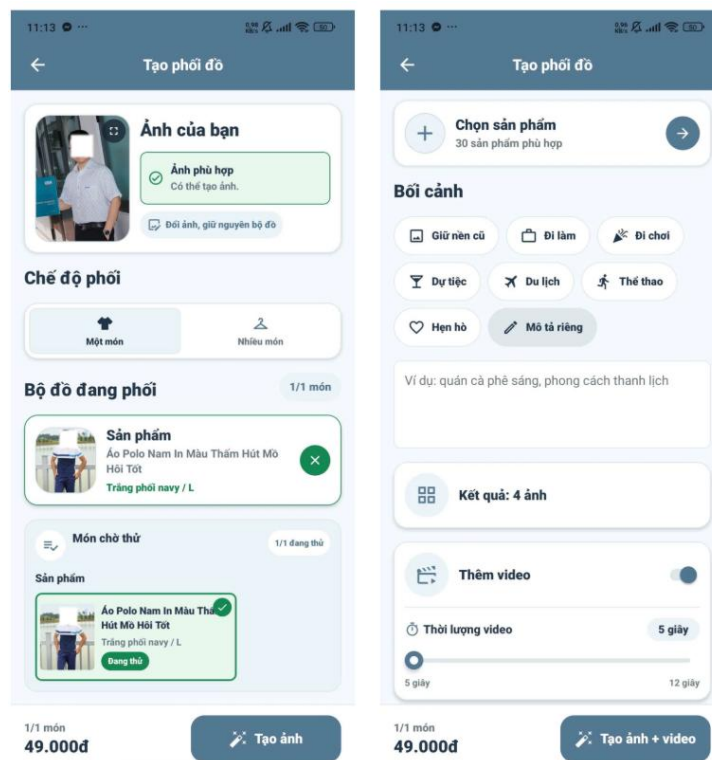


Figure 3.33 Mobile virtual try-on outfit-configuration interface

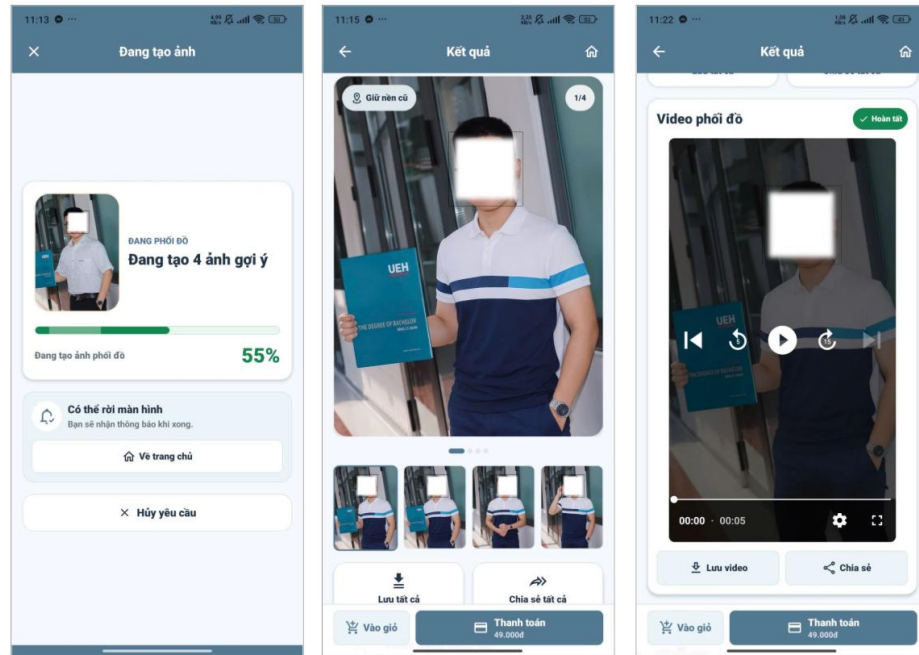


Figure 3.34 Mobile virtual try-on processing and result interface

Table 3.27 summarizes the main interface components.

Table 3.27 Main interface components for use in virtual try-on

No.	Control	Purpose	Default value
1	Source controls	Choose a saved person image, upload from the gallery, or use the camera	No source
2	Garment selectors	Choose one or more compatible product variants and outfit roles	No garment
3	Mode/context controls	Select outfit mode, scene preset, prompt, and optional video duration	Single item/image
4	Generate button	Create an idempotent try-on job	Generate
5	Progress actions	Monitor, cancel, or retry processing	Live status
6	Result actions	Browse, save, share, add garments to cart, or continue to checkout	Latest result

Table 3.28 summarizes the principal data used by the function.

Table 3.28 Data used by virtual try-on

No.	Data/model	Operations	Role in the function
1	VirtualTryOnAsset	Create, read, update, delete	Source and generated media assets
2	VirtualTryOnJob	Create, read, update, delete	Selections, prompt, status, progress, and result

No.	Data/model	Operations	Role in the function
3	VirtualTryOnSettings	Read	Limits, capabilities, provider, and feature flags
4	VirtualTryOnPrompt Violation; VirtualTryOnAccount Lock	Create, read	Policy enforcement
5	Product; Inventory	Read	Garment identity, variant, role, size, and availability
6	UserProductInteraction	Create	Try-on behavior signal

Processing: The customer uploads or selects a person asset, which is validated for file policy, image quality, person/pose suitability, and outfit mode. Compatible garment roles and variants are selected, and the prompt policy and account limits are checked before an idempotent job is queued. Image and optional video stages update progress through polling and real-time events. Successful outputs can be saved or shared, and the selected products can continue to cart or checkout.

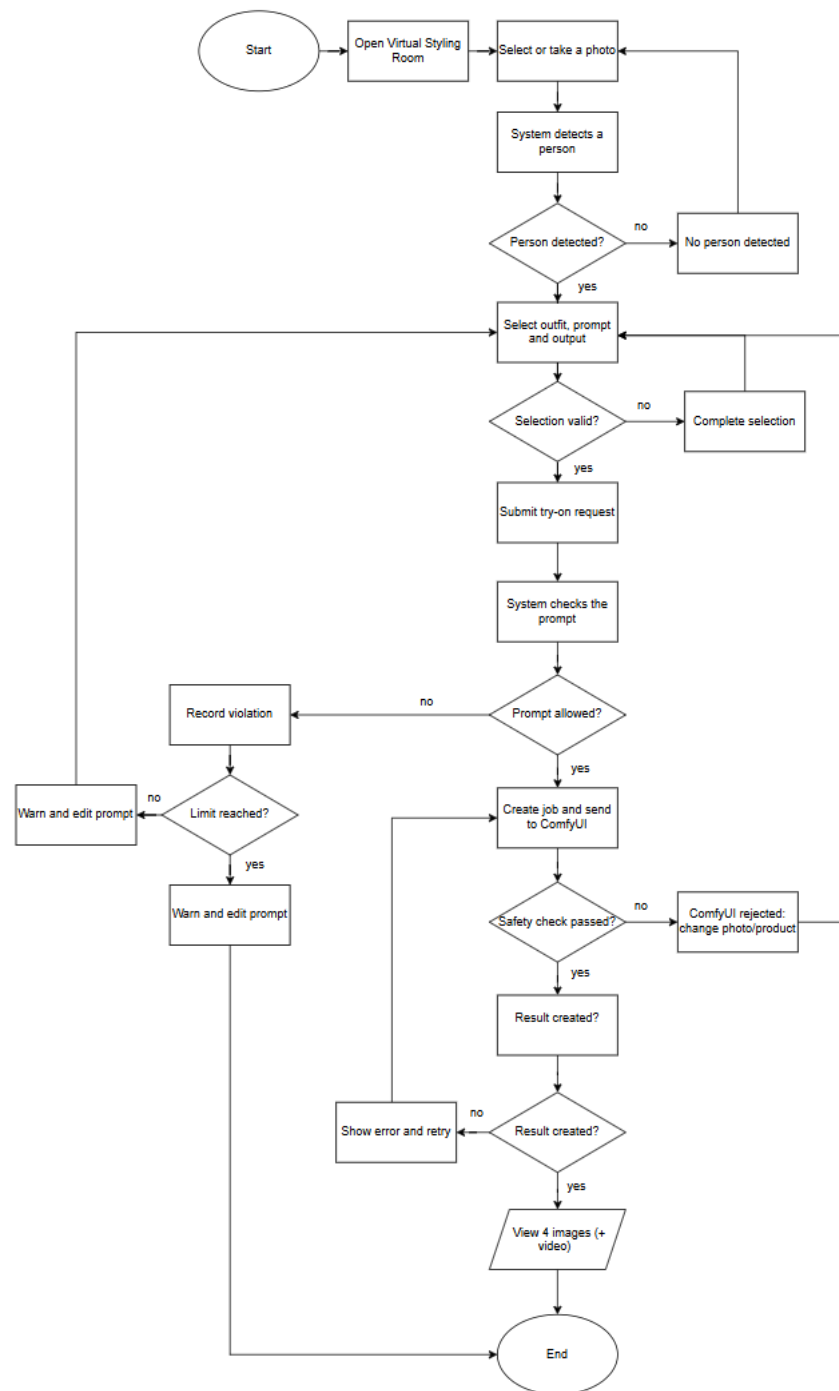


Figure 3.35 Mobile virtual try-on orchestration flow

3.5.15. Manage Personal Information

- **Purpose:** Let a signed-in customer keep identity, delivery, avatar, and password information current through one protected mobile workspace.
- **Users:** Signed-in customers.
- **Interface:**

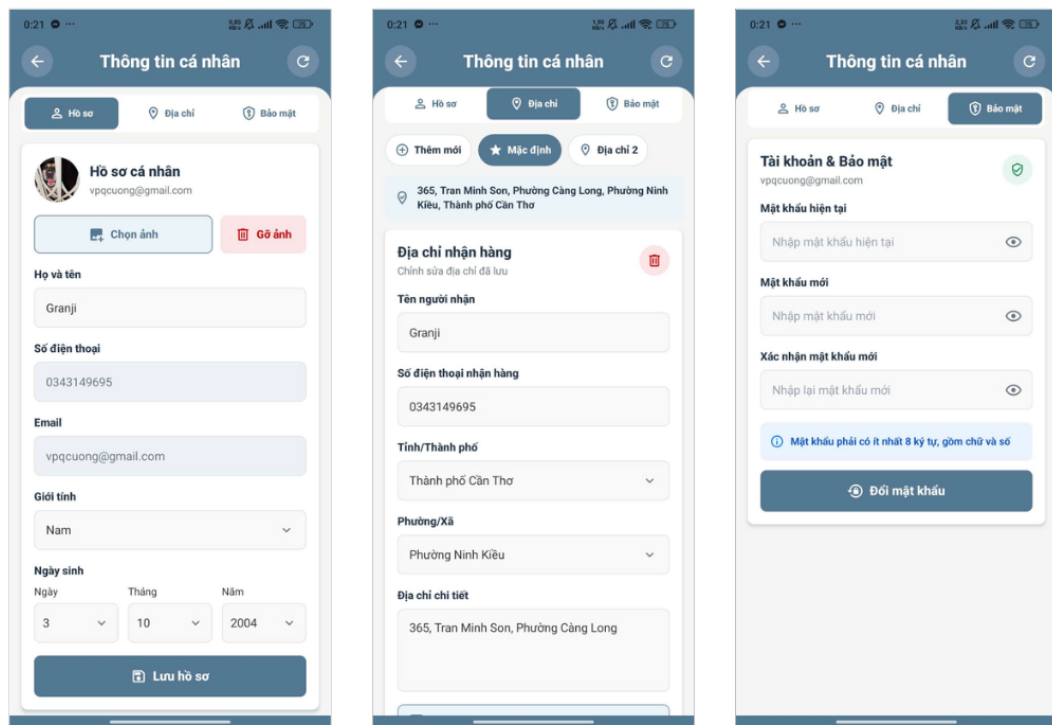


Figure 3.36 Mobile personal-information profile, address, and security interface

Table 3.29 summarizes the main interface components.

Table 3.29 Main interface components for managing personal information

No.	Control	Purpose	Default value
1	Profile / Address / Security tabs	Switch among identity, delivery address, and password tasks	Profile tab
2	Profile fields	Edit name, phone, gender, and date of birth; display email read-only	Current user data
3	Avatar controls	Choose, upload, preview, or remove the profile image	Current avatar or initial
4	Address selector	Choose an existing/default address or start a new address	Default or first address
5	Address form and actions	Validate recipient, phone, province, ward, street; add/update/delete/set default	Selected address values
6	Location picker	Load provinces and wards; permit manual ward only when supported	No selection for a new address
7	Password fields	Submit current, new, and confirmation values	Empty and masked

No.	Control	Purpose	Default value
8	Save, refresh, and status controls	Execute the active operation and show loading, validation, success, or error state	Loaded account state

Table 3.30 summarizes the principal data used by the function.

Table 3.30 Data used by manage personal information

No.	Data/model	Operations	Role in the function
1	User	Read, update	Name, read-only email, phone, gender, date of birth, avatar, and profile-completion state
2	User.address	Create, read, update, delete	Up to five recipient addresses, location/GHN mapping fields, and the single-default invariant
3	Province and ward data	Read	Managed location names/codes and controlled manual-ward fallback
4	Cloud media asset	Create, update, delete	Avatar image URL and provider public identifier
5	Authentication state	Read, update	Password hash, passwordChangedAt, refresh-token revocation, device/push/realtime access

Processing: The authenticated screen loads the current user and addresses in parallel, then loads province/ward choices while guarding against stale responses. Profile updates validate name, Vietnamese phone, gender, and age; the backend additionally enforces phone uniqueness and recomputes profileCompleted before the client refreshes its stored session user. Avatar upload accepts JPEG, PNG, or WebP and is authoritatively limited to 3 MB by the backend before managed media replacement. Address actions validate recipient/location fields, cap the list at five, and maintain a single default address. Password change verifies the current password, rejects reuse, hashes the replacement, clears the refresh token, and revokes device/realtime access. Unauthorized responses clear the local session and return the customer to Sign In.

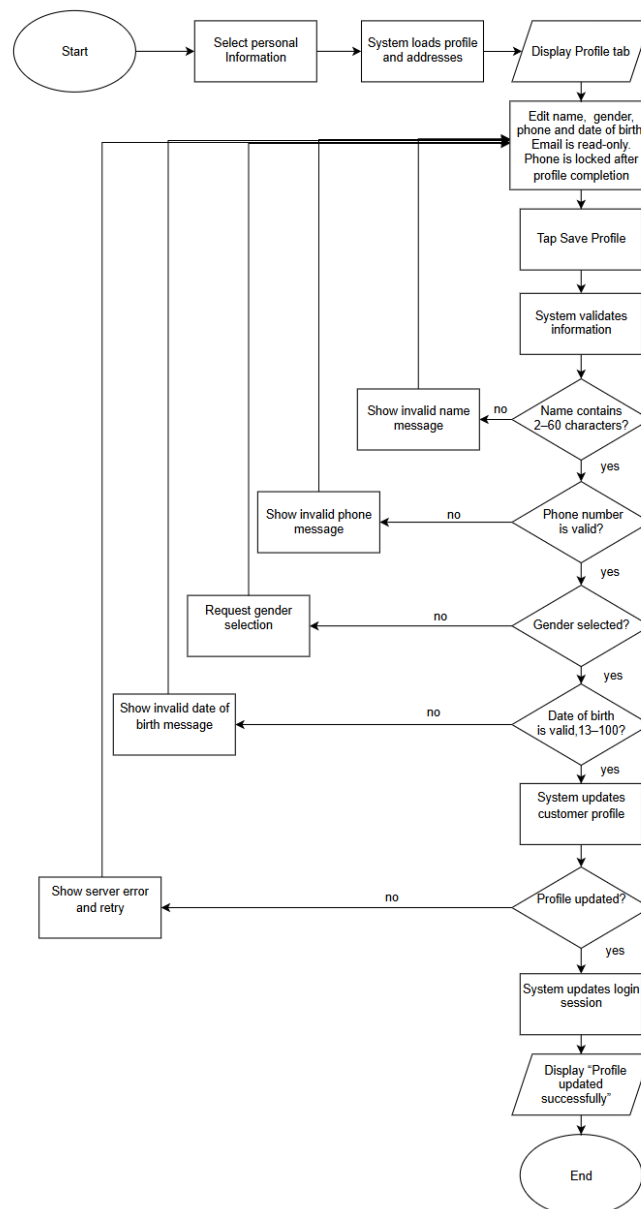


Figure 3.37 Mobile profile-update validation flow

3.5.16. Manage Customer Accounts

- **Purpose:** Inspect customer status and activity and perform controlled account-support actions.
- **Users:** Administrators and staff with customer read/manage permission.
- **Interface:**

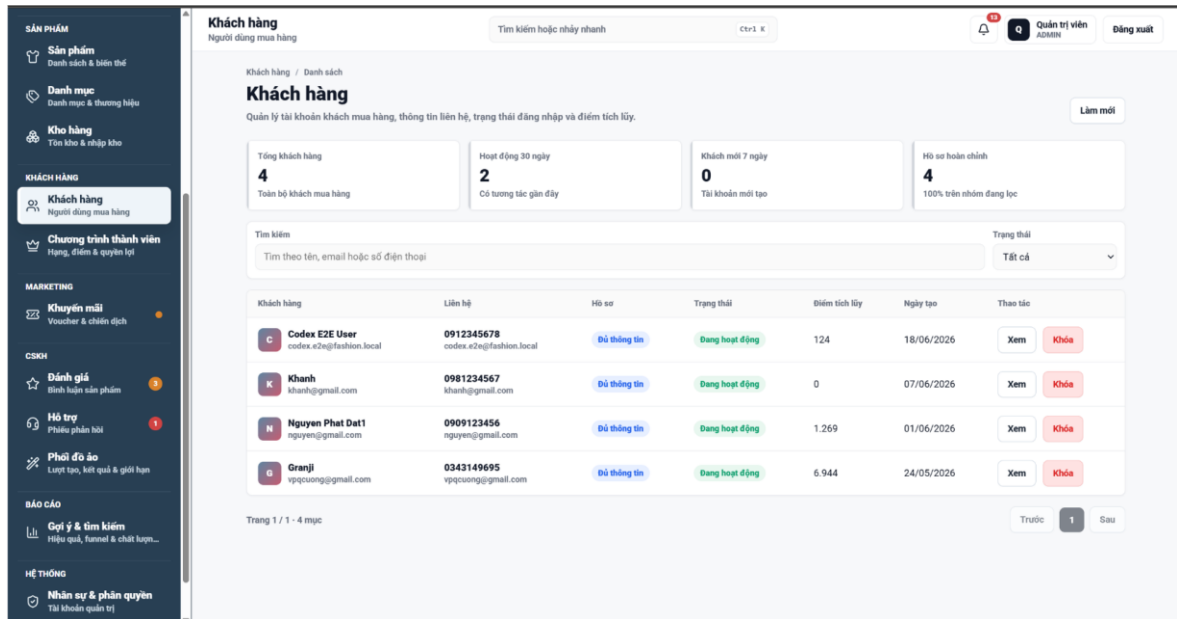


Figure 3.38 Admin customer-account management interface

Table 3.31 summarizes the main interface components.

Table 3.31 Main interface components for managing customer accounts

No.	Control	Purpose	Default value
1	KPI cards	Show customer totals and status summaries	Current data
2	Search/filters	Find by name, email, phone, tier, role, or state	All
3	Customer table	Show identity, contact, membership, points, and status	Page 1
4	Detail drawer	View orders, insights, and internal notes	Closed
5	Actions	Activate/deactivate, change allowed role, force password reset, or maintain notes	Permission-based

Table 3.32 summarizes the principal data used by the function.

Table 3.32 Data used by manage customer accounts

No.	Data/model	Operations	Role in the function
1	User	Read, update	Customer identity, role, status, membership, and security state
2	CustomerNote	Create, read, update, delete	Internal customer notes
3	Order; LoyaltyPointHistory	Read	Customer activity and value insights
4	AuditLog	Create	Administrative changes

Processing: The service filters and paginates managed users and computes summary counts. Authorized staff can open customer insights and notes; customers. Manage permission is required for mutations. Status, role, and forced-reset actions validate the target and actor, apply security consequences when needed, and record administrative activity.

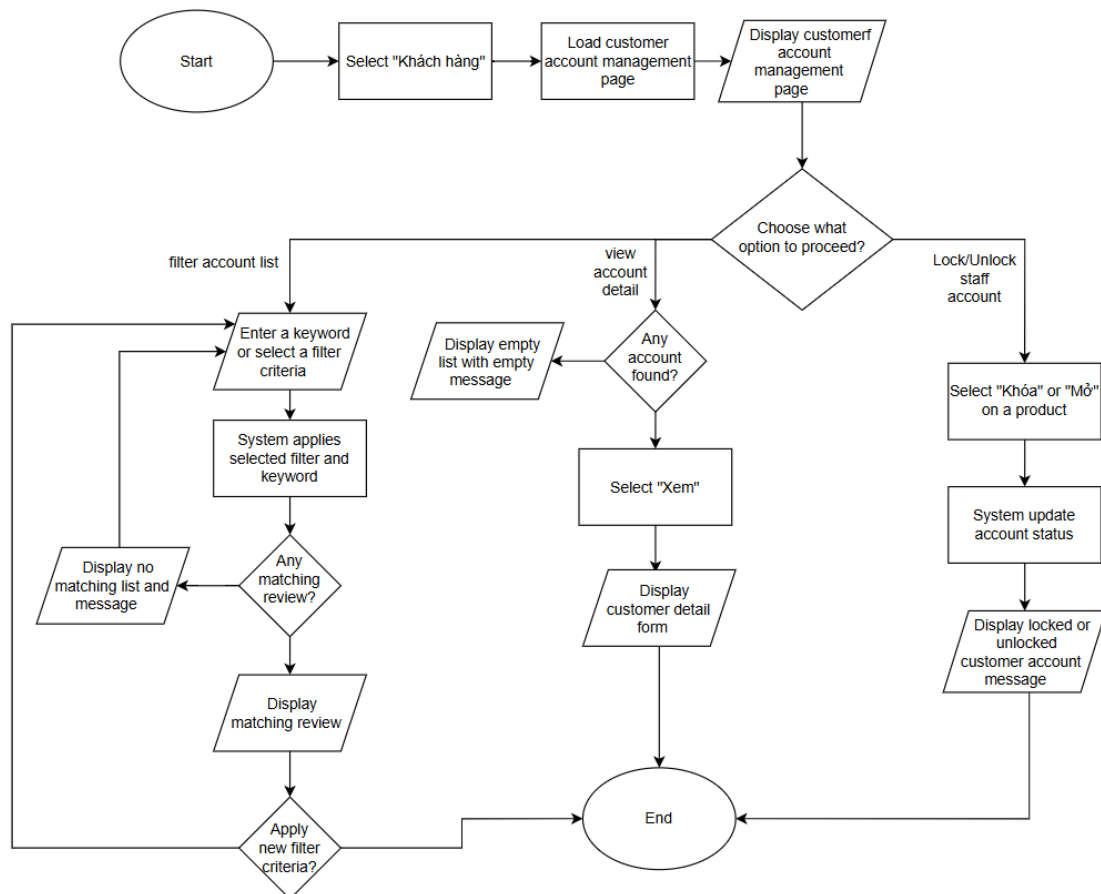


Figure 3.39 Admin customer-account management flow

3.5.17. Manage Roles and Permissions

- **Purpose:** Create staff accounts and assign only the permissions required for their work.
- **Users:** Administrators.
- **Interface:**

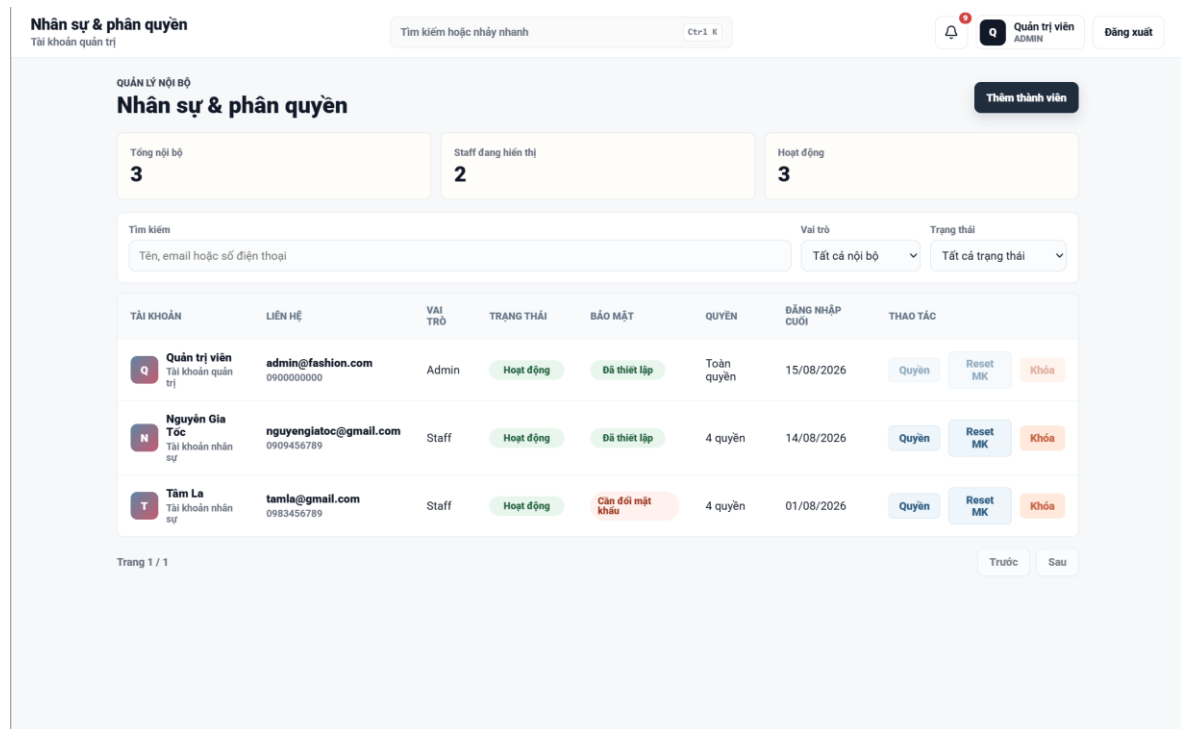


Figure 3.40 Admin role and permission management interface

Table 3.33 summarizes the main interface components.

Table 3.33 Main interface components for managing roles and permissions

No.	Control	Purpose	Default value
1	KPI/search filters	Summarize and find internal accounts	All
2	Create-staff form	Enter identity and temporary credentials	Empty
3	Permission editor	Assign module-level read, write, reply, moderate, manage, settings, and report permissions	Least privilege
4	Status/reset actions	Turn staff on/off or set a temporary password	-

Table 3.34 summarizes the principal data used by the function.

Table 3.34 Data used by manage roles and permissions

No.	Data/model	Operations	Role in the function
1	User	Create, read, update	Admin/staff identity, active state, permissions, and password state
2	AuditLog	Create, read	Account and permission changes

Processing: Only an administrator can create and maintain internal accounts. Submitted permissions are checked against the supported permission set; implied permissions such as support.reply, reviews.read, or virtual_try_on.read are expanded consistently. Status and temporary-password actions validate the target and preserve administrator safeguards.

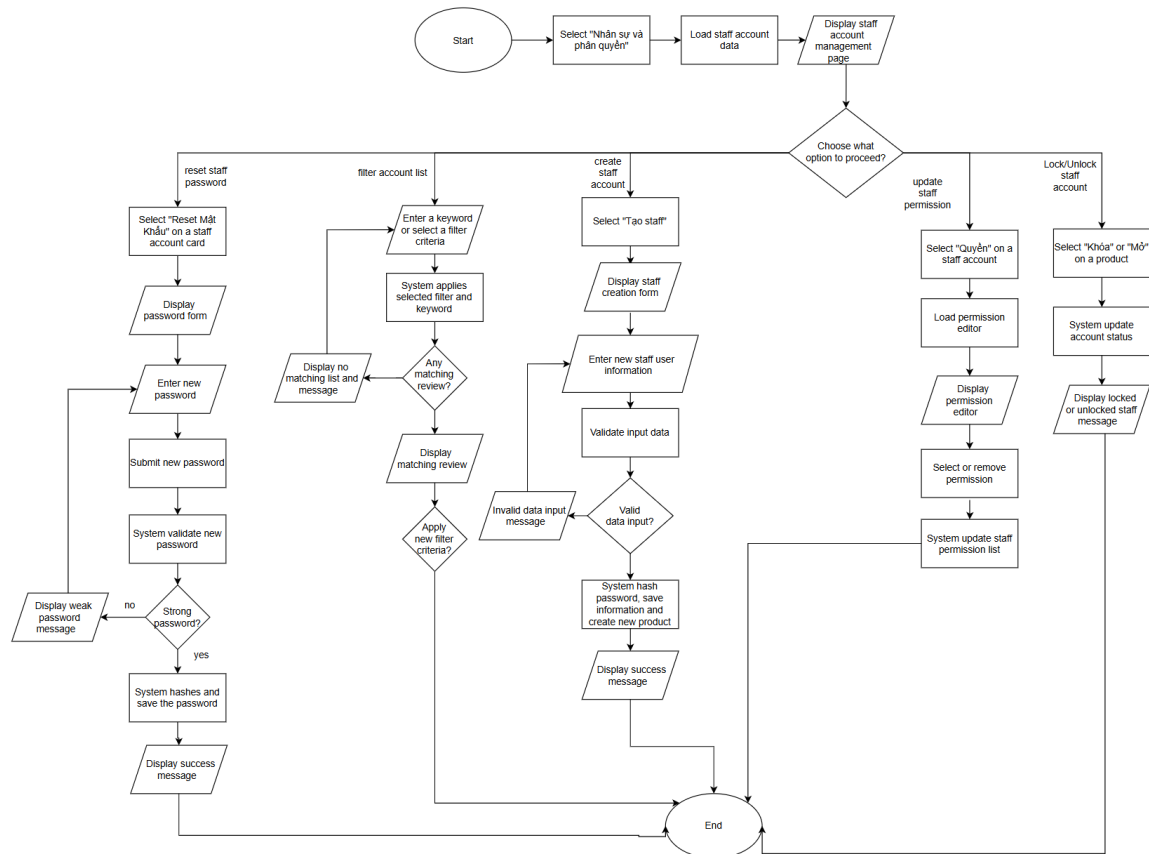


Figure 3.41 Admin staff-account and permission management flow

3.5.18. Manage Orders

- **Purpose:** Search and inspect orders, coordinate lifecycle, payment, shipping, return, invoice, export, and bulk operations.
- **Users:** Administrators and staff with order read/update permission.
- **Interface:**

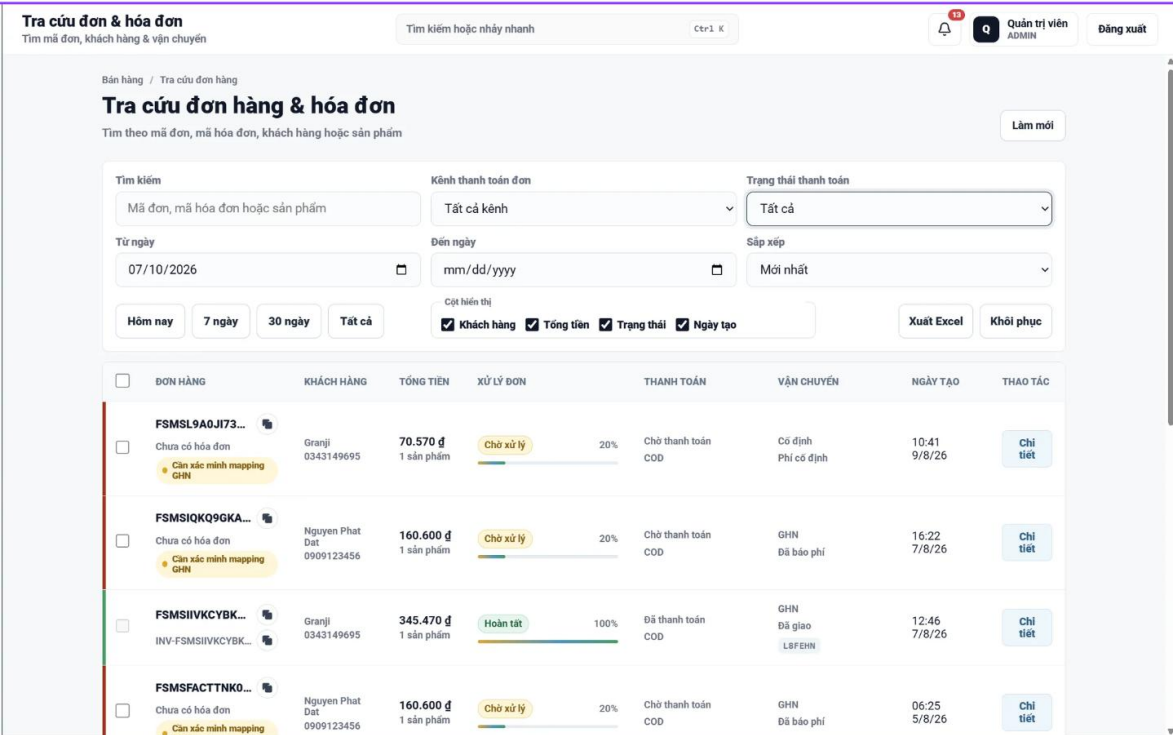


Figure 3.42 Admin order lookup and invoice interface

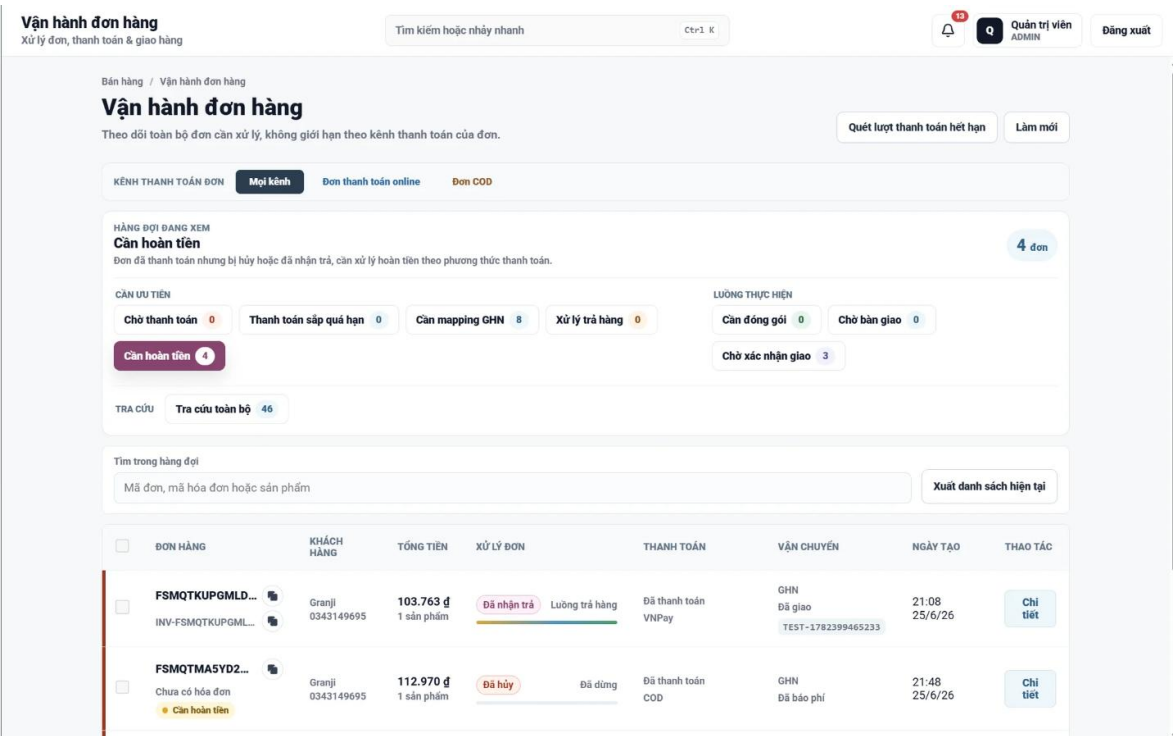


Figure 3.43 Admin order operations interface

Table 3.35 summarizes the main interface components.

Table 3.35 Main interface components for manage orders

No.	Control	Purpose	Default value
1	Queue and channel tabs	Switch among payment deadlines, packing, handoff, delivery, return, refund, all-order, online-payment, and COD work	All channels / current queue
2	Lookup and filters	Search order, invoice, customer, product, payment, shipping, and date criteria	Newest orders
3	Order table and export	Review operational state, select orders, choose visible columns, and export the authorized result set	Page 1
4	Order detail drawer	Inspect overview, line items, payment, shipping, return evidence, invoice, transactions, and history	Closed
5	Lifecycle controls	Confirm, pack, hand off, cancel, deliver, complete, or review a return through valid transitions	Permission- and state-based
6	Payment and shipping controls	Reconcile or refund VNPay, adjust permitted payments, resolve GHN mapping, and create/sync/cancel shipments	Current order state
7	Bulk action toolbar	Apply only the common eligible transition or GHN action and retain failed selections for correction	No orders selected

Table 3.36 summarizes the principal data used by the function.

Table 3.36 Data used by manage orders

No.	Data/model	Operations	Role in the function
1	Order	Read, update	Items, totals, invoice, lifecycle, return, payment, shipping, and history
2	Transaction; PaymentMethod	Create, read, update	Payment attempts, reconciliation, refund results, and saved refund destination
3	Inventory	Read, update	Reservation, commitment, restoration, and stock consistency during order transitions
4	ShippingAreaMapping; GHN payload	Create, read, update	Address mapping, fee/source state, shipment code, tracking, and webhook/sync data
5	User; CustomerNotification	Read, create	Customer context and order/payment/shipping notifications
6	AuditLog	Create	Actor, reason, outcome, and administrative trace for sensitive actions

Processing: The Admin Portal sends queue, lookup, filter, sort, pagination, and export parameters to permission-protected order endpoints. The backend returns authorized orders and validates every lifecycle transition against payment, shipping, return, and inventory rules. Detail actions use the current order state: payment operations record transaction outcomes; GHN operations require a verified mapping or preserve an explicit fallback state; return approval or rejection records evidence and restores the appropriate state; and bulk actions process each selected order independently, reporting successes and retaining failures. Sensitive mutations record the actor and reason and notify affected customers where applicable.

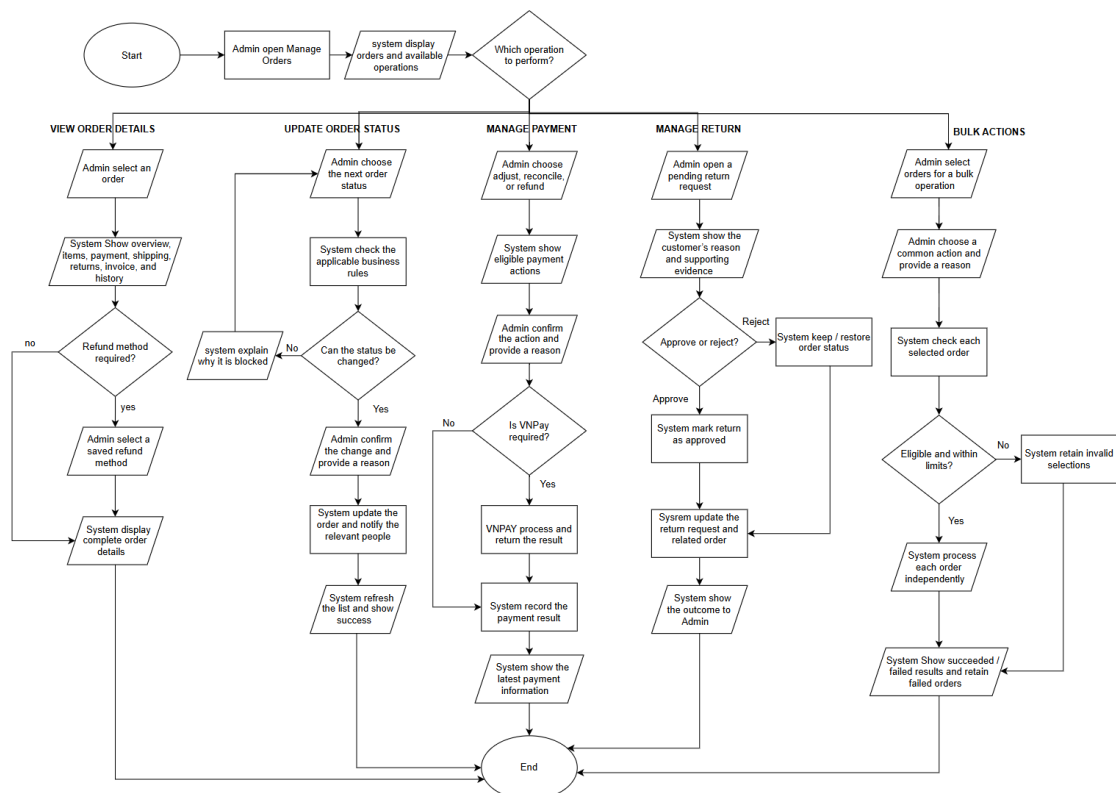


Figure 3.44 Admin order-management flow

3.5.19. Manage Customer Support

- **Purpose:** Triage tickets, communicate with customers, and maintain reusable support knowledge.
- **Users:** Administrators and staff with support reply/manage permission.
- **Interface:**

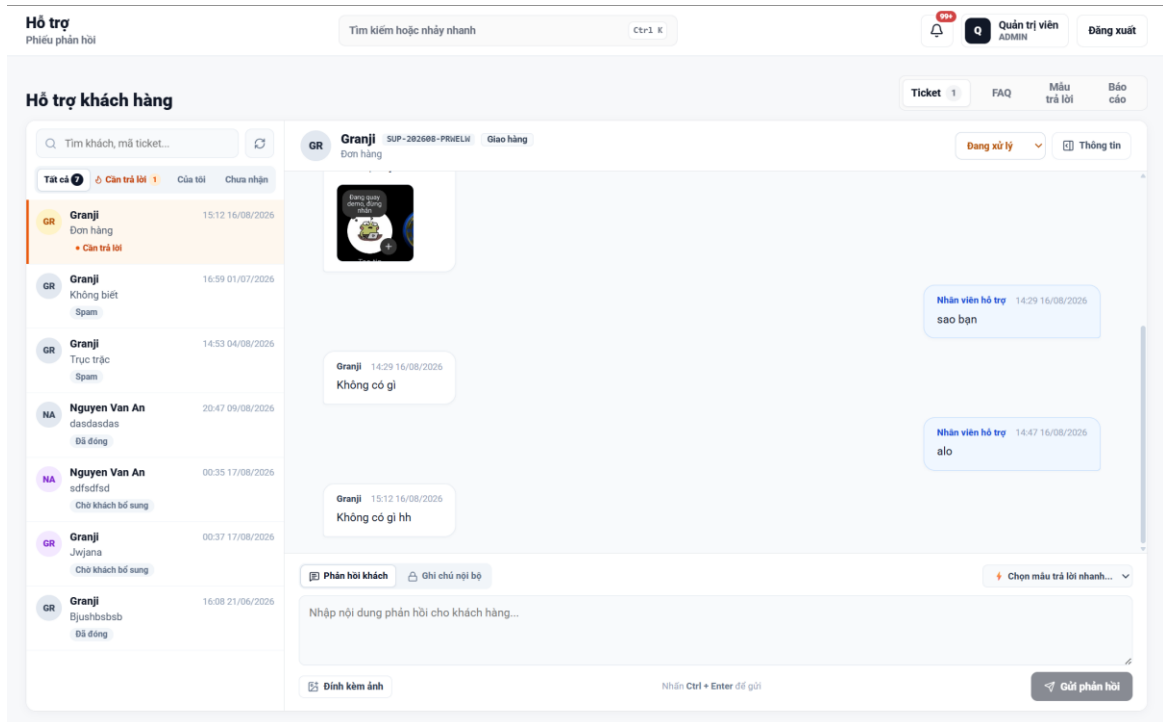


Figure 3.45 Admin customer-support management interface

Table 3.37 summarizes the main interface components.

Table 3.37 Main interface components for managing customer support

No.	Control	Purpose	Default value
1	KPI/filter controls	Filter status, type, category, priority, assignee, reply need, order, and date	Open workload
2	Inbox/detail panels	Review ticket context and conversation	Selected ticket
3	Reply controls	Send customer replies or internal notes with attachments/templates	Empty
4	Workflow controls	Assign, prioritize, resolve, close, reopen, or mark spam	Permission-based
5	Knowledge/analytics panels	Maintain canned responses and FAQs and review service metrics	-

Table 3.38 summarizes the principal data used by the function.

Table 3.38 Data used by manage customer support

No.	Data/model	Operations	Role in the function
1	SupportTicket	Read, update	Queue, assignment, priority, order link, and lifecycle
2	SupportMessage	Create, read	Staff replies and internal notes

No.	Data/model	Operations	Role in the function
3	SupportCannedResponse; FaqArticle	Create, read, update, delete	Reusable replies and public guidance
4	User; Order	Read	Customer and order context
5	AuditLog	Create	Ticket management actions

Processing: The admin queue excludes unverified guest feedback and prioritizes tickets needing a reply. Authorized staff can assign and update tickets only through valid state transitions. Public replies update reply flags and response timestamps, trigger email/push and real-time events, and increment canned-response usage; internal notes remain staff-only. FAQ and support analytics are maintained in the same module.

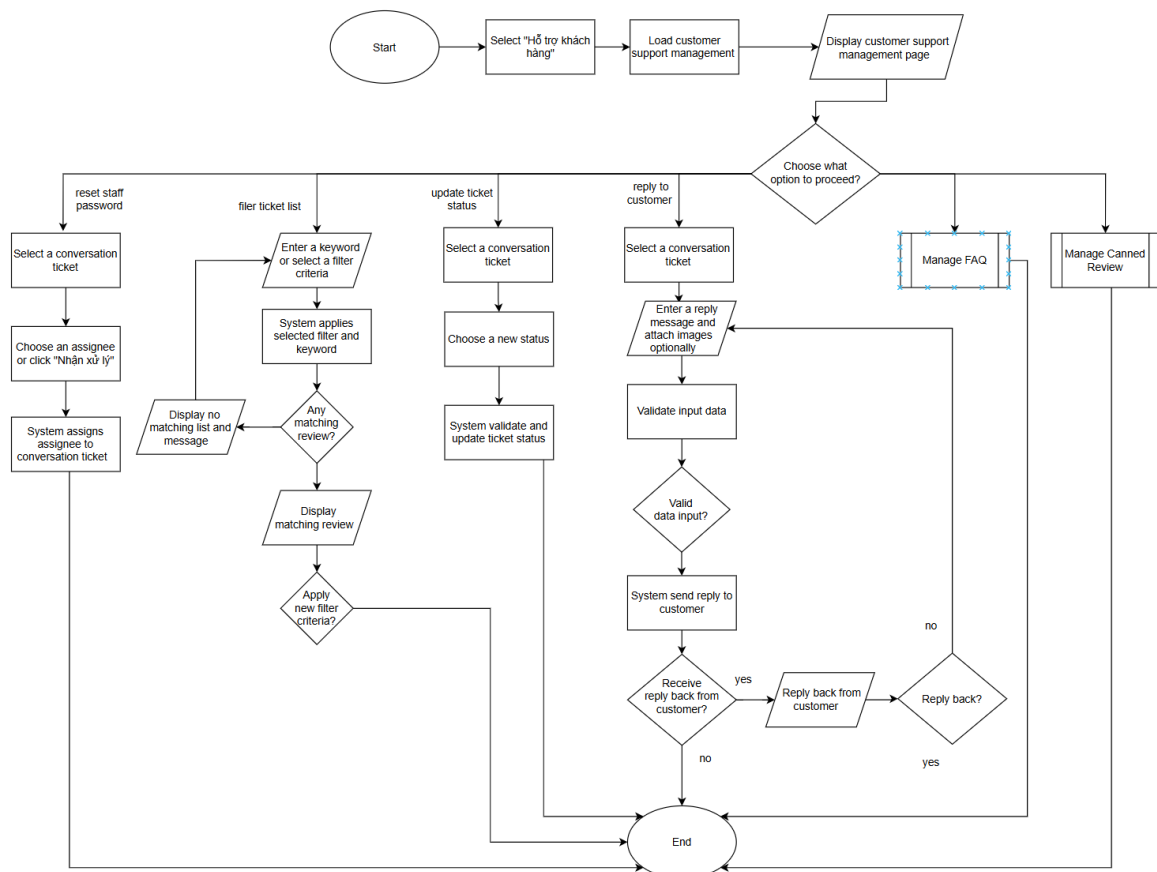


Figure 3.46 Admin customer-support management flow

3.5.20. Search & Recommendation Analytics

- **Purpose:** Measure recommendation engagement and discover search demand or search-result gaps.
- **Users:** Administrators and staff with reports read permission.

- Interface:

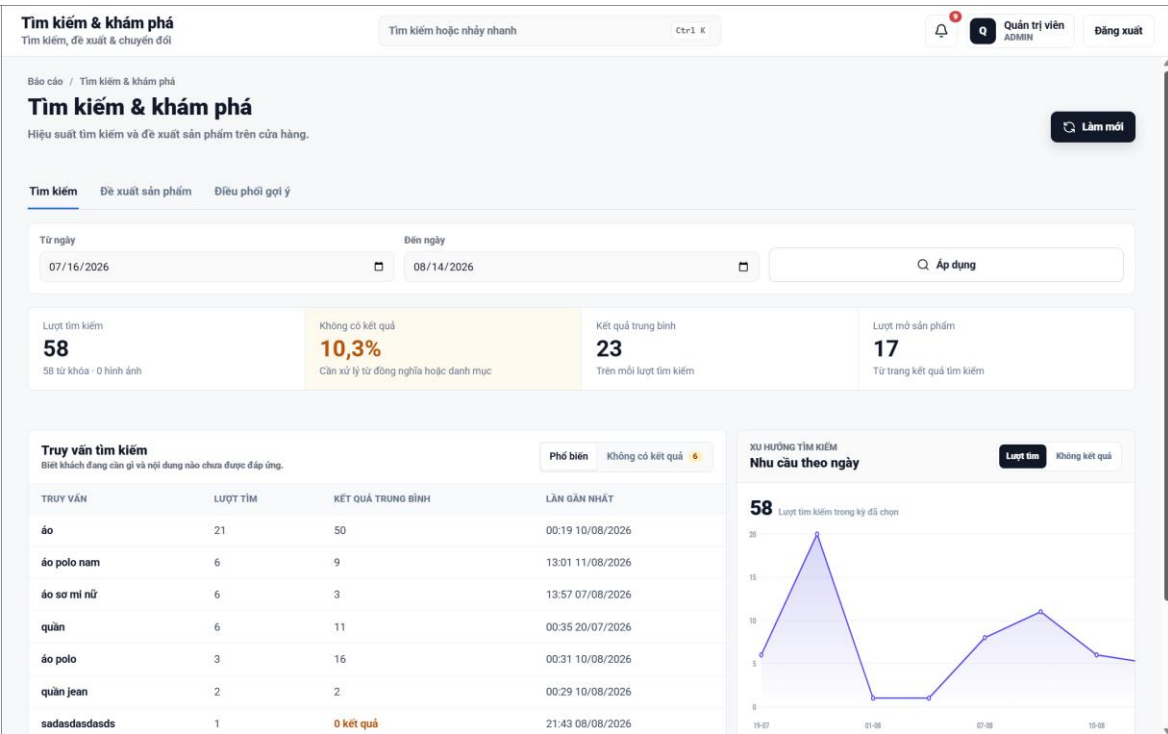


Figure 3.47 Admin search analytics interface

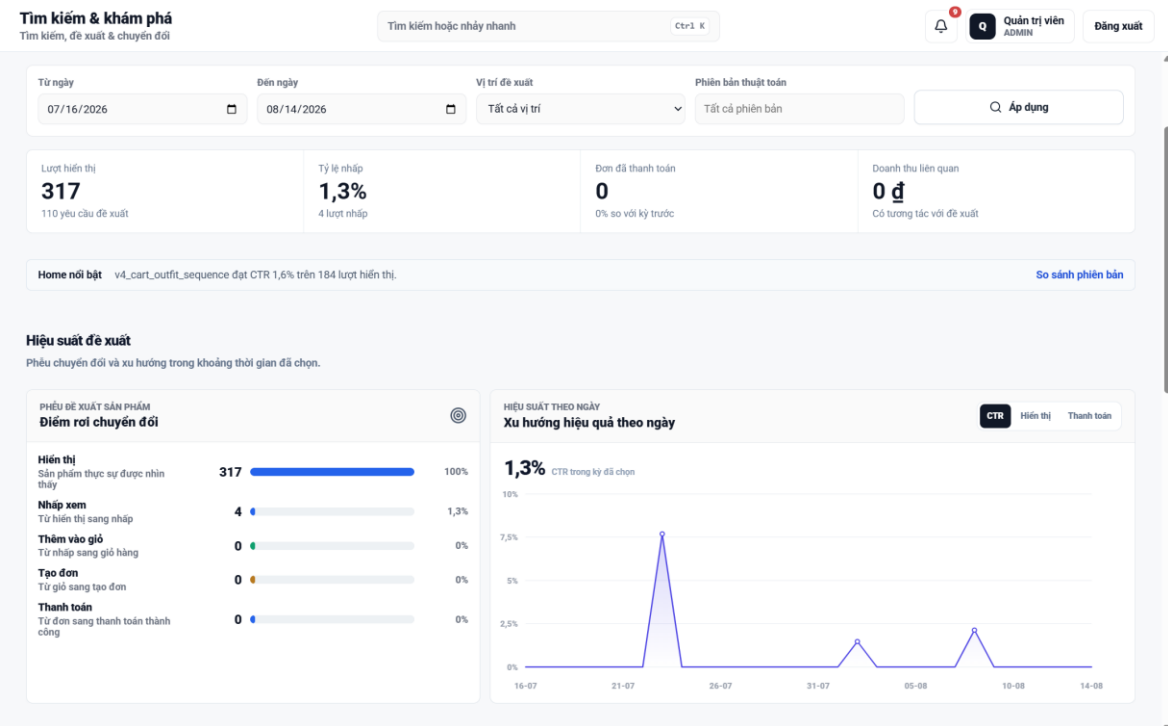


Figure 3.48 Admin recommendation analytics interface

Table 3.39 summarizes the main interface components.

Table 3.39 Main interface components for Search & Recommendation Analytics

No.	Control	Purpose	Default value
1	Filters	Choose date range, granularity, recommendation context, and algorithm version	30 days / all contexts
2	Metric cards	Show impressions, clicks, CTR, add-to-cart, purchase, conversion, coverage, and fallback.	Selected range
3	Trend chart	Compare recommendation performance over time	Daily
4	Search panels	Show query volume, no-click queries, and low-result terms	Top terms
5	Product panel	Show products driving recommendation conversion	Ranked

Table 3.40 summarizes the principal data used by the function.

Table 3.40 Data used by Search & Recommendation Analytics

No.	Data/model	Operations	Role in the function
1	RecommendationRequest	Read	Context, version, candidates, coverage, and fallback
2	RecommendationEvent	Read	Impression, click, cart, and purchase attribution
3	SearchHistory	Read	Queries, result counts, clicks, and conversion
4	UserProductInteraction; Product	Read	Behavior and product-segment context

Processing: The service validates a date-only range of at most 180 days and optional context/version filters. It aggregates request coverage and funnel events into CTR and conversion metrics, compares the previous period, and groups trends and segments. Search analytics identify repeated, no-click, and low-result queries and relate successful sessions to product conversion.

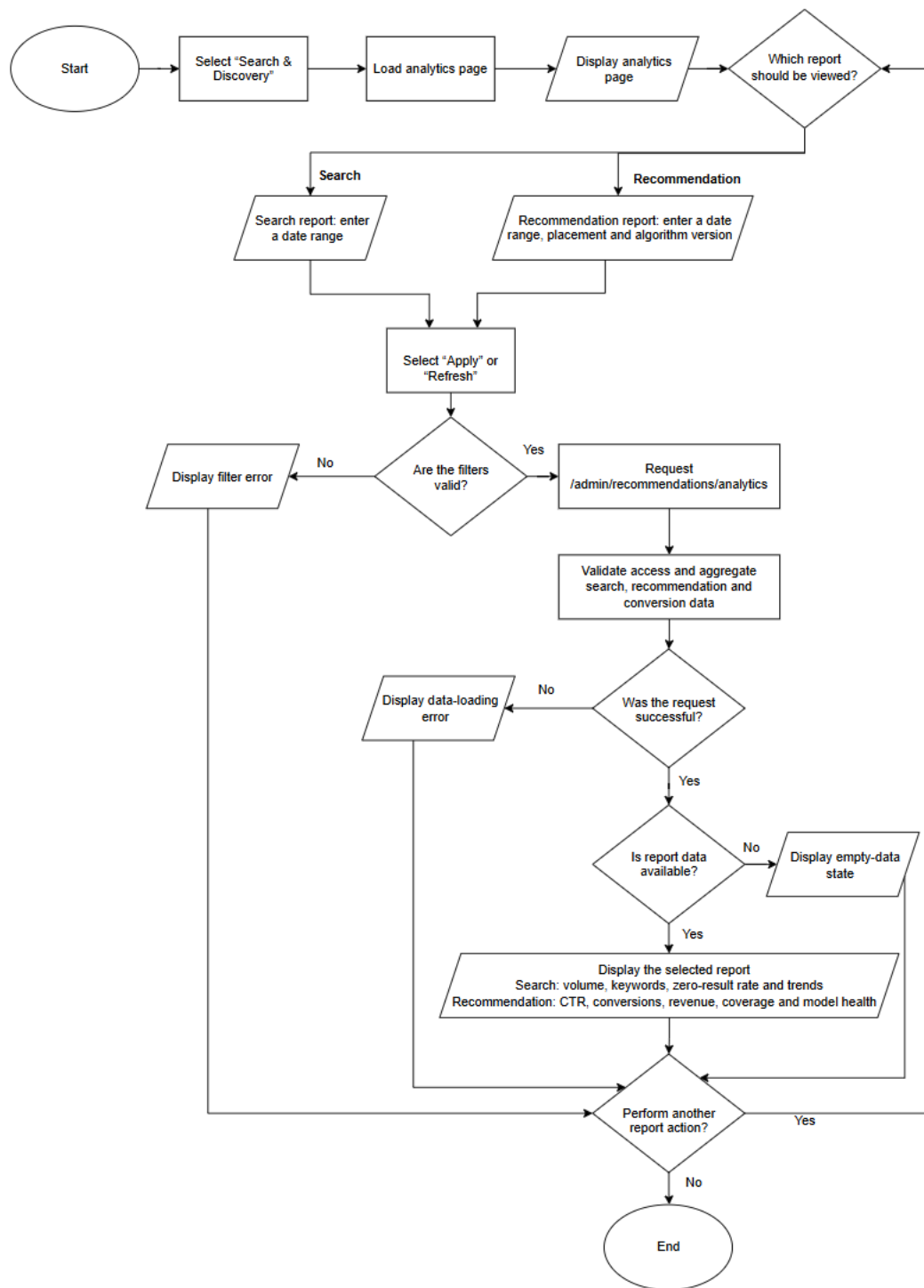


Figure 3.49 Admin search and recommendation analytics flow

3.5.21. Manage Virtual Try-On

- **Purpose:** Operate virtual try-on jobs, safety controls, customer restrictions, and provider configuration.
- **Users:** Administrators and staff with virtual try-on read/manage/settings permission.

- Interface:

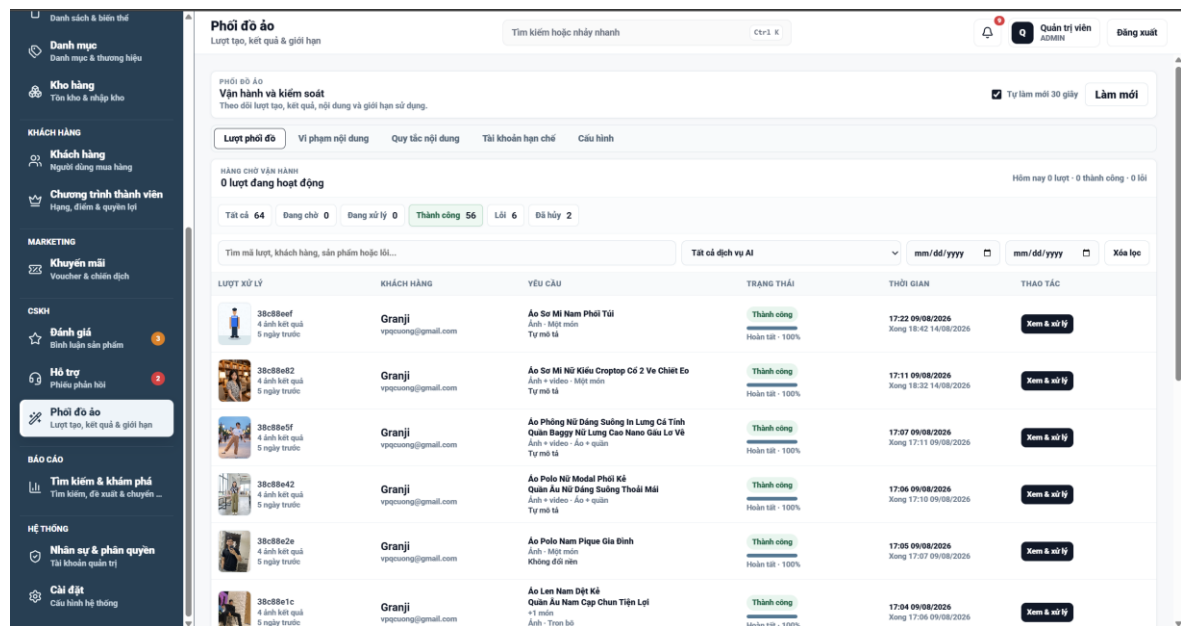


Figure 3.50 Admin virtual try-on management interface

Table 3.41 summarizes the main interface components.

Table 3.41 Main interface components for managing virtual try-on

No.	Control	Purpose	Default value
1	Summary/tabs	Show job health and switch among jobs, violations, rules, locks, and settings.	Jobs
2	Search/filters	Filter customer, product, service, date, and job state	All
3	Job actions	Inspect, retry image/video, cancel, or hide a job	Permission-based
4	Safety controls	Review violations and create, edit, or delete prompt rules	Current rules
5	Account controls	Lock or unlock a customer's try-on access with a reason	-
6	Settings controls	Test prompts, update versioned limits/providers, or roll back	Current version

Table 3.42 summarizes the principal data used by the function.

Table 3.42 Data used by manage virtual try-on

No.	Data/model	Operations	Role in the function
1	VirtualTryOnJob; VirtualTryOnAsset	Read, update, delete	Requests, progress, outputs, and media

No.	Data/model	Operations	Role in the function
2	VirtualTryOnSettings	Create, read, update	Versioned capabilities, providers, limits, and feature flags
3	VirtualTryOnPrompt Rule; VirtualTryOnPrompt Violation	Create, read, update, delete	Prompt-safety policy and evidence
4	VirtualTryOnAccount Lock; User	Create, read, delete	Customer restrictions and identity
5	AuditLog	Create	Operational and settings changes

Processing: Read permission exposes summaries and jobs; manage permission is required for retry, cancel, hide, violations, and account locks; settings permission is required for prompt rules and versioned configuration. Each mutation validates current state and records its reason. Settings updates use expected versions, prompt testing applies the same policy as production, and rollback creates a controlled new active version.

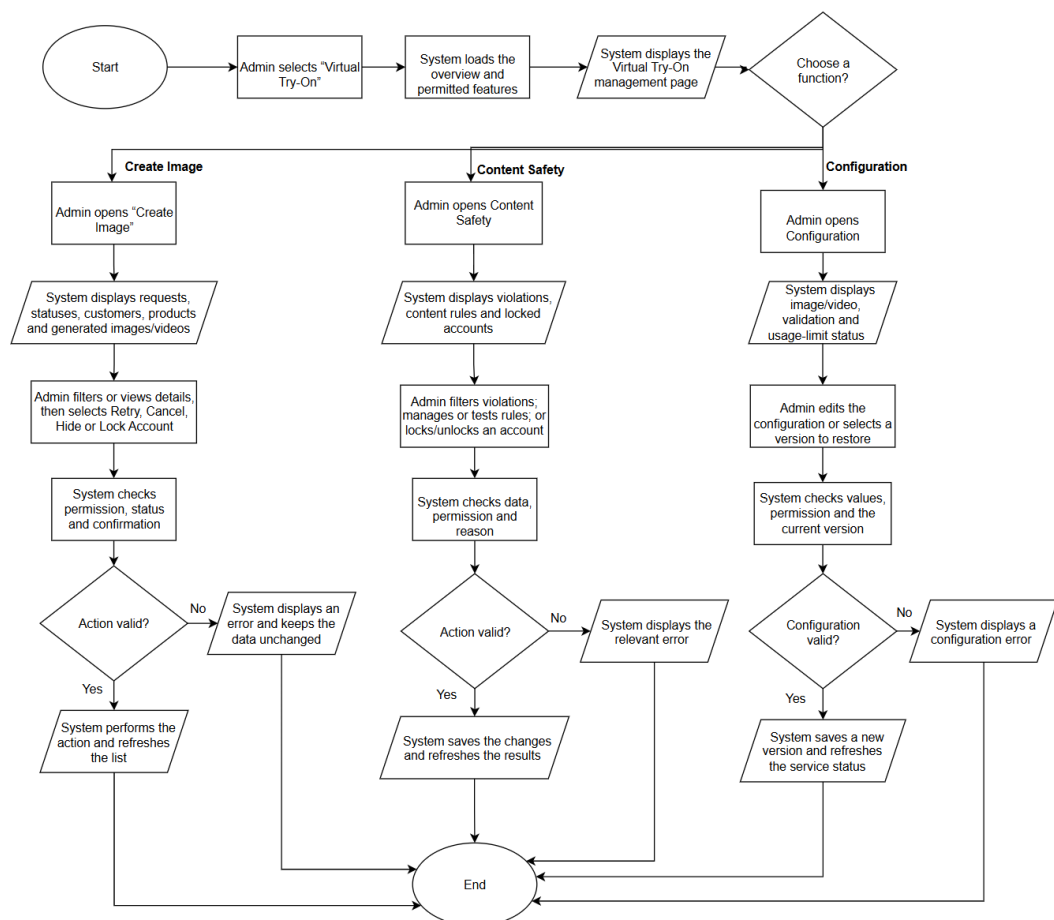


Figure 3.51 Admin virtual try-on operations and settings flow

3.6. Recommendation System Design

3.6.1. Formula selection procedure

Validation-selected coefficients were obtained through the controlled offline procedure described below. The deployed Product Detail configuration uses the validation-selected coefficients, whereas Home and Cart retain application-specific production configurations, as distinguished in Table 4.3. Each context uses its own relevance label, baseline, candidate set, and search space; only training data constructs profiles, popularity, metadata vocabularies, and association statistics.

- Prepare chronological splits and fixed candidates. Amazon Fashion supplies 646 validation and 100 locked final cases for Home and Product Detail; Polyvore supplies 1,000 validation outfits and 142 official FITB final cases for Cart.
- Define a traditional baseline per context: Most Popular for Home, cosine similarity for Product Detail, and Association Lift for Cart. Project and baseline receive identical candidates and diversity penalties.
- Run grid search on validation only. Home evaluates 201 preference/popularity pairs; Product Detail evaluates 672 unique coarse/fine combinations; Cart evaluates 1,771 four-weight simplex combinations.
- Select by NDCG@5, then HR@5, MAP@5, MRR@5, Recall@5, Precision@5, and coverage. Lock the coefficients before opening the final split; the final data never participates in selection.

Validation selected P only for Home, $0.125A + 0.875C$ for Product Detail, and $0.60T + 0.40L$ for Cart. Production retains $0.90P + 0.10B$ and $0.20R + 0.25T + 0.50L + 0.05B$ for Home and Cart because sale/new-arrival and garment-role safeguards are not fully represented in the public datasets. This distinction prevents an offline proxy optimum from silently replacing domain rules.

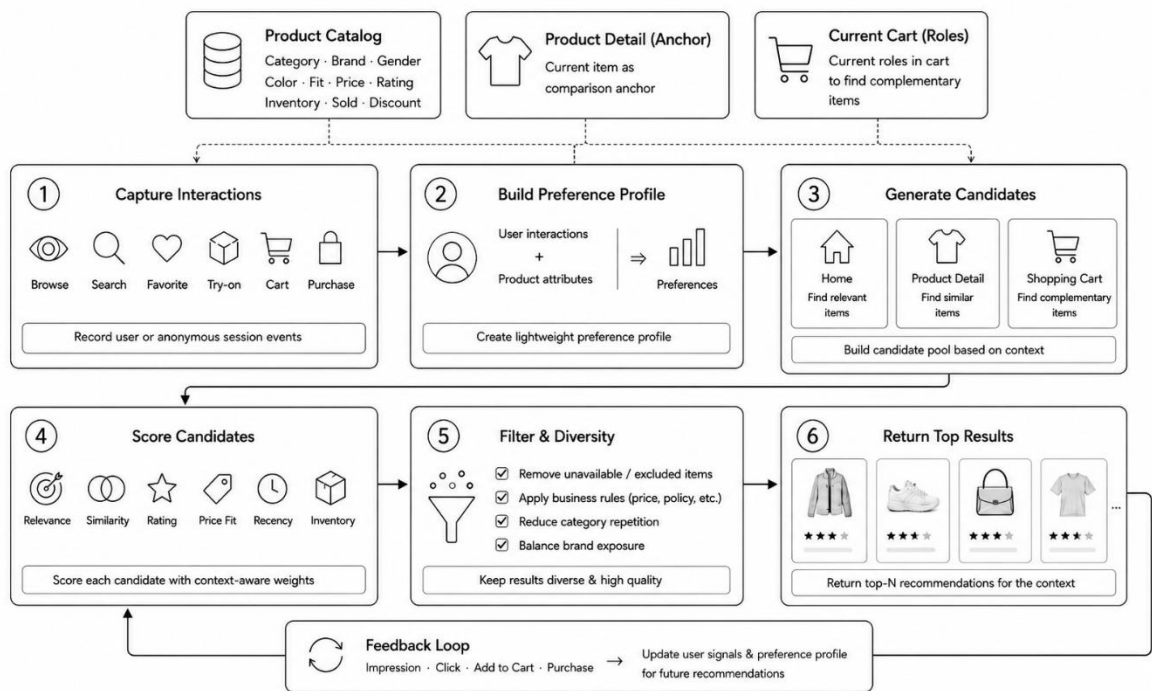


Figure 3.52 Data flow from event tracking to ranked recommendations.

3.6.2. Online ranking workflow

- Generate a context-specific candidate pool and remove inactive, unavailable, excluded, or already selected products.
- Build normalized signals from user interactions, product metadata, Cart roles/styles, training-basket association, and business flags.
- Apply the deployed context formula, sort deterministically, and rerank with repeated-category and repeated-brand penalties of 0.08 and 0.04.
- Return the Top-K list with reason codes; use popularity/business or rule-only fallback when personalized or association evidence is unavailable.
- Persist recommendation requests, impressions, clicks, and attributed downstream events for analytics and later recalibration.

3.7. Virtual Try-On System Design

Section 2.3 defines the models and their responsibilities. This section records how those components are assembled into the implemented request, prompt, queue, persistence, and delivery workflow.

3.7.1. End-to-End Processing Procedure

Based on the inputs and responsibilities above, the system processes each request in the following sequence. The sequence is deliberately staged: inexpensive validation is performed first, catalog evidence is snapshotted before generation, and every long-running provider operation is reflected in the persisted job state.

- **Acquire and persist the source image:** The customer uploads a gallery image or captures a camera image. The backend verifies authentication, account state, file metadata, and ownership, uploads the source to managed media storage, and creates a VirtualTryOnAsset record.
- **Validate the source image:** OpenCV evaluates resolution, blur, and brightness. YOLO11 Pose detects the principal person, estimates keypoint confidence, collapses nested person detections, and derives visible body regions. The service returns supportedModes, blockedModes, recommendedMode, and per-mode capabilities. A missing person is a hard block; the customer may confirm a soft warning.
- **Validate the selection:** The backend accepts one to four active catalog items, resolves the exact product/variant/color variant, verifies that roles are unique and compatible with outfitMode, snapshots name/color/image/price/size, and checks the source image capability for the selected roles.
- **Prepare garment references:** Catalog images are downloaded and sent to the FastAPI garment-processing service. Transparent images use alpha-based extraction; opaque or complex images use Grounding DINO followed by SAM 2.1. Each item returns a rectangular crop, confidence, warnings, issues, and isUsable. The service also creates a 768 x 768 white-background collage for workflows that accept only a single garmentImage input.
- **Choose the current workflow input form:** The checked-in Gemini workflow contains a multiGarment mapping, so the backend uploads the individual prepared garment crops as separate reference images. If a workflow lacks multiGarment but exposes garmentImage, the backend uploads the prepared collage instead. If preparation is disabled, it maps the original catalog images according to the workflow contract.
- **Validate and construct the prompt:** The optional custom scene text is normalized and checked against built-in and administrator-defined prompt-policy rules. The backend then assembles the positive prompt from source-framing, garment description, outfit mode, role, context, garment-fidelity, and realism blocks; it assembles a base negative list plus role-specific errors. Exact text appears in Appendix D.
- **Map provider inputs:** The backend clones the configured JSON workflow and maps personImage, prepared garment images, prompt, seed, model, aspect ratio, and resolution. The current defaults are a 3:4 aspect ratio and 2K image resolution. The current map has no separate negativePrompt node, so the negative prompt is appended to the positive input under a STRICT AVOID LIST section.
- **Submit and monitor generation:** The backend uploads inputs to ComfyUI, calls POST /prompt, obtains prompt_id, and polls workflow history until

outputs appear, a provider error occurs, or the configured timeout is reached. Rate-limit responses use bounded backoff.

- **Persist image outputs:** The Gemini workflow creates a single 2x2 portrait grid, then saves four cropped 3:4 images. The backend downloads all image outputs, uploads them to Cloudinary, creates generated_image assets, and stores their identifiers and URLs on VirtualTryOnJob.
- **Optionally generate video:** For image_and_video, the first successful image is uploaded as the exact first frame. The Kling workflow receives the video prompt, duration, 720p resolution, audio=false, and model identifier. A video-stage failure does not remove already successful images.
- **Deliver the result:** Job progress and terminal state are emitted through Socket.IO. The mobile client also polls REST every 2.5 seconds as a fallback and rejects stale updates using updatedAt, status order, and progress. The result interface can save/share media and transfer the selected-sized products to Cart or Checkout.

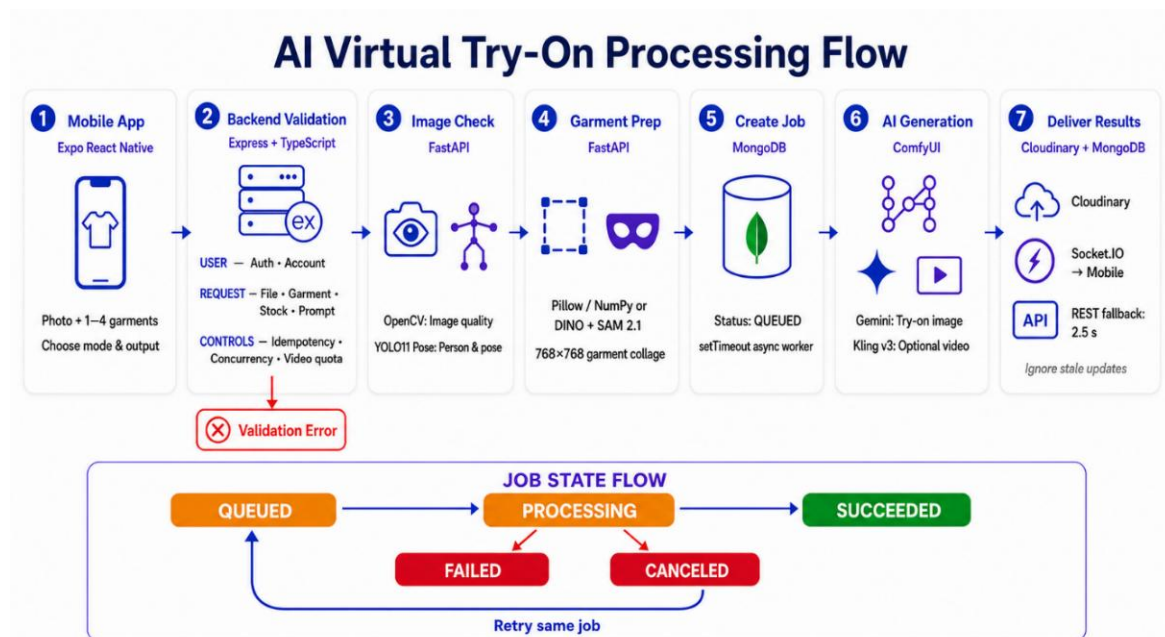


Figure 3.53 AI virtual try-on processing and job-state flow

Figure 3.53 connects these stages with the job lifecycle. From the customer's perspective, submission returns a job immediately; generation continues in the background, while Socket.IO events and REST polling expose progress. From the system's perspective, this separation prevents a long-running AI request from occupying a mobile HTTP connection and enables retry, cancellation, history, and administration.

3.7.2. Prompt Construction Method

After the visual references are validated, the prompt instructs the provider on how to use them. It is not a free-form instruction written directly by the customer. Instead, the backend constructs it deterministically, giving identity and catalog evidence higher priority than optional scene text. This design limits conflicting instructions and makes the effective request reproducible during debugging or evaluation. The blocks are assembled in the following order:

- **Identity and framing:** states that the person image is the sole identity source and prohibits expanding cropped photographs or inventing invisible body regions.
- **Garment description:** inserts each selected role, catalog name, color, size, and role-specific alignment requirements.
- **Outfit mode:** inserts one of the single-item, two-piece, or complete-outfit rules, including waist overlap and layering constraints.
- **Scene context:** keeps the old background for none, or requests a single coherent replacement environment for a preset or custom context. Custom text is treated as scene context only.
- **Garment fidelity and realism:** designates catalog images as the mandatory wardrobe source and specifies fabric, seams, closures, shadows, occlusion, and contact realism.
- **Negative constraints:** It combines a base safety/quality list with the error patterns relevant to the selected roles. Because the current workflow map has no separate negative input, the backend injects this list into the positive prompt as a strict avoid block.

The application-level prompt is then wrapped in a provider wrapper. This wrapper fixes the 2x2 output layout, assigns reference image 1 to the person and subsequent references to the garments, preserves the baseline pose in cell 1, and permits only subtle pose variation in cells 2-4. Thus, scene text can influence the environment but cannot legitimately replace identity or wardrobe evidence. Appendix D provides the exact fixed strings, runtime placeholders, role-specific blocks, negative constraints, and a fully expanded example.

3.7.3. Asynchronous Job State and Failure Handling

Because image and video generation may take longer than an ordinary API request, the system represents execution as a persisted state machine. The job status answers whether the image-centric task has completed, the processing stage explains what the worker is doing, and videoStatus records the optional second-stage result independently.

Table 3.43 Persisted VTO state model

Layer	States/stages	System behavior
Job status	queued, processing, succeeded, failed, canceled	Represents the overall image-centric job lifecycle.

Layer	States/stages	System behavior
Processing stage	queued, image_generation, image_persisting, video_generation, video_persisting, completed	Explains the current operation displayed by the client.
Video status	not_requested, queued, processing, succeeded, failed, canceled	Separates optional video outcome from the image result.
Worker	BullMQ image/video queues on Redis; in-process fallback	When queue mode is enabled, BullMQ persists and dispatches image/video jobs through Redis. When disabled, the same service uses in-process scheduling.
Retry	Reuse failed/canceled job.	Resets state and resubmits eligible work; policy-closed failures cannot be retried.
Delivery	Socket.IO + 2.5 s REST polling	Provides real-time progress with a resilient fallback.

The state model also clarifies evaluation. Operational success is not equivalent to visual quality: a provider may return a valid file that still changes identity or garment details. Technical evaluation should therefore measure rejection rate, provider success/failure rate, retry rate, mean and P95 processing time, and video completion. Visual evaluation should separately assess identity preservation, garment fidelity, pose consistency, artifact severity, and human preference. [2], [21]

CHAPTER 4. TESTING AND EVALUATION

4.1. Introduction

This chapter verifies the implemented Fashion E-commerce System against the current use cases and processing flows. The executed scope covers backend unit/integration logic, mobile application logic, Web Admin presentation and service contracts, and isolated backend API workflows. Results are reported exactly as executed on 14 August 2026; live-provider and model-runtime limitations are separated from deterministic application results.

4.1.1. Testing Objective

Testing verifies that the implemented functions follow the consolidated use cases, reject invalid or unauthorized operations, and preserve consistent data during state changes. Particular attention is given to account access, role-based authorization, pricing, inventory, payment, shipping, support, recommendation, and virtual try-on workflows. Transaction, idempotency, retry, timeout, and fallback behavior are also checked so that the results provide repeatable evidence for maintenance and later deployment testing.

4.1.2. Testing Scope

Testing focuses on deterministic behavior and API workflows that can be repeated locally. Backend Jest suites cover controllers, services, middleware, models, utilities, security rules, and provider adapters. Mobile Jest suites cover authentication, navigation guards, cache behavior, search, catalog selection, favorites, cart/checkout presentation, orders, reviews, support, notifications, recommendations, storefront settings, and virtual try-on state. Playwright unit tests cover Web Admin session, dashboard, catalog, inventory, order, invoice, recommendation-analytics, and HTTP contracts. Backend end-to-end suites exercise authenticated workflows with isolated persistence and controlled provider seams.

4.2. Test Plan

The test plan traces the current use cases to normal, alternate, authorization, and exception paths. High priority is assigned where an incorrect result could affect account access, money, stock, order state, customer data, or safety policy. The representative cases below summarize use-case coverage; the repository remains the source of detailed test functions.

4.2.1. Features To Be Tested

The tested scope follows the main customer journey from account registration and sign-in through product discovery, search, filtering, favorites, cart management, vouchers, checkout, payment, and order history. It then continues with post-order functions such as delivery status, returns, reviews, support, policies, membership, and notifications. Administration coverage includes customer and staff accounts, permissions, order and shipping operations, support, storefront settings, reports, recommendation analytics, and virtual try-on management. The intelligent modules are verified through recommendation ranking and event integrity, search analytics, virtual try-on validation and prompt policy, asynchronous job transitions, retry and cancellation behavior, and optional video state.

4.2.2. Features Not To Be Tested

This local execution does not certify live VNPay settlement, GHN delivery, SMS, email, Google identity, Cloudinary, or ComfyUI availability because their production credentials, networks, and service contracts are external to the deterministic environment. Full Android-device usability, cross-browser compatibility, visual acceptance, accessibility, penetration testing, long-duration reliability, and production-scale load testing are also outside this cycle. The image-validation and garment-processing pytest suites were not rerun because their model-specific runtime is maintained as a separate verification target.

4.2.3. Testing Approach

The project uses layered, risk-based verification. Unit tests isolate validation, calculation, ranking, permission, presentation, and state-transition rules. Controller and route tests verify request mapping, authentication, and authorization. Integration and end-to-end tests exercise multi-module workflows with isolated data. External providers are mocked or simulated where the test targets application behavior; whenever a test explicitly expects a live-like provider result, fallback output is reported as a failure rather than silently accepted. This distinction prevents deterministic correctness from being confused with live-service availability.

4.2.4. Pass/Fail Criteria

A case passes when its returned value, state transition, response status, side effects, and authorization decision match the expected result. It fails when an assertion differs, an unexpected exception or timeout occurs, a protected operation succeeds without permission, or a rejected transaction leaves a partial write. A test group passes only when every included case passes and the runner exits with code 0.

4.2.5. Suspension and Resumption Criteria

A test group is suspended only when its database, runtime, fixture, build, or mocked provider cannot initialize and the resulting infrastructure problem would invalidate the remaining observations. Execution resumes after the environment is restored, the blocking defect is corrected, and the affected group is rerun from clean data. A business-rule failure does not suspend unrelated groups; it remains recorded as a failed case while the affected workflow is isolated for correction.

4.3. Test Management

The developer prepares fixtures, executes the suites, records results, investigates failures, and reruns the affected group together with related regression tests. Test data is isolated from production data and external providers are mocked or simulated unless live integration is explicitly under evaluation.

4.3.1. Test Execution Process

Execution begins by tracing the implemented requirements to their critical normal, alternate, authorization, and exception paths. The developer prepares isolated fixtures, mocks, and in-memory persistence where required, then runs the backend Jest, mobile Jest, Web Playwright unit, and backend end-to-end suites from their project workspaces. Suite and case counts, exit status, duration, and environment exclusions are recorded. Any failure is investigated, corrected in either the implementation or the test expectation, and then reran with the affected group and its related regression tests.

4.3.2. Test Environment

The tests were executed locally on Windows. The backend environment uses Node.js, TypeScript, Jest, ts-jest, Express, Mongoose, and mongodb-memory-server for isolated persistence where required. The mobile environment uses React Native 0.81, Expo SDK 54, TypeScript, Jest, and jest-expo, while the Web Admin environment uses React 19, TypeScript, Vite, and Playwright component-level configuration. External services are mocked, simulated, or excluded unless an isolated provider-adapter test is available.

4.3.3. Testing Risks

Mocked provider tests can verify application handling but cannot prove the availability, latency, or contractual behavior of live payment, shipping, messaging, storage, identity, or generation services. Recommendation results also depend on the quantity and quality of production interaction data, so deterministic ranking tests do not demonstrate business uplift. Likewise, virtual try-on quality varies with pose,

garment images, prompts, model versions, and GPU resources; application-state tests do not measure perceptual realism. Automated logic tests therefore remain complementary to device, browser, accessibility, security, and load testing before production release.

4.4. Test Scenarios

Table 4.1 Test Scenario Summary

Scenario ID	Area	Related Use Cases and Verification Focus	Priority
TS-MOB-01	Customer identity and personal information	UC-MOB-01, UC-MOB-02, and UC-MOB-15: registration, sign-in, logout, profile, avatar, address, and password behavior	P1
TS-MOB-02	Product discovery and favorites	UC-MOB-03 to UC-MOB-07: product listing, search, filtering, product detail, variant selection, and add-favorite behavior	P1
TS-MOB-03	Cart, checkout, and payment	UC-MOB-08 and UC-MOB-09: stock-aware cart changes, checkout selection, authoritative quote, order creation, COD, and VNPay	P1
TS-MOB-04	Orders and reviews	UC-MOB-10 and UC-MOB-11: order history and detail, real-time state, and verified product reviews	P1
TS-MOB-05	Customer support	UC-MOB-12: public support content, ticket ownership, conversation state, staff replies, and polling fallback	P2
TS-MOB-06	Recommendations and virtual try-on	UC-MOB-13 and UC-MOB-14: contextual ranking, attribution integrity, input policy, asynchronous jobs, retry, history, and result media	P1
TS-ADM-01	Customer accounts, roles, and permissions	UC-ADM-01 and UC-ADM-02: customer-account support actions, Staff lifecycle, permission scopes, and protected targets	P1
TS-ADM-02	Order and customer-support administration	UC-ADM-03 and UC-ADM-04: order lifecycle, payment, shipping, return, bulk processing, ticket workflow, replies, and knowledge content	P1
TS-ADM-03	Search, recommendation, and virtual try-on administration	UC-ADM-05 and UC-ADM-06: search and recommendation analytics, try-on jobs and outputs, prompt policy, account locks, permissions, and versioned settings	P1

4.5. Test Cases

The representative cases provide one traceable test case for each of the 21 consolidated use cases: 15 customer mobile use cases and six Web Admin use cases. To keep this chapter focused on the testing rationale, Appendix A presents the detailed mapping, test data, expected results, and execution status; the repository remains the source of the complete test functions and fixtures.

4.6. Test Results

The executed suites were evaluated against the objectives, scope, approach, and pass/fail criteria described in this chapter. Because execution logs, case counts, pass/fail outcomes, and environment-specific observations are supporting evidence rather than the main testing argument, the detailed actual results are presented in Appendix B.

4.7. Recommendation System Offline Evaluation

4.7.1. Evaluation protocol

Functional tests check if things run smoothly, but this experiment focuses on how well the ranking works. In the home page, a future positive item is used, while the Product Detail page relies on a next-positive-item proxy based on the most recent training item. The Cart page aims for outfit completion. Both the project and baseline methods use the same set of candidates, filters, diversity reranking, and Top-K value, making comparisons fair and consistent.

Table 4.2 Offline recommendation evaluation design

Context	Public dataset	Validation	Locked final	Candidates	Baseline
Home	Amazon Fashion	646 cases	100 cases	200	Most Popular
Product Detail	Amazon Fashion	646 cases	100 cases	200	Cosine
Cart	Polyvore	1,000 outfits	142 FITB	4 / 200	Association Lift

4.7.2. Hyperparameter selection

NDCG@5 is the primary validation objective because it rewards relevant products near the top of a short mobile rail. The final split is evaluated only after the

selection process. Business data is excluded from the public-data search because Amazon and Polyvore lack corresponding sale or new-arrival indicators.

Table 4.3 Grid-search audit and selected configurations

Context	Search	Configs	Validation-selected formula	Production formula
Home	Full grid, 0.005	201	1.00P	0.90P + 0.10B
Product Detail	Coarse 0.05 + fine 0.005	672	0.125A + 0.875C	Same
Cart	4-D simplex, 0.05	1,771	0.60T + 0.40L	0.20R + 0.25T + 0.50L + 0.05B

4.7.3. Comparison results

The notation @K indicates that only the top K products are evaluated. HR@5 reports whether at least one relevant product appears in the top five. NDCG@K evaluates both relevance and ranking order, assigning greater weight to relevant products that appear closer to the top. FITB@1 is the proportion of Polyvore outfit-completion cases in which the correct missing garment is ranked first. Expanded NDCG@5 applies NDCG@5 to the expanded 200-candidate Cart evaluation. All reported metrics range from 0 to 1, with higher values indicating better performance. In Table 4.4, Difference equals Project minus Baseline: a positive value favors the project, zero indicates a tie, and a negative value favors the baseline.

Table 4.4 Locked final recommendation results.

Context	Final metric	Project	Baseline	Difference
Home	HR@5	0.080000	0.010000	+0.070000
Home	NDCG@5	0.061487	0.006309	+0.055178
Product Detail	NDCG@5	0.028175	0.028175	0.000000
Product Detail	NDCG@10	0.041380	0.041559	-0.000179
Cart	FITB@1	0.478873	0.408451	+0.070422
Cart	Expanded NDCG@5	0.088083	0.035336	+0.052747

Home improves clearly over Most Popular. Product Detail matches cosine at K=5 and remains nearly equal at K=10 while improving the former weighted-attribute formula. Cart reaches 0.478873 on official FITB@1, above Association Lift (0.408451), the current production hybrid (0.394366), and the rule-only formula (0.316901). Figures 4.1-4.3 present the comparison results. The corresponding evaluation notebooks are linked in Appendix E.

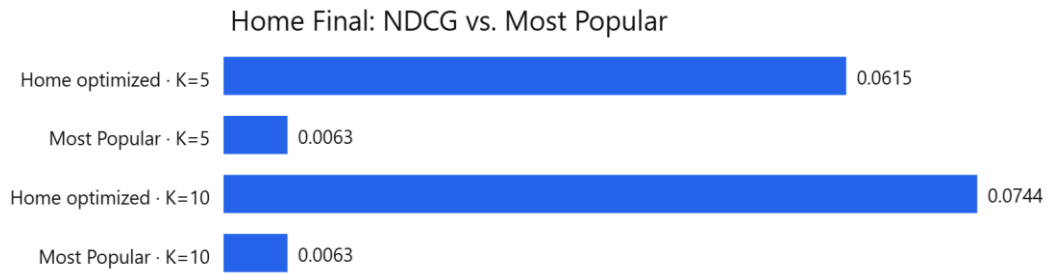


Figure 4.1 Comparison of recommendation formulas for the Home context

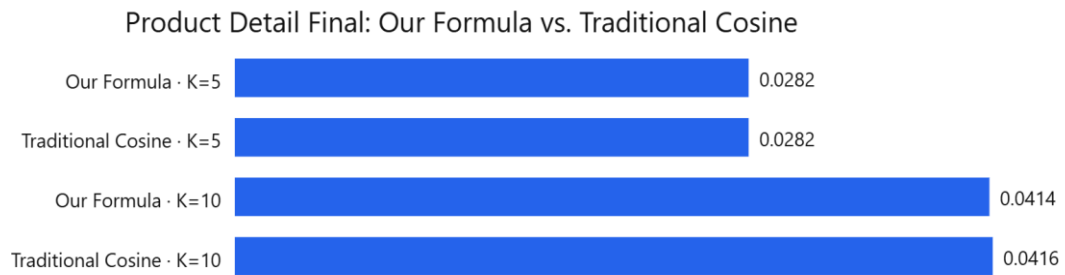


Figure 4.2 Comparison of recommendation formulas for Product Detail context

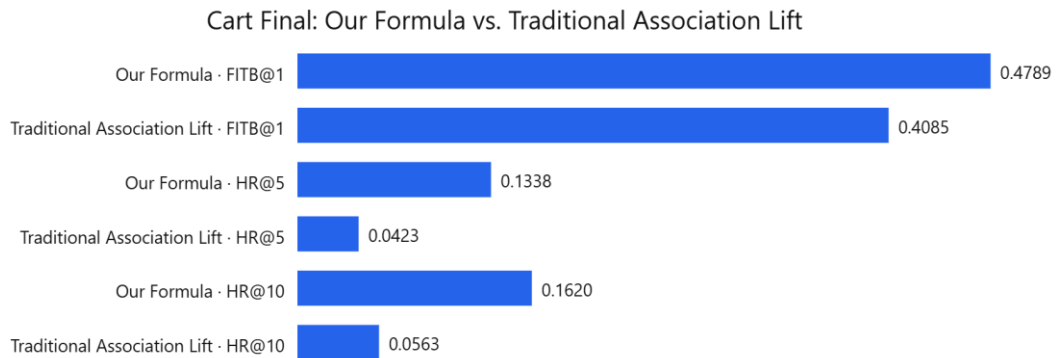


Figure 4.3 Comparison of recommendation formulas for the Cart context

4.7.4. Validity limits

These offline ranking results are based on implicit or proxy relevance, modest final samples, and public metadata that differs from the production catalog. They do not establish improvements in click-through, conversion, revenue, satisfaction, or perceived outfit quality; those claims require online A/B testing and user-based evaluation.

PART 3: CONCLUSION

1. Achieved Results

The project implemented a modular fashion e-commerce system that connects customer shopping, store administration, external commerce services, recommendation logic, and asynchronous virtual try-on. In the verification run on 14 August 2026, the backend unit/integration, backend end-to-end, mobile, and Web Admin unit groups all passed; the combined result was 1,160 passes out of 1,160 automated test cases.

1.1. Theoretical Contributions

This thesis achieved its main objective of designing and implementing an Android-focused fashion e-commerce system that combines essential online shopping functions with contextual product recommendations and virtual try-on capabilities. The system uses a modular client-server architecture comprising a customer mobile application, a responsive Web Administration Portal, backend services, a MongoDB database, and integrations with external payment, shipping, media storage, notification, and AI services.

The customer application supports the core fashion-shopping workflow, including account management, product discovery, product variants, favorites, cart and checkout, payment, order tracking, returns, reviews, customer support, notifications, personalized recommendations, and virtual try-on. The Web Administration Portal provides permission-controlled functions for managing catalog data, inventory, orders, payments, shipping, promotions, loyalty programs, customer support, reviews, system analytics, and virtual try-on operations.

Two intelligent modules were integrated into the system. The recommendation module applies deterministic, context-aware ranking for Home, Product Detail, and Shopping Cart. Offline evaluation showed clear gains over Most Popular for Home and Association Lift for Cart, while Product Detail reached parity or near-parity with cosine. The virtual try-on module validates person and garment images, prepares garment inputs, dispatches asynchronous image/video jobs, tracks progress, and stores generated results. The contribution lies in system design and integration rather than in training a new recommendation or generation model.

Testing covered the backend, mobile application, Web Administration Portal, and selected end-to-end workflows. The results indicate that the implemented functions, business rules, authorization mechanisms, state transitions, and fallback

behaviors generally performed as intended in the controlled test environment. However, integrations with live third-party services remain dependent on provider availability, credentials, and environment configuration. Therefore, additional integration and production-level testing are required before the system can be considered ready for commercial deployment.

Overall, the implemented prototype demonstrates the feasibility of combining mobile fashion commerce, contextual recommendations, and asynchronous virtual try-on within a unified system. It provides a practical foundation for further research, evaluation, and production-oriented development.

1.2. Software Implementation Results

The implementation delivered three coordinated components: an Android customer application, a responsive web application with a permission-controlled Administration Portal, and a modular backend system. The customer application supports the core shopping journey, from account management and product discovery to checkout, order tracking, post-purchase services, personalized recommendations, and virtual try-on.

The Administration Portal supports the core operational activities of a fashion retailer, including catalog, inventory, orders, payments, shipping, promotions, customer support, and content management. The backend coordinates these components with external services while enforcing authentication, role-based authorization, data validation, transaction consistency, and controlled state transitions. Together, these components provide an integrated workflow for customers and store personnel.

1.3. Practical Applications

The implemented system demonstrates the practical feasibility of integrating mobile fashion commerce, contextual product recommendations, and virtual try-on on a unified platform. Customers can discover suitable products, compare available variants, receive recommendations, preview selected garments, and complete the purchasing and post-purchase processes through the mobile application.

Store personnel can use the Administration Portal to manage daily business activities via permission-controlled functions and centralized operational information. Therefore, the system can serve as a prototype foundation for small and medium-sized fashion retailers seeking to develop mobile shopping and centralized store-management capabilities. However, further evaluation of usability, security, performance, and live-service operation is required before commercial deployment.

2. Limitation

Although the main objectives have been met, the mobile application currently focuses on Android and has not undergone a full iOS evaluation. Evidence for the recommendation remains limited by implicit/proxy labels, modest final samples, public-data metadata differences, and the use of deterministic scoring rather than a trained collaborative model. Virtual try-on quality depends on input images, external generation services, configured ComfyUI workflows, and computing resources; large-scale perceptual and user-based studies remain outside scope.

The system currently uses VNPay as its primary online payment gateway. Consequently, payment, shipping, storage, notification, and image-generation features depend on external credentials, service availability, and environment configuration. Furthermore, production-oriented evaluations, including large-scale load testing, security penetration testing, accessibility assessment, long-term reliability testing, and disaster-recovery testing, have not yet been comprehensively performed.

3. Future Development Direction

Future work should expand recommendation sampling, collect explicit similarity and outfit-quality judgments, and run controlled online A/B tests before claiming business uplift. With sufficient interaction data, the deterministic hybrid can be compared with collaborative, sequence-aware, or learning-to-rank models under the same locked protocol. Virtual try-on should improve garment-detail preservation, body-shape handling, multi-garment consistency, video generation, and perceptual quality evaluation.

In addition, payment and shipping integrations could be expanded to support more service providers. Related mechanisms, including timeout handling, retries, transaction reconciliation, refunds, and provider fallback, should also be strengthened to improve operational reliability. The mobile application could subsequently be extended to iOS while improving offline support, localization, accessibility, and usability across devices. Finally, preparation for production deployment should include expanded continuous integration, security assessments, load testing, centralized monitoring, backup and recovery procedures, and failover strategies for external service providers.

REFERENCES

- [1] R. Burke, “Hybrid Recommender Systems: Survey and Experiments,” *User Model User-Adap Inter*, vol. 12, no. 4, pp. 331–370, Nov. 2002, doi: 10.1023/A:1021240730564.
- [2] X. Han, Z. Wu, Z. Wu, R. Yu, and L. S. Davis, “VITON: An Image-Based Virtual Try-On Network,” presented at the Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition, 2018, pp. 7543–7552. Accessed: Jul. 17, 2026. [Online]. Available: https://openaccess.thecvf.com/content_cvpr_2018/html/Han_VITON_An_Image-Based_CVPR_2018_paper.html
- [3] “ASOS | Online Shopping for the Latest Clothes & Fashion,” ASOS. Accessed: Aug. 11, 2026. [Online]. Available: <https://www.asos.com/>
- [4] “DRESSX.” Accessed: Aug. 11, 2026. [Online]. Available: <https://app.dressx.com/>
- [5] “YODY - Look Good - Feel Good,” <https://yody.vn>. Accessed: Jul. 17, 2026. [Online]. Available: <https://yody.vn/>
- [6] “CANIFA | Fashion For All.” Accessed: Aug. 11, 2026. [Online]. Available: <https://canifa.com>
- [7] “WTO | Trade Topics - Electronic commerce - Work programme on electronic commerce.” Accessed: Jul. 17, 2026. [Online]. Available: https://www.wto.org/english/tratop_e/ecom_e/wkprog_e.htm
- [8] “Introduction · React Native.” Accessed: Jul. 17, 2026. [Online]. Available: <https://reactnative.dev/docs/getting-started>
- [9] “Develop an app with Expo,” Expo Documentation. Accessed: Jul. 17, 2026. [Online]. Available: <https://docs.expo.dev/workflow/overview/>
- [10] “Data Modeling in MongoDB - Database Manual - MongoDB Docs.” Accessed: Jul. 17, 2026. [Online]. Available: <https://www.mongodb.com/docs/manual/data-modeling/>
- [11] “Introduction | Socket.IO.” Accessed: Jul. 17, 2026. [Online]. Available: <https://socket.io/docs/v4/>
- [12] “Redis Cloud.” Accessed: Aug. 13, 2026. [Online]. Available: <https://redis.io/cloud/>
- [13] “Docker Docs,” Docker Documentation. Accessed: Aug. 13, 2026. [Online]. Available: <https://docs.docker.com/>
- [14] “Context-Aware Recommender Systems”, doi: 10.1609/aimag.v32i3.2364.
- [15] M. J. Pazzani and D. Billsus, “Content-Based Recommendation Systems,” in *The Adaptive Web: Methods and Strategies of Web Personalization*, P. Brusilovsky, A. Kobsa, and W. Nejdl, Eds., Berlin, Heidelberg: Springer, 2007, pp. 325–341. doi: 10.1007/978-3-540-72079-9_10.
- [16] Y.-L. Lin, S. Tran, and L. S. Davis, “Fashion Outfit Complementary Item Retrieval,” presented at the Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition, 2020, pp. 3311–3319. Accessed: Jul. 17, 2026. [Online]. Available:

- https://openaccess.thecvf.com/content_CVPR_2020/html/Lin_Fashion_Outfit_Complementary_Item_Retrieval_CVPR_2020_paper.html
- [17] Y. Hu, Y. Koren, and C. Volinsky, “Collaborative Filtering for Implicit Feedback Datasets,” in *2008 Eighth IEEE International Conference on Data Mining*, Dec. 2008, pp. 263–272. doi: 10.1109/ICDM.2008.22.
- [18] “Time weight collaborative filtering | Proceedings of the 14th ACM international conference on Information and knowledge management.” Accessed: Aug. 12, 2026. [Online]. Available: <https://dl.acm.org/doi/10.1145/1099554.1099689>
- [19] X. Han, Z. Wu, Y.-G. Jiang, and L. S. Davis, “Learning Fashion Compatibility with Bidirectional LSTMs,” in *Proceedings of the 25th ACM international conference on Multimedia*, in MM ’17. New York, NY, USA: Association for Computing Machinery, Oct. 2017, pp. 1078–1086. doi: 10.1145/3123266.3123394.
- [20] “ISO/IEC/IEEE 29119-2:2021,” ISO. Accessed: Jul. 17, 2026. [Online]. Available: <https://www.iso.org/standard/79428.html>
- [21] “Pose Estimation with YOLO,” Ultralytics Docs. Accessed: Jul. 17, 2026. [Online]. Available: <https://docs.ultralytics.com/tasks/pose>
- [22] “Amazon Reviews’23.” Accessed: Sep. 02, 2026. [Online]. Available: <https://amazon-reviews-2023.github.io/>
- [23] X. Han, *xthan/polyvore-dataset*. (Sep. 02, 2026). Accessed: Sep. 02, 2026. [Online]. Available: <https://github.com/xthan/polyvore-dataset>

APPENDIX A. REPRESENTATIVE TEST CASES

Table A.1 maps each representative test case directly to one of the 21 consolidated use cases. UC-MOB-01 to UC-MOB-15 cover the customer mobile application, including UC-MOB-15 Manage Personal Information; UC-ADM-01 to UC-ADM-06 cover the Web Administration Portal.

Table A.1 Representative Test Cases

Test Case ID	Use Case ID	Use Case	Test Data / Action	Expected Result	Status
TC-MOB-01-01	UC-MOB-01	Sign In	Submit valid credentials, then repeat invalid passwords until the lock threshold is reached.	Valid credentials create the authenticated session; repeated invalid attempts trigger the configured temporary lock.	Pass
TC-MOB-02-01	UC-MOB-02	Sign Up	Submit unique registration data with a verified phone OTP and current policy acceptance.	The system creates the customer account and default address; it rejects duplicate data or stale verification.	Pass
TC-MOB-03-01	UC-MOB-03	View Product List	Load active catalog pages and request the next page.	Products are returned in the expected order without duplicates or unnecessary extra-page requests.	Pass
TC-MOB-04-01	UC-MOB-04	Search Product	Search with Vietnamese and material keywords; select a suggestion and manage search history.	Suggestions and active results are standardized and constrained; saved history stays synchronized and free of duplicates.	Pass
TC-MOB-05-01	UC-MOB-05	Filter Products	Combine category, brand, color, size, price, and sorting filters, then reset them.	Only matching active products are returned; reset restores the default product listing.	Pass

Test Case ID	Use Case ID	Use Case	Test Data / Action	Expected Result	Status
TC-MOB-06-01	UC-MOB-06	View Product Detail	Open a product and choose available and unavailable color-size combinations.	Product data, price, media, and stock follow the valid variant; unavailable combinations cannot be selected.	Pass
TC-MOB-07-01	UC-MOB-07	Add Favorite	Add the same eligible product twice from the product-detail interface.	The product is added once, duplicate insertion is prevented, and the customer favorite state stays synchronized.	Pass
TC-MOB-08-01	UC-MOB-08	Manage Shopping Cart	Add a variant, change its quantity beyond available stock, select checkout items, and remove a line.	Valid cart changes persist; excessive quantity is rejected, and cart, inventory, and checkout selection remain consistent.	Pass
TC-MOB-09-01	UC-MOB-09	Place Order and Make Payment	Confirm an authoritative quote using COD or VNPay, then submit an outdated quote version.	A current quote commits order and inventory atomically and clears purchased lines; a stale quote creates no partial write.	Pass
TC-MOB-10-01	UC-MOB-10	View Order History	Filter the customer's orders, open an order, and receive a real-time status update.	The correct list tab, order detail, payment state, and timeline refresh without duplicating records.	Pass
TC-MOB-11-01	UC-MOB-11	Review and Rate Products	Create, update, and remove a review for an eligible completed and paid order item.	Only a verified purchaser can change the review; visible rating aggregates follow moderation and deletion rules.	Pass

Test Case ID	Use Case ID	Use Case	Test Data / Action	Expected Result	Status
TC-MOB-12-01	UC-MOB-12	Contact Support	Open public support content and create a ticket with a real-time staff reply.	Supported content loads; ticket ownership and lifecycle are enforced, and each reply appears once with polling fallback.	Pass
TC-MOB-13-01	UC-MOB-13	View Recommendations	Load recommendations on Home, Product Detail, and Cart; submit one valid and one invalid attribution event.	Each context returns eligible ranked products; only events tied to an issued recommendation are recorded.	Pass
TC-MOB-14-01	UC-MOB-14	Use Virtual Try-On	Submit invalid inputs, then run an eligible image job through retry and optional video states.	Validation blocks unsafe work; valid asynchronous transitions, history, cancellation, retry, and result media follow the job contract.	Pass
TC-MOB-15-01	UC-MOB-15	Manage Personal Information	Update the customer's profile, avatar, addresses, default address, and password with valid and invalid data.	Customer validation, uniqueness, avatar size, address limit, default address, and session-revocation rules are enforced.	Pass
TC-ADM-01-01	UC-ADM-01	Manage Customer Accounts	Read a customer with customers. Read, then attempt note, status, and reset actions without customers. manage.	Permitted customer information is returned; protected mutations are denied without leaving a partial change.	Pass

Test Case ID	Use Case ID	Use Case	Test Data / Action	Expected Result	Status
TC-ADM-02-01	UC-ADM-02	Manage Roles and Permissions	Create a Staff account and change permissions, status, and password; also target an Admin account.	Supported Staff operations succeed; invalid permissions and protected Admin-target operations are rejected.	Pass
TC-ADM-03-01	UC-ADM-03	Manage Orders	Search and inspect orders; process valid and invalid transitions; and perform shipping, payment, return, invoice, export, and bulk actions.	Permission, lifecycle, payment, shipping, return, inventory, and bulk-result rules are applied consistently.	Pass
TC-ADM-04-01	UC-ADM-04	Manage Customer Support	Triage a ticket, assign it, add an internal note, reply to the customer, and update FAQ content under split permissions.	Ticket and knowledge-base actions respect reply/manage permissions; public replies, notes, assignment, and state changes remain traceable.	Pass
TC-ADM-05-01	UC-ADM-05	Search and Recommendation Analytics	Request an inclusive 30-day report with valid, malformed, empty, and weak-sample data.	Authorized search and recommendation metrics render; invalid input is rejected, empty data is valid, and weak insights are withheld.	Pass
TC-ADM-06-01	UC-ADM-06	Manage Virtual Try-On	Inspect try-on jobs and outputs, then exercise eligible retry, cancel, hide, prompt-policy, account-lock, and versioned-setting operations under different permission scopes.	Read, manage, and settings permissions are enforced; valid state transitions, moderation actions, audit reasons, customer locks, and version checks remain traceable.	Pass

APPENDIX B. DETAILED TEST EXECUTION RESULTS

The following results record the automated execution performed on 14 August 2026. Backend unit/integration, backend end-to-end, mobile, and Web Admin unit groups all completed successfully. The backend end-to-end group executed six cases and all six passed.

Table B.1 Automated Test Execution Results

Test Group	Suites / Runs	Cases	Passed	Failed	Result
Backend Jest unit/integration	95	938	938	0	Passed
Mobile Jest	34	179	179	0	Passed
Web Playwright unit	1 run	37	37	0	Passed
Backend Jest end-to-end	3	6	6	0	Passed
Total executed	133	1,160	1,160	0	Passed

In total, 1,160 of 1,160 automated test cases passed (100 percent). This result confirms the deterministic application behavior covered by the executed backend, mobile, Web Admin, and isolated end-to-end suites. It does not certify live-service availability, device usability, accessibility, security, production-scale load, or perceptual virtual try-on quality; these remain subjects for pre-production validation.

APPENDIX C. DATABASE COLLECTION DATA DICTIONARY

Table C.1 Data dictionary for the users collection

No.	Attribute	Data Type	Primary Key	Foreign Key	Embedded	Description
1	_id	ObjectId	Yes	No	No	Unique document identifier.
2	name	String	No	No	No	Display name.
3	email	String	No	No	No	Account email address.
4	password	String	No	No	No	Hashed account password.
5	role	String	No	No	No	Account role.
6	phone	String	No	No	No	Contact phone number.
7	gender	String	No	No	No	Stores gender.
8	dateOfBirth	Date	No	No	No	Stores date of birth.
9	address	Array<Object>	No	No	Yes	Embedded address.
10	membership	ObjectId	No	Yes (membership rankings)	No	References membership rankings collection.
11	loyaltyPoint	Number	No	No	No	Stores loyalty point.
12	membershipUpdatedAt	Date	No	No	No	Membership updated timestamp.
13	refreshToken	String	No	No	No	Stores refresh token.
14	permissions	Array<String>	No	No	No	Granted permissions.
15	mustChangePassword	Boolean	No	No	No	Stores must change password.
16	createdBy	ObjectId	No	Yes (users)	No	References users collection.
17	passwordChangedAt	Date	No	No	No	Password changed timestamp.
18	lastLoginAt	Date	No	No	No	Last login timestamp.
19	profileCompleted	Boolean	No	No	No	Stores profile completed.
20	resetPasswordToken	String	No	No	No	Stores reset password token.
21	resetPasswordExpires	Date	No	No	No	Stores reset password expires.
22	loginUnlockOtpHash	String	No	No	No	Stores login unlock otp hash.
23	loginUnlockOtpExpiresAt	Date	No	No	No	Login unlock otp expires timestamp.
24	loginUnlockOtpAttempts	Number	No	No	No	Stores login unlock otp attempts.
25	loginUnlockOtpLockedUntil	Date	No	No	No	Stores login unlock otp locked until.
26	loginUnlockOtpSentAt	Date	No	No	No	Login unlock otp sent timestamp.
27	avatarImage	String	No	No	No	Stores avatar image.
28	avatarPublicId	String	No	No	No	Stores avatar public ID.
29	isActive	Boolean	No	No	No	Active-state flag.
30	legalConsent	Object	No	No	Yes	Embedded legal consent.
31	createdAt	Date	No	No	No	Creation timestamp.
32	updatedAt	Date	No	No	No	Last update timestamp.

Table C.2 Data dictionary for the categories collection

No.	Attribute	Data Type	Primary Key	Foreign Key	Embedded	Description
1	_id	ObjectId	Yes	No	No	Unique document identifier.

No.	Attribute	Data Type	Primary Key	Foreign Key	Embedded	Description
2	name	String	No	No	No	Display name.
3	parent_id	ObjectId	No	Yes (categories)	No	References categories collection.
4	level	Number	No	No	No	Stores level.
5	gender	String	No	No	No	Stores gender.
6	image	String	No	No	No	Stores image.
7	description	String	No	No	No	Detailed description.
8	isLeaf	Boolean	No	No	No	Leaf flag.
9	isSizeTemplateSource	Boolean	No	No	No	Size template source flag.
10	sizeTemplateName	String	No	No	No	Stores size template name.
11	sizeTemplateSourceId	ObjectId	No	Yes (categories)	No	References categories collection.
12	sizes	Array<String>	No	No	No	List of sizes.
13	measurementFields	Array<Object>	No	No	Yes	Embedded measurement fields.
14	fitTypes	Array<Object>	No	No	Yes	Embedded fit types.
15	isActive	Boolean	No	No	No	Active-state flag.
16	createdAt	Date	No	No	No	Creation timestamp.
17	updatedAt	Date	No	No	No	Last update timestamp.

Table C.3 Data dictionary for the brands collection

No.	Attribute	Data Type	Primary Key	Foreign Key	Embedded	Description
1	_id	ObjectId	Yes	No	No	Unique document identifier.
2	name	String	No	No	No	Display name.
3	image	String	No	No	No	Stores image.
4	isActive	Boolean	No	No	No	Active-state flag.
5	createdAt	Date	No	No	No	Creation timestamp.
6	updatedAt	Date	No	No	No	Last update timestamp.

Table C.4 Data dictionary for the products collection

No.	Attribute	Data Type	Primary Key	Foreign Key	Embedded	Description
1	_id	ObjectId	Yes	No	No	Unique document identifier.
2	category_id	ObjectId	No	Yes (categories)	No	References categories collection.
3	name	String	No	No	No	Display name.
4	brand_id	ObjectId	No	Yes (brands)	No	References brands collection.
5	variant	Array<Object>	No	No	Yes	Embedded variant.
6	description	String	No	No	No	Detailed description.
7	material	String	No	No	No	Stores material.
8	materialNormalized	String	No	No	No	Stores material normalized.
9	product_image	String	No	No	No	Stores product image.
10	isActive	Boolean	No	No	No	Active-state flag.

No.	Attribute	Data Type	Primary Key	Foreign Key	Embedded	Description
11	sold_quantity	Number	No	No	No	Stores sold quantity.
12	averageRating	Number	No	No	No	Stores average rating.
13	reviewCount	Number	No	No	No	Stores review count.
14	createdAt	Date	No	No	No	Creation timestamp.
15	updatedAt	Date	No	No	No	Last update timestamp.

Table C.5 Data dictionary for the inventories collection

No.	Attribute	Data Type	Primary Key	Foreign Key	Embedded	Description
1	_id	ObjectId	Yes	No	No	Unique document identifier.
2	productId	ObjectId	No	Yes (products)	No	References products collection.
3	variantId	ObjectId	No	No	No	Stores variant ID.
4	colorVariantId	ObjectId	No	No	No	Stores color variant ID.
5	size	String	No	No	No	Stores size.
6	sku	String	No	No	No	Stores sku.
7	quantity	Number	No	No	No	Item or stock quantity.
8	reservedQuantity	Number	No	No	No	Stores reserved quantity.
9	availableQuantity	Number	No	No	No	Stores available quantity.
10	createdAt	Date	No	No	No	Creation timestamp.
11	updatedAt	Date	No	No	No	Last update timestamp.

Table C.6 Data dictionary for the inventoryreservations collection

No.	Attribute	Data Type	Primary Key	Foreign Key	Embedded	Description
1	_id	ObjectId	Yes	No	No	Unique document identifier.
2	userId	ObjectId	No	Yes (users)	No	References users collection.
3	orderId	ObjectId	No	Yes (orders)	No	References orders collection.
4	productId	ObjectId	No	Yes (products)	No	References products collection.
5	variantId	ObjectId	No	No	No	Stores variant ID.
6	colorVariantId	ObjectId	No	No	No	Stores color variant ID.
7	size	String	No	No	No	Stores size.
8	sku	String	No	No	No	Stores sku.
9	quantity	Number	No	No	No	Item or stock quantity.
10	status	String	No	No	No	Current lifecycle status.
11	expiresAt	Date	No	No	No	Expires timestamp.
12	createdAt	Date	No	No	No	Creation timestamp.
13	updatedAt	Date	No	No	No	Last update timestamp.

Table C.7 Data dictionary for the imports collection

No.	Attribute	Data Type	Primary Key	Foreign Key	Embedded	Description
1	_id	ObjectId	Yes	No	No	Unique document identifier.
2	importCode	String	No	No	No	Stores import code.

No.	Attribute	Data Type	Primary Key	Foreign Key	Embedded	Description
3	receiptId	ObjectId	No	Yes (inventoryreceipts)	No	References inventoryreceipts collection.
4	receiptCode	String	No	No	No	Stores receipt code.
5	supplierName	String	No	No	No	Stores supplier name.
6	productId	ObjectId	No	Yes (products)	No	References products collection.
7	variantId	ObjectId	No	No	No	Stores variant ID.
8	colorVariantId	ObjectId	No	No	No	Stores color variant ID.
9	detail	Array<Object>	No	No	Yes	Embedded detail.
10	totalAmount	Number	No	No	No	Stores total amount.
11	createdAt	Date	No	No	No	Creation timestamp.
12	updatedAt	Date	No	No	No	Last update timestamp.

Table C.8 Data dictionary for the inventoryreceipts collection

No.	Attribute	Data Type	Primary Key	Foreign Key	Embedded	Description
1	_id	ObjectId	Yes	No	No	Unique document identifier.
2	receiptCode	String	No	No	No	Stores receipt code.
3	supplierName	String	No	No	No	Stores supplier name.
4	importDate	Date	No	No	No	Stores import date.
5	createdBy	ObjectId	No	Yes (users)	No	References users collection.
6	status	String	No	No	No	Current lifecycle status.
7	note	String	No	No	No	Stores note.
8	lines	Array<Object>	No	No	Yes	Embedded lines.
9	totalQuantity	Number	No	No	No	Stores total quantity.
10	totalAmount	Number	No	No	No	Stores total amount.
11	confirmedAt	Date	No	No	No	Confirmed timestamp.
12	cancelledAt	Date	No	No	No	Cancelled timestamp.
13	createdAt	Date	No	No	No	Creation timestamp.
14	updatedAt	Date	No	No	No	Last update timestamp.

Table C.9 Data dictionary for the carts collection

No.	Attribute	Data Type	Primary Key	Foreign Key	Embedded	Description
1	_id	ObjectId	Yes	No	No	Unique document identifier.
2	user_id	ObjectId	No	Yes (users)	No	References users collection.
3	product_list	Array<Object>	No	No	Yes	Embedded product list.
4	createdAt	Date	No	No	No	Creation timestamp.
5	updatedAt	Date	No	No	No	Last update timestamp.

Table C.10 Data dictionary for the orders collection

No.	Attribute	Data Type	Primary Key	Foreign Key	Embedded	Description
1	_id	ObjectId	Yes	No	No	Unique document identifier.
2	idempotencyKey	String	No	No	No	Stores idempotency key.
3	orderCode	String	No	No	No	Stores order code.
4	invoiceCode	String	No	No	No	Stores invoice code.
5	invoiceIssuedAt	Date	No	No	No	Invoice issued timestamp.
6	user_id	ObjectId	No	Yes (users)	No	References users collection.
7	order_list	Array<Object>	No	No	Yes	Embedded order list.
8	subTotal	Number	No	No	No	Stores sub total.
9	shippingFee	Number	No	No	No	Stores shipping fee.
10	couponCode	String	No	No	No	Stores coupon code.
11	couponId	ObjectId	No	Yes (coupons)	No	References coupons collection.
12	couponCodes	Array<String>	No	No	No	List of coupon codes.
13	couponIds	Array<ObjectId>	No	Yes (coupons)	No	References coupons collection.
14	promotionCampaignId	ObjectId	No	Yes (promotion_campaigns)	No	References promotion_campaigns collection.
15	couponDiscountAmount	Number	No	No	No	Stores coupon discount amount.
16	shippingDiscountAmount	Number	No	No	No	Stores shipping discount amount.
17	appliedMembershipTierId	ObjectId	No	Yes (membership_rankings)	No	References membership_rankings collection.
18	appliedMembershipDiscountPercent	Number	No	No	No	Stores applied membership discount percent.
19	membershipDiscountAmount	Number	No	No	No	Stores membership discount amount.
20	loyaltyPointsAwarded	Number	No	No	No	Stores loyalty points awarded.
21	loyaltyPointsClawedBack	Number	No	No	No	Stores loyalty points clawed back.
22	loyaltyRuleSnapshot	Object	No	No	Yes	Embedded loyalty rule snapshot.
23	taxAmount	Number	No	No	No	Stores tax amount.
24	totalAmount	Number	No	No	No	Stores total amount.
25	status	String	No	No	No	Current lifecycle status.
26	paymentMethod	String	No	No	No	Stores payment method.
27	paymentMethodId	ObjectId	No	Yes (payment_methods)	No	References payment_methods collection.
28	paymentStatus	String	No	No	No	Stores payment status.
29	paymentDeadlineAt	Date	No	No	No	Payment deadline timestamp.
30	paymentDeadlineWarningSentAt	Date	No	No	No	Payment deadline warning sent timestamp.
31	deliveredAt	Date	No	No	No	Delivered timestamp.
32	receivedAt	Date	No	No	No	Received timestamp.
33	returnRequest	Object	No	No	Yes	Embedded return request.
34	cancellation	Object	No	No	Yes	Embedded cancellation.

No.	Attribute	Data Type	Primary Key	Foreign Key	Embedded	Description
35	shipping	Object	No	No	Yes	Embedded shipping.
36	shippingAddress	Object	No	No	Yes	Embedded shipping address.
37	orderNote	String	No	No	No	Stores order note.
38	createdAt	Date	No	No	No	Creation timestamp.
39	updatedAt	Date	No	No	No	Last update timestamp.

Table C.11 Data dictionary for the transactions collection

No.	Attribute	Data Type	Primary Key	Foreign Key	Embedded	Description
1	_id	ObjectId	Yes	No	No	Unique document identifier.
2	user_id	ObjectId	No	Yes (users)	No	References users collection.
3	order_id	ObjectId	No	Yes (orders)	No	References orders collection.
4	amount	Number	No	No	No	Monetary amount.
5	paymentMethod	String	No	No	No	Stores payment method.
6	paymentMethodId	ObjectId	No	Yes (paymentmethods)	No	References paymentmethods collection.
7	txnRef	String	No	No	No	Stores txn ref.
8	attemptNo	Number	No	No	No	Stores attempt no.
9	expiredAt	Date	No	No	No	Expired timestamp.
10	resolvedAt	Date	No	No	No	Resolved timestamp.
11	failureReason	String	No	No	No	Stores failure reason.
12	createdBy	String	No	No	No	Stores created by.
13	gatewayTransactionId	String	No	No	No	Stores gateway transaction ID.
14	gatewayProvider	String	No	No	No	Stores gateway provider.
15	paymentDetail	Mixed	No	No	No	Stores payment detail.
16	status	String	No	No	No	Current lifecycle status.
17	createdAt	Date	No	No	No	Creation timestamp.
18	updatedAt	Date	No	No	No	Last update timestamp.

Table C.12 Data dictionary for the reviews collection

No.	Attribute	Data Type	Primary Key	Foreign Key	Embedded	Description
1	_id	ObjectId	Yes	No	No	Unique document identifier.
2	user_id	ObjectId	No	Yes (users)	No	References users collection.
3	product_id	ObjectId	No	Yes (products)	No	References products collection.
4	order_id	ObjectId	No	Yes (orders)	No	References orders collection.
5	order_item_id	ObjectId	No	No	No	Stores order item ID.
6	rating	Number	No	No	No	Stores rating.
7	comment	String	No	No	No	Stores comment.
8	criteria	Object	No	No	Yes	Embedded criteria.
9	images	Array<Object>	No	No	Yes	Embedded images.

No.	Attribute	Data Type	Primary Key	Foreign Key	Embedded	Description
10	moderationStatus	String	No	No	No	Stores moderation status.
11	moderationReasons	Array<String>	No	No	No	List of moderation reasons.
12	moderationHistory	Array<Object>	No	No	Yes	Embedded moderation history.
13	helpfulCount	Number	No	No	No	Stores helpful count.
14	adminReply	String	No	No	No	Stores admin reply.
15	repliedAt	Date	No	No	No	Replied timestamp.
16	repliedBy	ObjectId	No	Yes (users)	No	References users collection.
17	createdAt	Date	No	No	No	Creation timestamp.
18	updatedAt	Date	No	No	No	Last update timestamp.

Table C.13 Data dictionary for the virtualtryonjobs collection

No.	Attribute	Data Type	Primary Key	Foreign Key	Embedded	Description
1	_id	ObjectId	Yes	No	No	Unique document identifier.
2	userId	ObjectId	No	Yes (users)	No	References users collection.
3	sourceAssetId	ObjectId	No	Yes (virtualtryonassets)	No	References virtualtryonassets collection.
4	sourceImageUrlSnapshot	String	No	No	No	Stores source image url snapshot.
5	selectedItems	Array<Object>	No	No	Yes	Embedded selected items.
6	outfitMode	String	No	No	No	Stores outfit mode.
7	contextPreset	String	No	No	No	Stores context preset.
8	contextPrompt	String	No	No	No	Stores context prompt.
9	outputMode	String	No	No	No	Stores output mode.
10	videoDurationSeconds	Number	No	No	No	Stores video duration seconds.
11	status	String	No	No	No	Current lifecycle status.
12	progress	Number	No	No	No	Processing progress.
13	processingStage	String	No	No	No	Stores processing stage.
14	generatedImageAssetId	ObjectId	No	Yes (virtualtryonassets)	No	References virtualtryonassets collection.
15	generatedImageUrl	String	No	No	No	Stores generated image url.
16	generatedImageAssetIds	Array<ObjectId>	No	Yes (virtualtryonassets)	No	References virtualtryonassets collection.
17	generatedImageUrls	Array<String>	No	No	No	List of generated image urls.
18	generatedVideoAssetId	ObjectId	No	Yes (virtualtryonassets)	No	References virtualtryonassets collection.
19	generatedVideoUrl	String	No	No	No	Stores generated video url.
20	videoStatus	String	No	No	No	Stores video status.
21	videoProgress	Number	No	No	No	Stores video progress.
22	videoSourceImageAssetId	ObjectId	No	Yes (virtualtryonassets)	No	References virtualtryonassets collection.

No.	Attribute	Data Type	Primary Key	Foreign Key	Embedded	Description
23	videoSourceImageUrlSnapshot	String	No	No	No	Stores video source image URL snapshot.
24	videoProvider	String	No	No	No	Stores video provider.
25	videoProviderJobId	String	No	No	No	Stores video provider job ID.
26	videoProviderMetadata	Mixed	No	No	No	Stores video provider metadata.
27	videoErrorCode	String	No	No	No	Stores video error code.
28	videoErrorMessage	String	No	No	No	Stores video error message.
29	videoStartedAt	Date	No	No	No	Video started timestamp.
30	videoCompletedAt	Date	No	No	No	Video completed timestamp.
31	provider	String	No	No	No	Stores provider.
32	providerJobId	String	No	No	No	Stores provider job ID.
33	providerMetadata	Mixed	No	No	No	Stores provider metadata.
34	errorCode	String	No	No	No	Processing error code.
35	errorMessage	String	No	No	No	Processing error message.
36	idempotencyKey	String	No	No	No	Stores idempotency key.
37	startedAt	Date	No	No	No	Started timestamp.
38	completedAt	Date	No	No	No	Completed timestamp.
39	deletedAt	Date	No	No	No	Soft-deletion timestamp.
40	createdAt	Date	No	No	No	Creation timestamp.
41	updatedAt	Date	No	No	No	Last update timestamp.

In addition to the core collections shown in the relationship diagram, the system uses several supporting collections for specialized operations. These collections manage authentication attempts and OTP verification, saved payment methods, push notifications, favorites, promotions, loyalty points, customer support, search history, recommendation events, and virtual try-on configuration. Other supporting collections store storefront settings, administrative audit logs, shipping mappings, visual-search indexes, distributed locks, and rate-limit data. They are omitted from the simplified diagram to maintain readability because they mainly support operational, analytical, security, and integration tasks rather than represent the system's principal business relationships.

APPENDIX D. VIRTUAL TRY-ON PROMPT CONFIGURATION

Section 3.7.2 explains how the prompt is constructed and why each block exists; this appendix records the exact implementation-level text for reproducibility. Values inside braces are runtime substitutions. The application prompt is joined with ' '; the negative prompt is joined with ', ' and the ComfyUI wrapper is joined with blank lines. Because the current workflow map exposes prompt but not negativePrompt, the backend appends the strict avoid block to the effective positive prompt.

D.1 Assembly Order and Placeholders

```
APPLICATION_PROMPT = JOIN(' ', [
    SOURCE_FRAMING_BLOCK(sourceImageProfile),
    'selected garments: ' + GARMENT_DESCRIPTIONS,
    OUTFIT_MODE_BLOCK(outfitMode, selectedRoles),
    ROLE_ALIGNMENT_BLOCKS(selectedRoles),
    CONTEXT_BLOCK(contextPreset, contextPrompt),
    GARMENT_FIDELITY_BLOCK,
    REALISM_BLOCK
])

NEGATIVE_PROMPT = JOIN(', ', [BASE_NEGATIVE, ROLE_NEGATIVES(selectedRoles)])

EFFECTIVE_PROMPT = COMFY_2X2_WRAPPER + APPLICATION_PROMPT
IF workflow has no negativePrompt input:
    EFFECTIVE_PROMPT += '\n\nSTRICT AVOID LIST – none of the following outcomes may appear in
the result:'
    EFFECTIVE_PROMPT += '\n\n' + NEGATIVE_PROMPT

{ROLE}: top, bottom, dress, shoes, outerwear, or accessory.
{ITEM_NAME}, {COLOR}, and {SIZE}: snapshots resolved from the selected
catalog product/variant.
{VISIBLE_REGION_CONDITION}: one condition derived from YOLO11 Pose
output.
{CONTEXT}: one fixed preset scene or the normalized user custom scene.
{DURATION_SECONDS}: integer from 5 to 12 for optional video.
```

D.2 Exact ComfyUI 2x2 Image Wrapper

Create one single 2x2 grid image for virtual fashion try-on.

The full returned image must be a vertical 3:4 portrait canvas, so each cropped grid cell is also a vertical 3:4 portrait.

The grid order is fixed: cell 1 is top-left, cell 2 is top-right, cell 3 is bottom-left, and cell 4 is bottom-right.

Cell 1 is the baseline try-on: apply the selected outfit while preserving the source person pose, expression, camera angle, perspective, crop, and body alignment as closely as possible.

Cells 2, 3, and 4 may each use a different subtle, natural fashion pose, but must keep the same person identity, body shape, body proportions, skin tone, visible body coverage, camera framing, and complete selected outfit as cell 1.

Do not turn pose variation into a different person, a body transformation, a wardrobe variation, or a different crop.

{SOURCE_GRID_FRAMING_INSTRUCTION}

Use reference image 1 as the only source for the visible person identity cues, face if visible, hair if visible, expression if visible, pose, body shape, body proportions, height cues, shoulder width, waist if visible, legs if visible, hands if visible, feet if visible, and skin tone.

Never copy or blend in the face, body, pose, age, gender presentation, skin tone, background, or other garments from catalog garment reference images.

GARMENT SOURCE OF TRUTH: every selected garment or accessory must come from the garment reference images after reference image 1. Those garment images, not the text prompt, are the mandatory and exclusive wardrobe source.

Reproduce the selected garments faithfully: keep the same garment type, color, pattern, print, logo already present, fabric appearance, silhouette, neckline, sleeves, closures, seams, pockets, waistband, hem, hardware, footwear shape, and other visible construction details.

Never invent, redesign, replace, recolor, omit, combine into a different item, or add any fashion item that is not present in the selected garment references.

Treat all custom or user-written prompt text as scene context only: it may affect background, lighting, mood, occasion, and explicitly permitted pose variation, but it must not change the wardrobe. If text conflicts with a garment reference, ignore the conflicting wardrobe instruction and follow the garment reference.

Do not add visible borders, gutters, labels, captions, watermarks, or extra text between grid cells.

Reference image 1 is the person photo and controls the person appearance.

Reference image 2: top – {TOP_NAME}, {TOP_COLOR}.

Reference image 3: bottom – {BOTTOM_NAME}, {BOTTOM_COLOR}.

{APPLICATION_PROMPT}

STRICT AVOID LIST – none of the following outcomes may appear in the result:

{NEGATIVE_PROMPT}

D.3 Exact Source and Framing Block

virtual fashion try-on for the person or visible body part in the source image. use the source person image as the only reference for visible identity cues, hair, expression, pose, body shape, body proportions, and skin tone. preserve the original camera framing, crop, perspective, and visible body coverage from the source image. do not expand a cropped source image into a full-body image, and do not invent unseen face, torso, legs, feet, or hands. do not copy or infer any face, body shape, pose, age, gender presentation, or skin tone from catalog garment images or garment models. preserve visible hands, fingers, neck, legs, feet, and body boundaries unless covered by selected garments. replace or overlay only the selected fashion items realistically within the visible crop. {VISIBLE_REGION_CONDITION}

Conditional final sentence

unknown: if the person crop is ambiguous, keep the original crop and only edit clearly visible clothing areas

full_body: source image supports full-body try-on; keep the full body visible with the same framing and ground contact

feet_crop: source image is a feet or footwear crop; focus on feet, ankles, footwear, floor contact, and shadows without inventing upper body or face

lower_body: source image is a lower-body crop; focus on visible waist, hips, legs, feet, and garment fit without inventing face or upper torso

upper_body: source image is an upper-body crop; focus on visible head, neck, shoulders, torso, arms, and upper garments without inventing legs or feet

partial: source image shows only part of the body; edit only the visible body regions and avoid hallucinating missing regions

Fallback when no sourceImageProfile is available

virtual fashion try-on for the person in the source image. the source person image is the only reference for face, identity, hair, expression, pose, body shape, body proportions, height, shoulder width, waist, legs, and skin tone. do not copy or infer any face, body shape, pose, age, gender presentation, or skin tone from catalog garment images or garment models. preserve hands, fingers, neck, legs, and visible body boundaries unless covered by selected garments. replace or overlay only the selected fashion items realistically

Provider-level grid framing substitution

full_body: Each of the four cells should show the same full-body framing as the source person wearing the selected outfit, with slight styling variation and grounded feet.

feet_crop: Each of the four cells should preserve the source feet/footwear crop, showing only the visible feet or ankles region with selected footwear applied; do not invent torso, face, or full-body framing.

lower_body: Each of the four cells should preserve the source lower-body crop, showing only the visible waist, hips, legs, or feet region wearing the selected items; do not invent face or upper-body framing.

upper_body: Each of the four cells should preserve the source upper-body crop, showing only the visible head, shoulders, torso, or arms region wearing the selected items; do not invent legs, feet, or full-body framing.

partial: Each of the four cells should preserve the same partial-body crop and visible body regions as the source image; do not invent missing body parts or expand to full-body framing.

unknown: Each of the four cells should preserve the same camera framing and visible body coverage as the source image; if the source is cropped, do not expand it to full-body framing.

D.4 Exact Garment Description and Role Blocks

selected garments: upper-body garment such as shirt, blouse, polo, sweater, or t-shirt, {ITEM_NAME}, color {COLOR}, size {SIZE}, align neckline, shoulders, sleeves, chest fit, armhole

es, side seams, tucked or untucked hem, and visible print placement; lower-body garment such as pants, jeans, skirt, or shorts, {ITEM_NAME}, color {COLOR}, size {SIZE}, align waistband, belt loops, hips, rise, crotch area, leg shape, inseam, cuffs, and hem length; one-piece dress or long one-piece garment, {ITEM_NAME}, color {COLOR}, size {SIZE}, align neckline, shoulders, waistline, skirt fall, hem length, side seams, and continuous one-piece silhouette; footwear, matching pair of shoes or sandals, {ITEM_NAME}, color {COLOR}, size {SIZE}, place the complete matching pair on both feet with correct left-right pairing, sole contact, shadows, scale, and perspective; outerwear layer such as jacket, blazer, coat, or cardigan, {ITEM_NAME}, color {COLOR}, size {SIZE}, layer over the inner outfit with correct lapels, collar, shoulder line, sleeve length, cuffs, closure, opening, and drape; fashion accessory such as bag, hat, scarf, belt, glasses, or watch, {ITEM_NAME}, color {COLOR}, size {SIZE}, place the accessory with correct scale, orientation, hand or body contact, strap path, occlusion, and natural shadow

Single-item mode

Single Top

single top try-on: replace only the upper-body garment. preserve the original pants or skirt, shoes, accessories, and hand position unless they are naturally occluded by the new top. make the top follow the torso angle, shoulder slope, arm bend, and chest depth

Single Bottom

single bottom try-on: replace only the lower-body garment. preserve the original top, outerwear, shoes, accessories, and hands unless natural overlap is required at the waist. make the waistband sit on the hips or waist with correct rise, leg opening, and body perspective

Single Dress

single dress try-on: replace the visible outfit with the selected one-piece dress. the dress must read as one continuous garment from upper body to hem. preserve the person, legs, shoes, hands, and hair where they remain visible

Single Shoes

single footwear try-on: replace only the shoes or sandals. preserve the outfit above the ankles and keep pants, dress, and skin unchanged. both selected shoes must be worn on the correct feet with realistic floor contact and shadows

Single Outerwear

single outerwear try-on: add or replace only the outer layer. keep the inner top visible at neckline, front opening, cuffs, and hem when appropriate. do not replace pants, skirt, dress, shoes, or accessories unless hidden by natural jacket overlap

Single Accessory

single accessory try-on: add only the selected accessory. preserve all clothing and body features while placing the accessory at a natural contact point. keep straps, handles, lenses, belt line, or hardware aligned with the person perspective

Top-and-bottom mode

two-piece outfit try-on: combine the selected upper garment and lower garment as a coherent outfit. resolve the waist overlap naturally with correct tucked, untucked, cropped, or layered behavior. keep the top above the bottom with believable fabric stacking, shadows, and no duplicate waistband. preserve shoes, accessories, face, hands, and hair unless they are naturally occluded

Conditional addition when outerwear is selected:

if an outerwear item is selected, layer it over the top without hiding the top completely

Conditional addition when shoes are selected:

if footwear is selected, make the full outfit read from shoulder to feet with shoes clearly grounded

Full-set mode

complete outfit try-on: coordinate all selected garments into one wearable full look. respect the vertical outfit order from shoulders and torso to waist, legs, feet, and accessories. avoid inventing extra garments; every visible changed fashion item must come from the selected catalog items

Conditional addition for dress:

if a dress is selected, treat it as the main body garment and avoid adding separate pants or skirt unless that item is explicitly selected

Conditional addition for top and bottom:

if top and bottom are selected together, maintain a clean waist transition and consistent proportions between torso and legs

Conditional addition for outerwear:

if outerwear is selected, make it sit above the top or dress with believable sleeve and collar layering

Conditional addition for shoes:

if shoes are selected, show the pair on the feet with correct scale, ground contact, and no floating footwear

Conditional addition for accessory:

if accessories are selected, place them naturally without covering the face or hiding important garment details

D.5 Exact Context Blocks

Keep original background (none)

preserve the original background, lighting, camera angle, and room details. make only the clothing change look natural in the existing photo

Work

background edit is required: replace the entire original background with one coherent new environment. keep the source person, facial identity, hair, pose, body proportions, selected garments, crop, camera angle, and subject scale unchanged while changing the setting. a contemporary professional office with glass partitions, light wood desks, subtle green plants, and large windows. soft diffused daylight, uncluttered business environment, realistic depth, polished workday atmosphere. rebuild the environment continuously behind and around the person; do not retain recognizable parts of the old background. match the new background perspective, ground plane, depth of field, shadows, color temperature, and light direction to the subject. blend cleanly around hair, hands, garments, and feet with no cutout halo or pasted-on appearance

Casual

background edit is required: replace the entire original background with one coherent new environment. keep the source person, facial identity, hair, pose, body proportions, selected garments, crop, camera angle, and subject scale unchanged while changing the setting. a clean urban pedestrian street with modern storefronts, a few trees, and subtle city depth. natural daytime light, relaxed everyday atmosphere, realistic street perspective, softly blurred distant pedestrians. rebuild the environment continuously behind and around the person; do not retain recognizable parts of the old background. match the new background perspective, ground plane, depth of field, shadows, color temperature, and light direction to the subject

ject. blend cleanly around hair, hands, garments, and feet with no cutout halo or pasted-on appearance

Party

background edit is required: replace the entire original background with one coherent new environment. keep the source person, facial identity, hair, pose, body proportions, selected garments, crop, camera angle, and subject scale unchanged while changing the setting. an upscale rooftop evening reception with elegant decor, warm decorative lights, and a distant city skyline. tasteful golden-hour-to-evening lighting, refined event atmosphere, soft background bokeh, realistic floor and depth. rebuild the environment continuously behind and around the person; do not retain recognizable parts of the old background. match the new background perspective, ground plane, depth of field, shadows, color temperature, and light direction to the subject. blend cleanly around hair, hands, garments, and feet with no cutout halo or pasted-on appearance

Travel

background edit is required: replace the entire original background with one coherent new environment. keep the source person, facial identity, hair, pose, body proportions, selected garments, crop, camera angle, and subject scale unchanged while changing the setting. a scenic seaside promenade with open sky, distant water, tasteful railings, and travel-destination depth. bright airy daylight, natural outdoor shadows, relaxed vacation atmosphere, realistic horizon and perspective. rebuild the environment continuously behind and around the person; do not retain recognizable parts of the old background. match the new background perspective, ground plane, depth of field, shadows, color temperature, and light direction to the subject. blend cleanly around hair, hands, garments, and feet with no cutout halo or pasted-on appearance

Sport

background edit is required: replace the entire original background with one coherent new environment. keep the source person, facial identity, hair, pose, body proportions, selected garments, crop, camera angle, and subject scale unchanged while changing the setting. a modern fitness studio with clean training equipment, open floor space, and large daylight windows. fresh athletic lighting, energetic but realistic environment, subtle equipment depth, clean sporty atmosphere. rebuild the environment continuously behind and around the person; do not retain recognizable parts of the old background. match the new background perspective, ground plane, depth of field, shadows, color temperature, and light direction to the subject. blend cleanly around hair, hands, garments, and feet with no cutout halo or pasted-on appearance

Date

background edit is required: replace the entire original background with one coherent new environment. keep the source person, facial identity, hair, pose, body proportions, selected garments, crop, camera angle, and subject scale unchanged while changing the setting. a cozy upscale cafe with warm pendant lights, tasteful tables, plants, and softly blurred interior details. flattering warm light, intimate but natural atmosphere, realistic indoor depth, elegant approachable mood. rebuild the environment continuously behind and around the person; do not retain recognizable parts of the old background. match the new background perspective, ground plane, depth of field, shadows, color temperature, and light direction to the subject. blend cleanly around hair, hands, garments, and feet with no cutout halo or pasted-on appearance

Custom

background edit is required: replace the entire original background with one coherent new environment. keep the source person, facial identity, hair, pose, body proportions, selected garments, crop, camera angle, and subject scale unchanged while changing the setting. scene requested by user: {NORMALIZED_CONTEXT_PROMPT}. rebuild the environment continuously behind and around the person; do not retain recognizable parts of the old background. match the new background perspective, ground plane, depth of field, shadows, color temperature, and light direction to the subject. blend cleanly around hair, hands, garments, and feet with no cutout halo or pasted-on appearance

D.6 Exact Garment-Fidelity and Realism Blocks

GARMENT SOURCE OF TRUTH: the selected catalog garment images are the mandatory and exclusive source for every wardrobe item in the result. never use preset, scene, or custom prompt text to invent, replace, redesign, recolor, omit, or add clothing, footwear, or accessories. faithfully transfer garment type, color, fabric texture, pattern, print placement, silhouette, and visible details. respect seams, buttons, zippers, pockets, collars, cuffs, waistbands, hems, pleats, shoe soles, logos already present on the catalog item, and accessory hardware. keep selected catalog items recognizable while adapting them to the body perspective. when the catalog image contains a model or extra clothes, use only the selected garment role and ignore unrelated garments, body parts, props, hangers, labels, and background. if preset, scene, or custom prompt text conflicts with a selected garment image, ignore the clothing instruction and follow the selected garment image. photorealistic fashion ecommerce result. natural fabric drape, realistic folds and wrinkles, correct scale, clean garment edges. accurate shadows, highlights, occlusion, depth, and contact points. sharp but natural image, balanced exposure, realistic skin and fabric interaction

D.7 Exact Negative Prompt

Base negative list

nudity, underwear-only result, explicit content, suggestive pose, childlike body, violence, blood, weapon, face swap, changed identity, changed body shape, changed skin tone, extra limbs, missing limbs, malformed limbs, distorted hands, distorted fingers, distorted face, wrong garment color, wrong garment type, missing selected garment, unselected garment replacement, duplicated clothing, mismatched clothing layers, floating garment, garment detached from body, duplicated shoes, mismatched shoes, single shoe only, floating shoes, shoes not on feet, warped fabric, melted garment, broken seams, unnatural folds, bad occlusion, jagged mask edge, text overlay, watermark, fake logo, logo hallucination, low quality, blurry, overexposed, underexposed

Role-specific additions

top: wrong neckline, missing sleeves, duplicated collar, shirt fused into skin, changed pants or shoes when only top is selected

bottom: wrong waistband, missing legs, duplicated pants, skirt turned into pants, changed shirt or shoes when only bottom is selected

dress: dress split into separate top and bottom, pants added under dress unless selected, broken waistline, unnatural skirt fall

shoes: bare feet, socks only, missing footwear, shoes on wrong feet, extra pair of shoes, distorted soles, shoes floating above ground

outerwear: jacket fused with inner shirt, missing lapels, impossible sleeve layering, outerwear under the shirt, changed bottom when only outerwear is selected

accessory: oversized accessory, tiny accessory, accessory floating, strap disconnected, accessory covering face or hiding selected garments

D.8 Fully Expanded Example: Full-Body Top + Bottom, Casual Context

The following is the effective prompt for the current checked-in workflow. Only catalog values inside braces are replaced at runtime.

Create one single 2x2 grid image for virtual fashion try-on.

The full returned image must be a vertical 3:4 portrait canvas, so each cropped grid cell is also a vertical 3:4 portrait.

The grid order is fixed: cell 1 is top-left, cell 2 is top-right, cell 3 is bottom-left, and cell 4 is bottom-right.

Cell 1 is the baseline try-on: apply the selected outfit while preserving the source person pose, expression, camera angle, perspective, crop, and body alignment as closely as possible.

Cells 2, 3, and 4 may each use a different subtle, natural fashion pose, but must keep the same person identity, body shape, body proportions, skin tone, visible body coverage, camera framing, and complete selected outfit as cell 1.

Do not turn pose variation into a different person, a body transformation, a wardrobe variation, or a different crop.

Each of the four cells should show the same full-body framing as the source person wearing the selected outfit, with slight styling variation and grounded feet.

Use reference image 1 as the only source for the visible person identity cues, face if visible, hair if visible, expression if visible, pose, body shape, body proportions, height cues, shoulder width, waist if visible, legs if visible, hands if visible, feet if visible, and skin tone.

Never copy or blend in the face, body, pose, age, gender presentation, skin tone, background, or other garments from catalog garment reference images.

GARMENT SOURCE OF TRUTH: every selected garment or accessory must come from the garment reference images after reference image 1. Those garment images, not the text prompt, are the mandatory and exclusive wardrobe source.

Reproduce the selected garments faithfully: keep the same garment type, color, pattern, print, logo already present, fabric appearance, silhouette, neckline, sleeves, closures, seams, pockets, waistband, hem, hardware, footwear shape, and other visible construction details.

Never invent, redesign, replace, recolor, omit, combine into a different item, or add any fashion item that is not present in the selected garment references.

Treat all custom or user-written prompt text as scene context only: it may affect background, lighting, mood, occasion, and explicitly permitted pose variation, but it must not change the wardrobe. If text conflicts with a garment reference, ignore the conflicting wardrobe instruction and follow the garment reference.

Do not add visible borders, gutters, labels, captions, watermarks, or extra text between grid cells.

Reference image 1 is the person photo and controls the person appearance.

Reference image 2: top – {TOP_NAME}, {TOP_COLOR}.

Reference image 3: bottom – {BOTTOM_NAME}, {BOTTOM_COLOR}.

virtual fashion try-on for the person or visible body part in the source image. use the source person image as the only reference for visible identity cues, hair, expression, pose, body shape, body proportions, and skin tone. preserve the original camera framing, crop, perspective, and visible body coverage from the source image. do not expand a cropped source image into a full-body image, and do not invent unseen face, torso, legs, feet, or hands. do not copy or infer any face, body shape, pose, age, gender presentation, or skin tone from c

atalog garment images or garment models. preserve visible hands, fingers, neck, legs, feet, and body boundaries unless covered by selected garments. replace or overlay only the selected fashion items realistically within the visible crop. source image supports full-body try-on; keep the full body visible with the same framing and ground contact. selected garment s: upper-body garment such as shirt, blouse, polo, sweater, or t-shirt, {TOP_NAME}, color {TOP_COLOR}, size {TOP_SIZE}, align neckline, shoulders, sleeves, chest fit, armholes, side seams, tucked or untucked hem, and visible print placement; lower-body garment such as pants, jeans, skirt, or shorts, {BOTTOM_NAME}, color {BOTTOM_COLOR}, size {BOTTOM_SIZE}, align waistband, belt loops, hips, rise, crotch area, leg shape, inseam, cuffs, and hem length. two-piece outfit try-on: combine the selected upper garment and lower garment as a coherent outfit. resolve the waist overlap naturally with correct tucked, untucked, cropped, or layered behavior. keep the top above the bottom with believable fabric stacking, shadows, and no duplicate waistband. preserve shoes, accessories, face, hands, and hair unless they are naturally occluded. align neckline, shoulders, sleeves, chest fit, armholes, side seams, tucked or untucked hem, and visible print placement. align waistband, belt loops, hips, rise, crotch area, leg shape, inseam, cuffs, and hem length. background edit is required: replace the entire original background with one coherent new environment. keep the source person, facial identity, hair, pose, body proportions, selected garments, crop, camera angle, and subject scale unchanged while changing the setting. a clean urban pedestrian street with modern storefronts, a few trees, and subtle city depth. natural daytime light, relaxed everyday atmosphere, realistic street perspective, softly blurred distant pedestrians. rebuild the environment continuously behind and around the person; do not retain recognizable parts of the old background. match the new background perspective, ground plane, depth of field, shadows, color temperature, and light direction to the subject. blend cleanly around hair, hands, garments, and feet with no cutout halo or pasted-on appearance. GARMENT SOURCE OF TRUTH: the selected catalog garment images are the mandatory and exclusive source for every wardrobe item in the result. never use preset, scene, or custom prompt text to invent, replace, redesign, recolor, omit, or add clothing, footwear, or accessories. faithfully transfer garment type, color, fabric texture, pattern, print placement, silhouette, and visible details. respect seams, buttons, zippers, pockets, collars, cuffs, waistbands, hems, pleats, shoe soles, logos already present on the catalog item, and accessory hardware. keep selected catalog items recognizable while adapting them to the body perspective. when the catalog image contains a model or extra clothes, use only the selected garment role and ignore unrelated garments, body parts, props, hangers, labels, and background. if preset, scene, or custom prompt text conflicts with a selected garment image, ignore the clothing instruction and follow the selected garment image. photorealistic fashion ecommerce result. natural fabric drape, realistic folds and wrinkles, correct scale, clean garment edges. accurate shadows, high lights, occlusion, depth, and contact points. sharp but natural image, balanced exposure, realistic skin and fabric interaction

STRICT AVOID LIST – none of the following outcomes may appear in the result:

nudity, underwear-only result, explicit content, suggestive pose, childlike body, violence, blood, weapon, face swap, changed identity, changed body shape, changed skin tone, extra limbs, missing limbs, malformed limbs, distorted hands, distorted fingers, distorted face, wrong garment color, wrong garment type, missing selected garment, unselected garment replacement, duplicated clothing, mismatched clothing layers, floating garment, garment detached from body, duplicated shoes, mismatched shoes, single shoe only, floating shoes, shoes not on feet, warped fabric, melted garment, broken seams, unnatural folds, bad occlusion, jagged mask edge, text overlay, watermark, fake logo, logo hallucination, low quality, blurry, overexposed, underexposed, wrong neckline, missing sleeves, duplicated collar, shirt fused into skin, changed pants or shoes when only top is selected, wrong waistband, missing legs, duplicated pants, skirt turned into pants, changed shirt or shoes when only bottom is selected

D.9 Exact Video Prompt

Positive video prompt template

Use the input image as the exact first frame of one continuous {DURATION_SECONDS}-second photorealistic fashion shot at normal real-time speed. begin almost still, ease smoothly into one simple movement, and gently settle into a balanced final pose; use natural acceleration and deceleration with no abrupt stop. {MOTION_BY_CONTEXT_PRESET}. keep the motion anatomically connected: planted feet and hips initiate the weight transfer, followed by the torso, shoulders, head, and relaxed visible arms and hands. keep facial animation minimal: eyes stay

y naturally open with a calm steady gaze and stable eyelids; do not animate blinking; keep lips closed and still. fabric reacts a moment after the body, with subtle inertia, realistic folds, and gentle settling; heavy fabric moves less than loose fabric. locked-off camera, stable perspective and focal length; the camera and background remain still. preserve the person, face, hair, proportions, skin tone, complete outfit, garment details, accessories, shoes, background, lighting, and framing exactly as shown. keep the existing user scene context: {CONTEXT_PROMPT} [only when supplied]

Motion substitution by preset

none: the person slowly transfers a little weight to one leg while the hips, torso, shoulders, and head follow as one connected movement, then makes a slight three-quarter turn

work: the person gently straightens their posture, shifts weight with quiet confidence, and makes a small composed shoulder turn while the face and gaze remain relaxed and steady

casual: the person takes an easy breath, makes a relaxed side-to-side weight shift, and turns slightly as if naturally showing the outfit to a friend

party: the person eases into one understated elegant pose with a small coordinated hip and shoulder turn while maintaining a calm steady gaze toward the camera

travel: the person makes a relaxed sightseeing-style weight shift and slight turn while a faint steady breeze softly lifts only loose hair and fabric edges

sport: the person makes one small controlled athletic weight shift with softly flexing knees, coordinated hips and shoulders, and both feet remaining planted

date: the person takes a soft breath and makes a gentle pose change with a slight head tilt while keeping a soft expression and steady gaze

custom: the person slowly transfers a little weight to one leg while the hips, torso, shoulders, and head follow as one connected movement, then makes a slight three-quarter turn

Negative video prompt

identity, face, body shape, or skin tone change, wardrobe, garment color, pattern, print, logo, accessory, or shoe change, missing or extra garment, fabric melting, texture morphing, robotic motion, stiff or frozen pose, mechanical turn, unnatural rhythm, abrupt start or stop, sudden acceleration, repeated or looping gesture, foot sliding, floating feet, body gliding, lost balance, disconnected arm motion, rubber limbs, twitching hands, fused fingers, frequent or repeated blinking, rapid blinking, eyelid flutter, eyelid twitching, half-closed eyes, darting gaze, talking, lip movement, mouth opening, exaggerated expression, flicker, jitter, temporal inconsistency, warping, duplicated limbs, extra fingers, deformed hands or face, camera cut, transition, speed ramp, slow motion, camera shake or drift, pan, tilt, orbit, dolly, zoom, changing crop, moving or replaced background, lighting change, text, watermark, nudity, explicit content

APPENDIX E. RECOMMENDATION DATA, BASELINES, AND EVALUATION EQUATIONS

E.1 Datasets and Reproducibility Links

The reported evaluation uses only the controlled subsets described in Section 4.7. Raw files remain outside Git; checksums and retrieval metadata are recorded in `evaluation/recommendation/results/dataset-provenance.json`.

Amazon Reviews 2023 (Amazon Fashion) [22]

Polyvore Dataset (Bi-LSTM compatibility/FITB) [23]

The three notebooks below contain the complete preprocessing assumptions, grid-search audit, tables, and the three charts presented in Section 4.7:

Home recommendation evaluation: [Open notebook on GitHub](#)

Product Detail recommendation evaluation: [Open notebook on GitHub](#)

Cart recommendation evaluation: [Open notebook on GitHub](#)

* *Our github:* <https://github.com/bloodthirster2510/fashion-ecommerce-system>

E.2 Traditional Baseline Formulas

Most Popular

The Most Popular baseline ranks items by positive training-interaction counts. This implementation applies the monotonic log normalization in Equation (E.1), which preserves the ordering by count.

Home ranks candidates by positive training-interaction count n_i , normalized by the maximum training count $n_{\max}$:

$$S_{\text{pop}}(i) = \frac{\log(1+n_i)}{\log(1+n_{\max})} \quad (\text{E.1})$$

Cosine similarity

Following content-based recommendation principles [15], this implementation represents products using binary metadata vectors and computes their standard cosine similarity.

Product Detail represents category/role, gender, brand, color, fit, and price bucket as a binary metadata vector [15]:

$$S_{\text{cos}}(q, i) = \frac{x_q \cdot x_i}{\|x_q\| \|x_i\|} \quad (\text{E.2})$$

Association Lift

Cart association statistics are estimated from training baskets $\mathcal{B}$ only:

$$\text{support}(A) = |\{b \in \mathcal{B} : A \in b\}| / |\mathcal{B}| \quad (\text{E.3})$$

$$\text{support}(A,B) = |\{b \in B : A,B \in b\}| / |B| \quad (\text{E.4})$$

$$\text{confidence}(A \rightarrow B) = \text{support}(A,B) / \text{support}(A) \quad (\text{E.5})$$

$$\text{lift}(A \rightarrow B) = \text{support}(A,B) / [\text{support}(A) \cdot \text{support}(B)] \quad (\text{E.6})$$

$$L(c,i) = \log[1 + \max_{a \in c} \text{lift}(a \rightarrow i)] / \log(1 + \text{lift}_{\max, \text{train}}) \quad (\text{E.7})$$

E.3 Evaluation Equations

Let U be the set of N evaluation cases, C_u the fixed candidate set, G_u the held-out relevant set, and $R_u@K$ the ordered Top- K list. Binary relevance is used. The definitions follow standard recommender evaluation practice and match the benchmark implementation.

Interpretation of reported metrics

The suffix $@K$ restricts evaluation to the first K recommendations. $HR@K$ is the proportion of cases whose Top- K list contains at least one relevant item. $\text{Precision}@K$ is the relevant fraction of the Top- K list, whereas $\text{Recall}@K$ is the fraction of all relevant items recovered within that list.

$\text{MAP}@K$ rewards lists that place all relevant items early, while $\text{MRR}@K$ focuses on the position of the first relevant item. $\text{NDCG}@K$ evaluates the full ranking order with larger rewards for relevant items near the top and normalizes the result against the ideal ranking.

$\text{FITB}@1$ is the proportion of Polyvore fill-in-the-blank cases in which the correct missing garment is ranked first. Expanded $\text{NDCG}@5$ uses the $\text{NDCG}@5$ definition for the harder Cart evaluation with 200 candidates rather than the official four-choice FITB set.

$\text{Coverage}@K$ measures how much of the candidate catalog appears across all Top- K lists. $\text{CategoryDiversity}@K$ and $\text{BrandDiversity}@K$ measure the mean fraction of distinct categories and brands within each list. All reported metrics lie between 0 and 1, and larger values are preferable, although coverage and diversity must be interpreted alongside ranking relevance.

Binary relevance and Hit Rate

$$\text{rel}_{u,r} = 1(i_{u,r} \in G_u) \quad (\text{E.8})$$

$$\text{Hit}_u@K = 1(R_u@K \cap G_u \neq \emptyset) \quad (\text{E.9})$$

$$\text{HR}@K = (1/N) \sum_{u=1}^N \text{Hit}_u@K \quad (\text{E.10})$$

Precision and Recall

$$\text{Precision}_u@K = |R_u@K \cap G_u| / K \quad (\text{E.11})$$

$$\text{Recall}_u@K = |R_u@K \cap G_u| / |G_u| \quad (\text{E.12})$$

$$\text{Precision}@K = (1/N) \sum_{u=1}^N \text{Precision}_u@K \quad (\text{E.13})$$

$$\text{Recall@K} = (1/N) \sum_{u=1}^N \text{Recall}_u@K \quad (\text{E.14})$$

Average Precision and MAP

$$P_u@r = (1/r) \sum_{j=1}^r \text{rel}_{u,j} \quad (\text{E.15})$$

$$\text{AP}_u@K = [\sum_{r=1}^K (P_u@r \cdot \text{rel}_{u,r})] / \min(|G_u|, K) \quad (\text{E.16})$$

$$\text{MAP@K} = (1/N) \sum_{u=1}^N \text{AP}_u@K \quad (\text{E.17})$$

Reciprocal Rank and MRR

$$\text{RR}_u@K = 1(r_u^* \leq K) \cdot (1/r_u^*) \quad (\text{E.18})$$

$$\text{MRR@K} = (1/N) \sum_{u=1}^N \text{RR}_u@K \quad (\text{E.19})$$

DCG and NDCG

$$\text{DCG}_u@K = \sum_{r=1}^K \text{rel}_{u,r} / \log_2(r+1) \quad (\text{E.20})$$

$$\text{IDCG}_u@K = \sum_{r=1}^m 1 / \log_2(r+1), m = \min(|G_u|, K) \quad (\text{E.21})$$

$$\text{NDCG@K} = (1/N) \sum_{u=1}^N [\text{DCG}_u@K / \text{IDCG}_u@K] \quad (\text{E.22})$$

Catalog coverage and list diversity

$$\text{Coverage@K} = |U_u R_u@K| / |U_u C_u| \quad (\text{E.23})$$

$$\text{CategoryDiversity@K} = (1/N) \sum_{u=1}^N |\text{categories}(R_u@K)| / |R_u@K| \quad (\text{E.24})$$

$$\text{BrandDiversity@K} = (1/N) \sum_{u=1}^N |\text{brands}(R_u@K)| / |R_u@K| \quad (\text{E.25})$$

E.4 Auxiliary Paired Significance Test

For the secondary Polyvore analysis, the two-sided exact McNemar test uses b cases hit only by the project method and c cases hit only by the baseline. Only discordant cases contribute:

$$p = \min\{1, 2 \sum_{j=0}^{\min(b,c)} [C(b+c, j) / 2^{b+c}]\} \quad (\text{E.26})$$

A non-significant p -value does not prove equivalence; it means that the available paired sample does not provide sufficient evidence to reject equal discordant-hit probabilities.

E.5 Recommendation Symbol Glossary

Table E.1 summarizes the notation used in scoring, association, evaluation, and significance testing.

Table E.1 Recommendation symbols and notation

Symbol / Metric	Meaning and Use
Ranking entities and signals	
u	User or offline evaluation case.
i	Candidate product being scored or ranked.
q	Query or anchor product on the Product Detail page.
c	Current shopping-cart context used to recommend complementary items.
a_i	Action type of interaction event i , such as view, click, favorite, try-on, add-to-cart, or purchase.
$\omega(a_i)$	Base importance assigned to action type a_i .
Δt_i	Age of interaction event i , measured in days.
$d(\Delta t_i)$	Recency multiplier applied to an interaction according to its age.
w_i	Effective time-aware weight of interaction event i : $\omega(a_i)d(\Delta t_i)$.
$P(u,i)$	Preference match between user u and product i .
$A(q,i)$	Weighted attribute similarity between anchor product q and candidate i .
$C(q,i)$	Cosine similarity between metadata vectors of q and i .
$R(c,i)$	Fashion-role compatibility of product i with the roles already present in cart c .
$T(c,i)$	Style compatibility of product i with the items in cart c .
$L(c,i)$	Normalized association-lift signal between candidate i and items in cart c .
$B(i)$	Business score of product i , derived from sale and new-arrival flags.
$S_{\text{home}}(u,i)$	Final deployed Home score for product i in user context u .
$S_{\text{detail}}(q,i)$	Final deployed Product Detail score for candidate i relative to anchor q .
$S_{\text{cart}}(c,i)$	Final deployed Shopping Cart score for candidate i in cart context c .
$n_i; n_{\text{max}}$	Positive training-interaction count for product i and the maximum such count.
$x_q; x_i$	Binary metadata vectors of anchor product q and candidate product i .
Association statistics	
$\mathcal{B}$	Set of training baskets used to estimate product associations.
$b \in \mathcal{B}$	One basket in the training-basket set.
A, B	Item sets used in support, confidence, and lift definitions; distinct from business score $B(i)$.
$\text{support}(A)$	Proportion of training baskets containing item set A .
$\text{support}(A,B)$	Proportion of training baskets containing both A and B .
$\text{confidence}(A \rightarrow B)$	Conditional association strength $\text{support}(A,B)/\text{support}(A)$.

Symbol / Metric	Meaning and Use
$\text{lift}(A \rightarrow B)$	Association strength normalized by the marginal support of B.
$\text{lift}_{\max, \text{train}}$	Maximum lift observed in training data, used to normalize $L(c, i)$.
Offline evaluation notation and metrics	
$U; N$	Set of evaluation cases and its size $N = U $.
C_u	Fixed candidate set for evaluation case u .
G_u	Held-out relevant-item set for evaluation case u .
$R_u@K$	Ordered list of the Top-K recommendations returned for case u .
K	Recommendation cut-off, such as $K = 5$ or $K = 10$.
i_{ur}	Item ranked at position r for evaluation case u .
rel_{ur}	Binary relevance of the item at rank r for case u .
$\text{Hit}_u@K$	Indicator that at least one relevant item occurs in $R_u@K$.
$\text{HR}@K$	Hit Rate: mean of $\text{Hit}_u@K$ over all evaluation cases.
$\text{Precision}_u@K$	Fraction of the Top-K list that is relevant for case u .
$\text{Recall}_u@K$	Fraction of relevant items retrieved in the Top-K list for case u .
$\text{Precision}@K; \text{Recall}@K$	Mean per-case precision and recall at cut-off K .
$P_u@r$	Precision calculated through rank r for evaluation case u .
$\text{AP}_u@K$	Average Precision at K for evaluation case u .
$\text{MAP}@K$	Mean Average Precision at K over all evaluation cases.
r_u^*	Rank of the first relevant item for case u .
$\text{RR}_u@K$	Reciprocal Rank at K for evaluation case u .
$\text{MRR}@K$	Mean Reciprocal Rank at K over all evaluation cases.
$\text{DCG}_u@K$	Discounted Cumulative Gain at K for case u .
$\text{IDCG}_u@K$	Ideal Discounted Cumulative Gain at K for case u .
$\text{NDCG}@K$	Mean normalized DCG at K ; the primary grid-search objective is $\text{NDCG}@5$.
$\text{Coverage}@K$	Fraction of the candidate catalog exposed by all Top-K recommendation lists.
$\text{CategoryDiversity}@K$	Mean fraction of distinct categories within each Top-K list.
$\text{BrandDiversity}@K$	Mean fraction of distinct brands within each Top-K list.
$\text{FITB}@1$	Fill-in-the-Blank accuracy at rank 1 for Polyvore outfit completion.
Paired significance test	
b	Number of paired cases hit only by the project method in the McNemar test.
c	Number of paired cases hit only by the baseline in the McNemar test.
j	Summation index in the exact binomial form of the McNemar test.
p	Two-sided exact McNemar p-value.